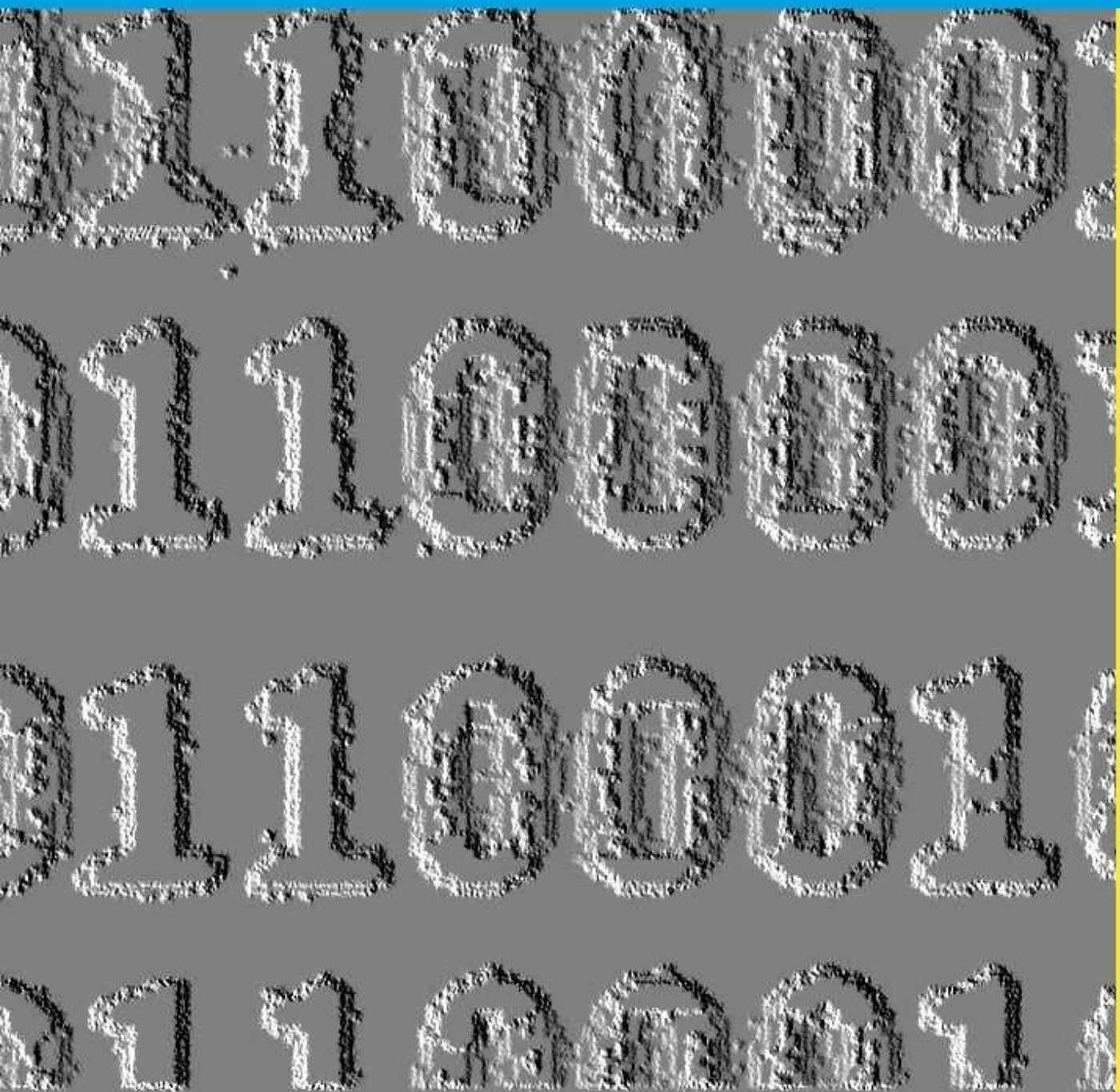
BINARY, OCTAL, AND HEXADECIMAL ARITHMETIC FOR MICROCOMPUTERS



WILLIAM BARDEN

Binary, Octal, and Hexadecimal Arithmetic in Microcomputers

William Barden

Turing made it look easy.

Binary, Octal, and Hexadecimal Arithmetic in Microcomputers

TABLE OF CONTENTS

BINARY, OCTAL, AND HEXADECIMAL ARITHMETIC

IN MICROCOMPUTERS

WILLIAM BARDEN

BINARY, OCTAL, AND HEXADECIMAL ARITHMETIC

IN MICROCOMPUTERS

PREFACE

CHAPTER 1

THE BINARY SYSTEM-WHERE IT ALL BEGINS

BIG ED LEARNS BINARY

MORE ON BITS, BYTES, AND BINARY

CONVERTING FROM BINARY TO DECIMAL

CONVERTING FROM DECIMAL TO BINARY

PADDING OUT TO EIGHT OR SIXTEEN BITS

EXERCISES

CHAPTER 2

OCTAL, HEXADECIMAL, AND OTHER NUMBER BASES

CHILI IS BIG ON REGULUS

HEXADECIMAL

Converting Between Hex and Decimal

Converting From Decimal to Hexadecimal

OCTAL
Converting Between Binary and Octal
Converting Between Octal and Decimal

WORKING IN OTHER NUMBER BASES

STANDARD CONVENTIONS

EXERCISES

CHAPTER 3

SIGNED NUMBERS AND TWO'S COMPLEMENT NOTATION

BIG ED AND THE BINACUS

ADDING AND SUBTRACTING

BINARY NUMBERS

TWO'S COMPLEMENT REPRESENTATION

SIGN EXTENSION

ADDING AND SUBTRACTING

TWO'S COMPLEMENT NUMBERS

EXERCISES

CHAPTER 4

CARRIES, OVERFLOW, AND FLAGS

THIS RESTAURANT HOLDS +127

PEOPLE. AVOID OVERFLOW!

OVERFLOW

CARRY

OTHER FLAGS

FLAGS IN TYPICAL MICROCOMPUTERS

EXERCISES

CHAPTER 5

LOGICAL OPERATIONS AND SHIFTING

THE BRITISH ENIGMA

LOGICAL OPERATIONS

ORS

ANDS

EXCLUSIVE ORS

OTHER LOGICAL OPERATIONS

SHIFT OPERATIONS
- Rotates
- Logical Shifts
- Arithmetic Shifts

EXERCISES

CHAPTER 6

MULTIPLICATION AND DIVISION

ZELDA LEARNS HOW T0

SHIFT FOR HERSELF

MULTIPLICATION ALGORITHMS
- Successive Addition
- Successive Addition of Powers of Two
- Shift and Add Multiplication

SIGNED VS UNSIGNED MULTIPLICATION

DIVISION ALGORITHMS
- Successive Subtraction
- Restoring Division
- Signed Vs. Unsigned Divisions

EXERCISES

CHAPTER 7

MULTIPLE PRECISION

IS THE FIBONACCI SERIES RELATED

TO ITALIANSOCCER?

ADDITION AND SUBTRACTION

USING MULTIPLE PRECISION

MULTIPLE-PRECISION MULTIPLICATION

EXERCISES

CHAPTER 8

FRACTIONS AND SCALING

BIG ED WEIGHS THE NUMBERS

FRACTIONS IN BINARY
- Working with Fractions in Binary
- Keeping a Separate Fraction
- Scaling

CONVERSION OF DATA

EXERCISES

CHAPTER 9

ASCII CONVERSIONS

BIG ED AND THE INVENTOR

ASCII CODES

CONVERSION FROM ASCII TO

BINARY INTEGERS

CONVERSION FROM ASCII TO

BINARY FRACTIONS

CONVERSION FROM BINARY

INTEGERS TO ASCII

CONVERSION FROM BINARY

FRACTIONS TO ASCII

EXERCISES

CHAPTER 10

FLOATING-POINT NUMBERS

… AND 3000 COMBINATION PLATES

FOR THE MOTHER SHIP . . .

SCIENTIFIC NOTATION VS.

FLOATING POINT

USING POWERS OF TWO IN PLACE OF

POWERS OF TEN

DOUBLE-PRECISION FLOATING-POINT

NUMBERS

CALCULATIONS USING BINARY

FLOATING-POINT NUMBERS

EXERCISES

APPENDIX A

ANSWERS TO EXERCISES

CHAPTER 1

CHAPTER 2

CHAPTER 3

CHAPTER 4

CHAPTER 5

CHAPTER 6

CHAPTER 7

CHAPTER 8

CHAPTER 9

CHAPTER 10

APPENDIX B

GLOSSARY OF TERMS

APPENDIX C

BINARY, DECIMAL, OCTAL, HEXADECIMAL TABLE

APPENDIX D

TWO'S COMPLEMENT/DECIMAL TABLE

OTHER RECENT BOOKS BY WILLIAM BARDEN

Preface

It wasn’t that long ago that a purchaser of a microcomputer system such as a Commodore, Radio Shack, or Apple encountered ominous references to "binary numbers," "hexadecimal values", "ANDing two numbers to get the result," or "shifting the result to multiply by two." Sometimes these references assumed that the reader knew the binary system and operations; other times one gets the distinct feeling that the writer of the reference manual didn’t really know all that much about the operations either.

The original purpose of this book was to remove some of the mystery surrounding the specialized math operations that are used in various assembly languages and even higher level computer languages such as BASIC and C. These explanations are still valid today, if one does assembly-language programming on microcontrollers such as the Arduino and Microchip (Pic) series or newer versions of the Intel 8086 family or JavaScript. Even such applications as Microsoft Excel are based upon low–level binary operations such as adding floating-point numbers, although the details may be one or two levels down.

Such operations as binary number representation, octal and hexadecimal representation, two’s complement operation, addition and subtraction of binary numbers, “flag" bits in microcomputers, logical operations and shifting,

multiplication and division algorithms, multiple-precision operations, fractions and scaling, and floating-point operations form a good basis for understanding all types of computers in general, and especially for understanding embedded microcontrollers or experimenters systems.

This book explains binary, octal, hexadecimal and other microcomputer math operations in detail, and provides practical examples and exercises for self-testing. If you can add, subtract, multiply, and divide decimal numbers, then you can easily perform the same operations in binary and other number bases, such as hexadecimal. This book will show you how. This book makes an excellent companion to any course in assembly language or advanced computer languages.

The book is organized into ten chapters. Most of the chapters are based upon material contained in preceding chapters. Each chapter has self-test exercises at the end. It is to your benefit to go through the exercises, as they help fix the material in your mind, but we won't hold it against you if you use the book for reference only. As you read, you'll notice some words are set in boldface. These are computer terms that are defined in the Glossary. Using the Glossary, even the complete novice can easily learn and profit from the book.

The book is arranged as follows: Chapter I discusses the binary system from the ground up and covers the conversions between binary and decimal numbers, while Chapter 2 describes octal and hexadecimal numbers, and conversions between these "bases" and decimal numbers. Hexadecimal numbers are used in both assembly language and higher-level languages. ‘

Signed numbers and two's complement notation are covered in Chapter 3. Two's complement notation is used for negative numbers.

Chapter 4 discusses carries, overflow, and flags. These topics are used mostly in machine language or assembly-language programming, but can be important in special high-level language programs.

Logical operations, such as ANDs, ORs, and NOTs, are described in Chapter 5, along with the types of shifting possible in assembly language.

Chapter 6 discusses multiplication and division algorithms, including both "unsigned" and "signed" operations.

Chapter 7 describes multiple-precision operations. Multiple precision may

be used in both BASIC and assembly language to implement “unlimited precision" of any number of digits.

Chapter 8 covers binary fractions and scaling. This material is necessary to understand the internal format of floating-point numbers in higher level languages or applications.

Next, ASCII codes and ASCII conversions as they relate to numeric quantities are described in Chapter 9.

Finally, Chapter 10 provides an explanation of floating-point number representation as these numbers are used in many higher-level languages.

In the last section of the book, Appendix A contains the answers to the self-test questions, Appendix B is a glossary of terms, and Appendix C contains a listing of binary octal, decimal, and hexadecimal numbers from 0 through 256. The listing may be used for conversion from one type of number to the other. Finally, Appendix D contains a listing of two's complement numbers from -1 to -128, a handy reference that is not usually found in other texts.

William Barden, 2015

CHAPTER 1

The Binary System-Where It All Begins

ln the binary system, all numbers are represented by an "on/off” condition. Let's look at a quick example of binary in easy-to-understand terms.

BIG ED LEARNS BINARY

”Big Ed" Hackenbyte owns "Big Ed's," a restaurant that serves quick lunches and slow dinners in the middle of the area near San Jose, CA. This area, known as "Silicon Valley," has dozens of companies manufacturing microprocessor components for microcomputers. Ed has eight serving people - Zelda, Olive, Trudy, Thelma, Fern, Fran, Selma, and Sidney. Depersonalization being what it is, they are assigned numbers for payroll. The numbers assigned are:

Zelda 0
Olive 1
Trudy 2
Thelma 3
Fern 4
Fran 5
Selma 6
Sidney 7

When Big Ed first implemented a "call board," he had eight lights, one for each of the serving people, as shown in Fig. 1-1.

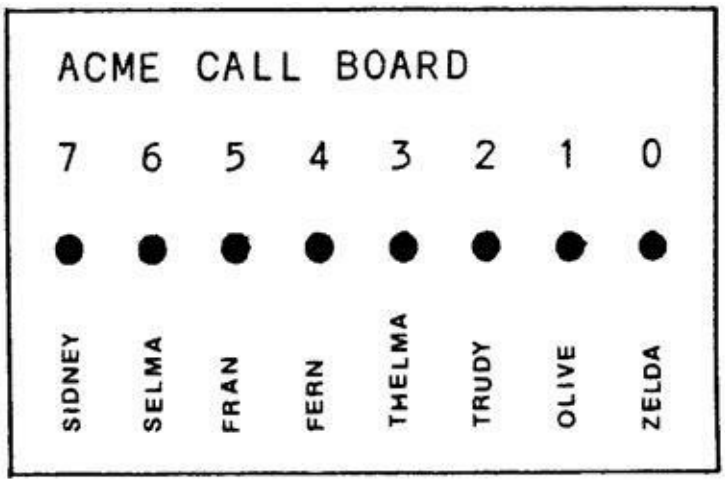


Figure 1-1. Big Ed’s Call Board

One day, though, Bob Borrow, a design engineer at a microprocessor components company known as Inlog, called Ed over.

"Ed, you could be a lot more efficient with your call board. l can show you

how we would have designed the board with binary.”

Ed, being interested in the new technology, was receptive. The redesigned call board is shown in Fig. 1-2. It has three lights, controlled from the kitchen. When a serving person is called, a bell sounds. How is it possible to call any one of the eight serving people by lighting combinations of the three lights?

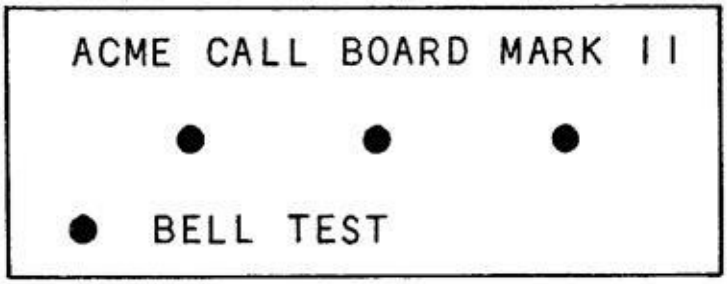


Figure 1-2. A Binary Call Board

"You see, Ed, this board is very efficient. It uses five fewer lights than your first call board. There are eight different combinations of lights. We really call them permutations, as there's a definite order to the light arrangement. I’ve prepared a chart of the permutations of the lights and the serving person called."

He gave Ed the chart shown in Fig. 1-3.

○	○	○	ZELDA	0
○	○	●	OLIVE	1
○	●	○	TRUDY	2
○	●	●	THELMA	3
●	○	○	FERN	4
●	○	●	FRAN	5
●	●	○	SELMA	6
●	●	●	SIDNEY	7

Figure 1-3. Code Chart for the Call Board

“There are only eight different permutations of lights, Ed - no more, no less. These lights are arranged in binary fashion. We use the binary system in our computers for two reasons. First of all, it saves parts. We reduced your number of lights from eight to three. Secondly, inexpensive computer components can usually represent only an on/off state, just as the lights are either on or off.” He paused to sample some of his "Big Edburger."

"I’ll give these codes to my help to memorize,” said Ed.

"Each serving person has only to memorize his unique code, Ed. I'll give you the key, so that you can decode which serving person is being called without reference to that chart. You see, each of the lights represents a power of two. The light on the right represents two to the zero power, the next light is two to the first power, and the leftmost light is two to the second power. This is really very similar to the decimal system, where each digit represents a power of ten."

He scratched an example on the tablecloth, as shown in Figure 1-4.

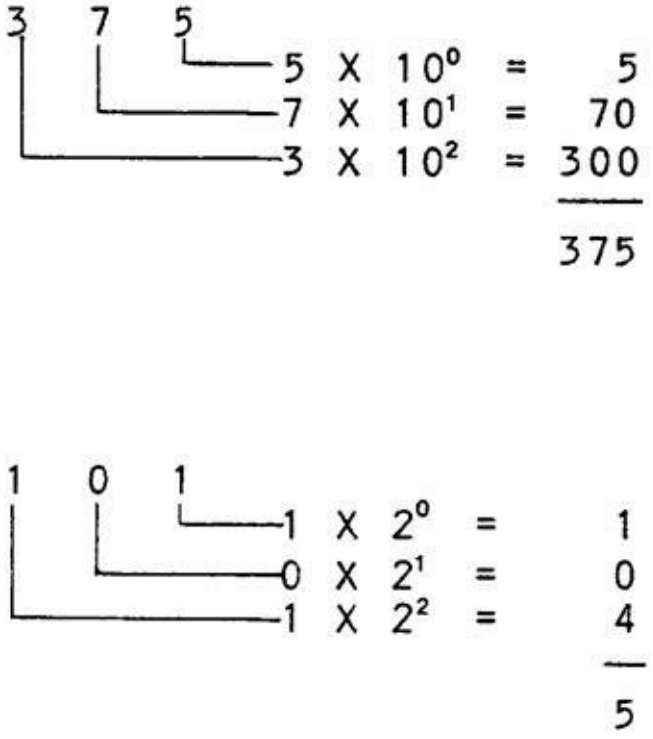


Figure 1-4. A Comparison of Decimal and Binary Notation

"Just as we can carry out the powers of ten to huge numbers, we can use as many powers of two as we want. We could use 32 lights, if we wanted. Now, to convert the three lights in binary to their decimal equivalent, add up the power of two for each light that is on." He scratched another figure on the tablecloth (Fig. 1-5).

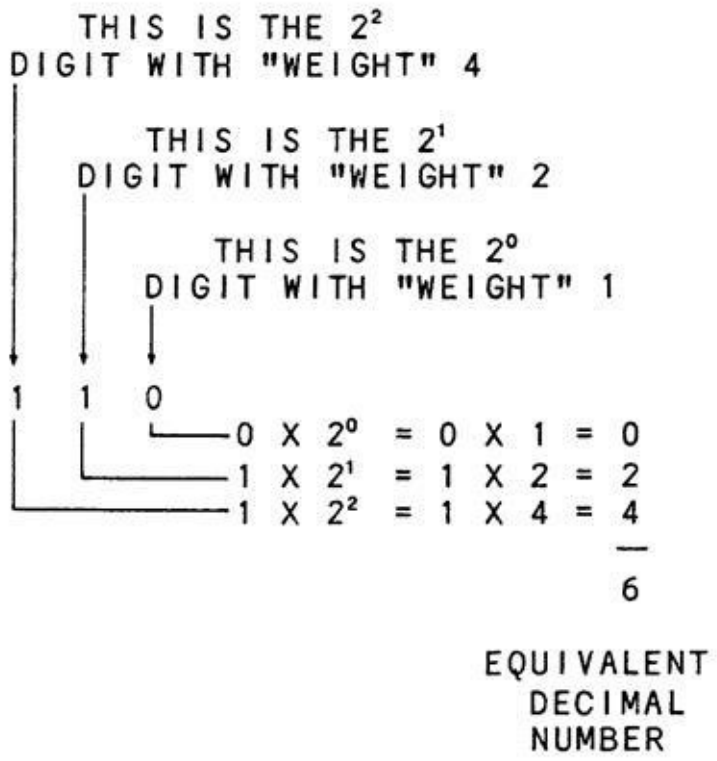


Figure 1-5.Binary-to-Decimal Conversion

"Well, that seems simple enough," Ed admitted. "From right to left, the lights represent 1, 2, and 4. If we had more waiters and waitresses, the lights would represent 1, 2, 4, 8, 16, 32, 64, 128…"

His voice trailed off, as he lost track of the next power of two. "Exactly right, Ed. Typically, we have the equivalent of eight or sixteen lights in our microprocessors. Sometimes even 32. We don't use lights, of course. We use microprocessor components that are either off or on."

He scratched another figure on the tablecloth, which by now was overflowing with diagrams (Fig. 1-6).

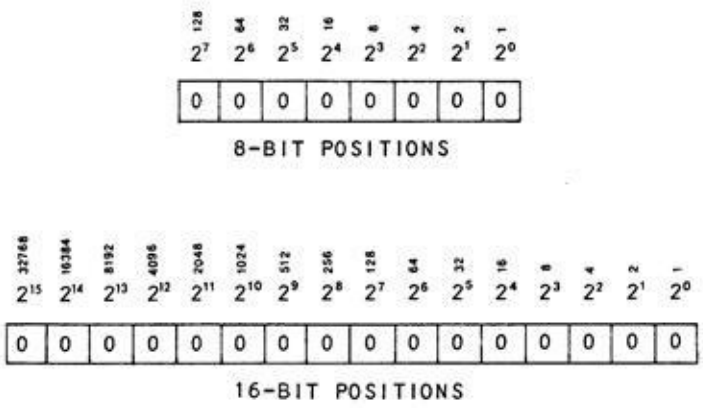


Figure 1-6. 8- and 16-Bit Representation

“We call each of the eight or sixteen positions **bit positions**. 'Bit’ is a contraction of the term 'binary digit.' After all, that's what we’re talking about here, binary digits, just as decimal digits make up a decimal number. Your call board represents a 3-bit number.”

“In eight bits we can represent any number between 0 and 1 + 2+ 4 + 8 + 16 + 32 + 64 + 128. Adding all of those up, we get 255, the largest number that can be held in eight bits."

"How about sixteen bits?" asked Ed.

"You figure it out," said Bob. "I've got to get back to the job of designing microprocessors.”

Ed drew up a list of all the powers of two up to fifteen. He then added them together to come up with the result shown in Fig. 1-7, a total of 65,535-the largest number that can be held in sixteen bits.

“This microprocessor business is easy,” Ed said with a grin, as he bit into his Big Ed's Jumboburger with an 8-bit byte.

WEIGHT	BIT POSITION
1	0
2	1
4	2
8	3
16	4
32	5
64	6
128	7
256	8
512	9
1024	10
2048	11
4096	12
8192	13
16384	14
32768	15
65535	

Figure 1-7. Powers of Two and Bit Positions

MORE ON BITS, BYTES, AND BINARY

Bob Borrow's explanation of binary pretty well capsulizes microcomputer binary representation. The basic unit is a **bit**, or **binary digit**. A bit can be either on or off. Because it's much easier to write a 0 or 1, these digits are used in place of "on" or "off" when we're representing binary values.

The **bit position** of the binary number refers to the position of the bit in the binary value. The bit position in most microcomputers has a number associated with it, as shown in Fig. 1-8. As bit positions really represent a power of two, it's convenient to number them according to the power of two represented. The rightmost bit is two to the zero power, and the bit position is therefore zero. The bit positions moving to the left are numbered 1 (two to the first power), 2 (two to the second power), 3, 4, 5, and so forth. ln all current computers, a collection of eight bits is called a **byte**. It's somewhat obvious to see how this evolved if one imagines early computer engineers having lunch at the equivalent of Big Ed's.

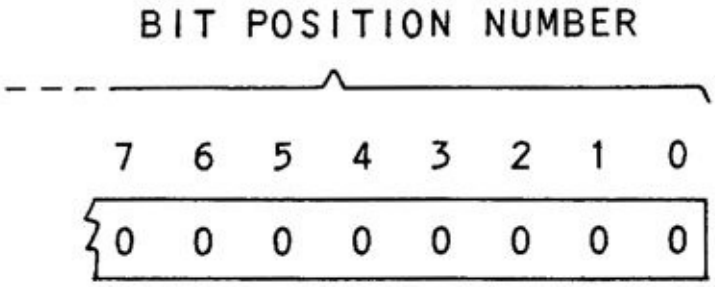


Figure 1-8. Bit Position Numbering

Microcomputer memory is often specified in the number of bytes it represents. Each byte roughly corresponds to one character, as we shall see in later chapters. Operations to and from memory are generally made a byte at a time. Registers in the microprocessor portion of a microcomputer are also one or two bytes in length. Registers are really nothing more than fast-access memory locations that are used for temporary storage in the microprocessor. There are typically dozens of bytes of register storage in the microprocessor and from 65,535 bytes to gigabytes in the memory of a microcomputer, as illustrated in Figs. 1-9 and 1-10.

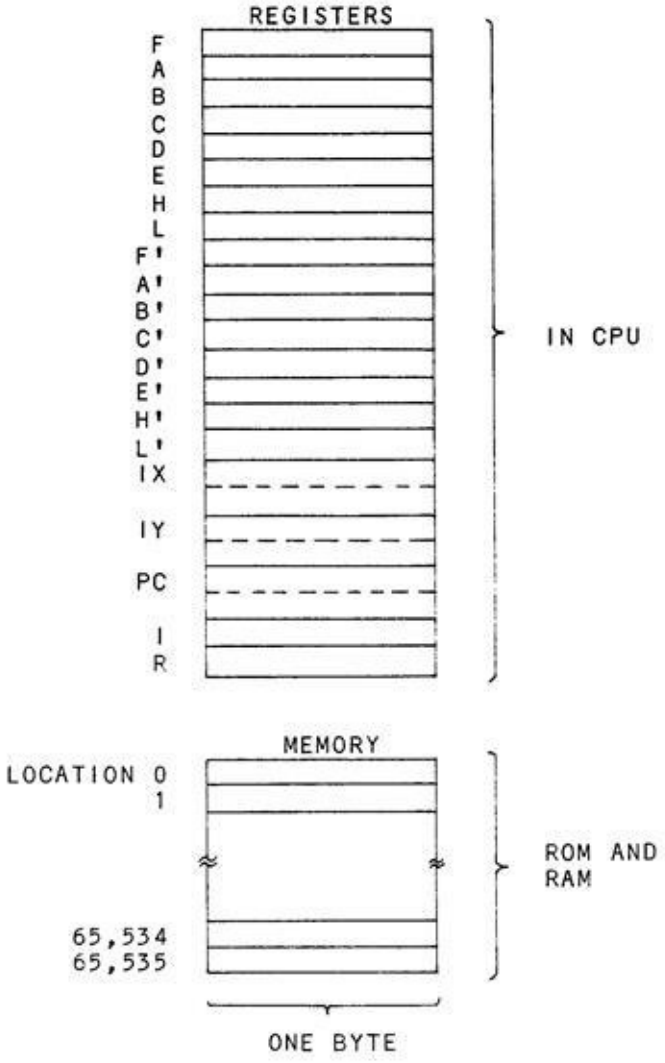


Figure 1-9. Smaller Microcontroller Registers and Memory

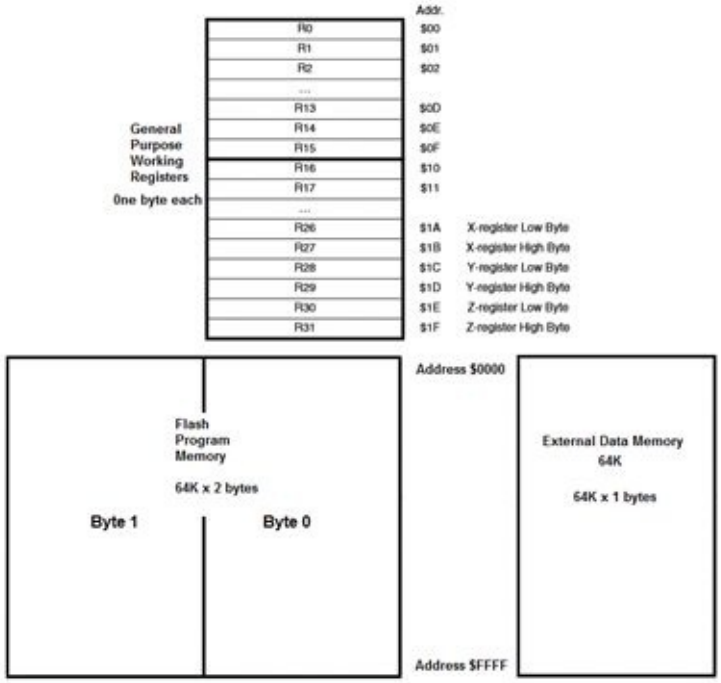


Figure 1-10. Registers and Memory in AtMega 128 (Arduino)

Are there any other groupings above or below bytes? Larger microcomputers talk about words, which may be two or more bytes but, generally, bytes are the most commonly used collection of bits we talk about. Sometimes, groupings of four bits are referred to as a **nibble**.(Early computer engineers must have been obsessed with food!)

Assembly language and higher-level languages generally work with eight or sixteen bits of data at a time - one or two bytes, although more powerful microprocessors may do 32-bit or 64-bit operations. The early BASIC language used PEEK or POKE instructions, which allowed the user to read or write data into a memory location. This is valuable when used to change some of the system parameters that otherwise would be inaccessible from BASIC.

Two bytes are used for simple integer variable representation. ln this type of format, a variable can hold values of -32,768 through+32,767. This number is held as a signed integer value, which we'll look at in Chapter 3.

If you're interested in assembly language on your microcomputer, then you will be working with single bytes to perform arithmetic operations such as addition and subtraction, and with one to four bytes to represent the machine language equivalent of the instruction to the microprocessor. Because we'll be continually referencing one- and two-byte values, let’s investigate them further.

CONVERTING FROM BINARY TO DECIMAL

Big Ed found the maximum values that could be held in one and two bytes by adding up all of the powers of two. They were 255 for a one-byte value, and 65,535 for a two-byte value. Any number in between can be represented by setting the appropriate bits.

Suppose that we had the 16-bit binary value 0000101001011101.To find the equivalent decimal number of this binary value, we could use Big Ed's method of adding up the powers of two. This is done in Fig. 1-11, where we find the result of 2653. Is there an easy way to convert from binary to decimal? Yes, there are a number of easier ways.

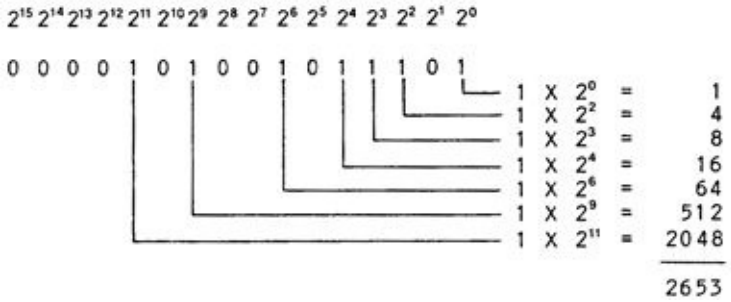


Figure 1-11. Binary to Decimal conversion

The first way is by reference to a table of values. We've included such a table in Appendix C, which shows the relationship between binary, octal, decimal, and hexadecimal for values up to 256. As displaying values up to 65,535 for sixteen bits takes up quite a bit of space, there must be a more efficient way.

A method that works surprisingly well is called **double-dabble.** After a little bit of practice, it becomes very easy to convert ten or so digits from binary to decimal. In the next chapter, we'll show you a method that works for sixteen bits or more. Double-dabble is a strictly mechanical procedure that automates the conversion process. It's harder to describe than do. To use it, do the following:

1. Find the first (leftmost) 1 bit of the binary number.

2. Multiply the 1 bit by two and add the next (rightmost) bit of 0 or 1.

3. Multiply the result by two and add this result to the next bit of 0 or 1.

4. Repeat this process until the last (rightmost) bit has been added to the

total. The result is the decimal equivalent of the binary number.

This procedure is shown in Fig. 1-12 for the decimal number 2653 used in the previous example.

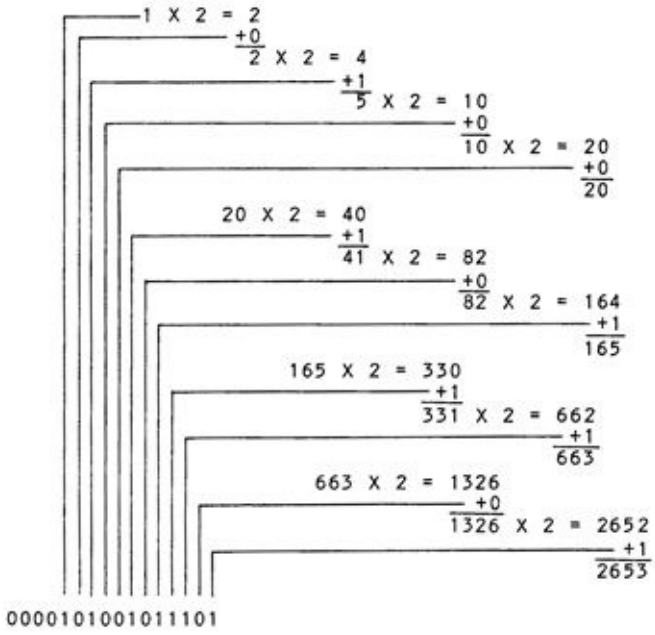


Figure 1-12. Binary to Decimal Conversion Using Double-Dabble

When you've been working with binary numbers for some time, you'll probably immediately recognize 1111 as decimal 15 and 11111 as 31. There are also little tricks that you may employ, such as realizing that 11110 is one less than 31, or that 101000 is actually 1010 = 10 times 4 for a result of 40. For the time being, however, just do a few examples to get used to the procedure; proficiency will automatically come with time, and it probably isn't worth it to spend dozens of hours becoming a binary whiz.

CONVERTING FROM DECIMAL TO BINARY

How about conversion in the opposite direction? This is a somewhat different process. Suppose that we have the decimal number 250 to convert to binary. We'll look at several methods. The first method is the **inspection by powers of two** method. It could also be called successive subtraction of powers of two. ln this method, we try to subtract a power of two and put a 1 bit into the proper bit position, if we can (see Fig. 1-13).

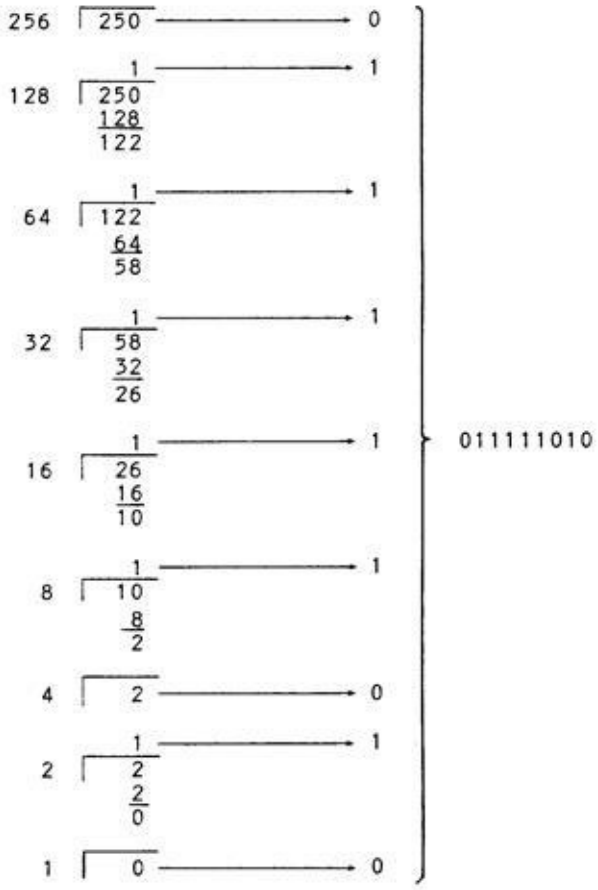


Figure 1-13. Decimal to Binary Conversion by Inspection

Some of the powers of two are: 256, 128, 64, 32, 16,8, 4, 2, and 1, starting with the larger powers. It's obvious that 256 won't go, so we'll put a 0 in that bit position. The power 128 does go, leaving 122. We put a 1 into bit-position 7. The power 64 goes into 122, leaving 58, so we put a 1 into bit-position 6. This process is repeated until the last bit position has been calculated, as shown in Fig.1-13

The above method is pretty tedious. Is there a better way? One better

method is called the **divide by two and save remainders** method. ln this method, we do exactly what the name implies; it is illustrated in Fig. 1-14. The first division is 2 into 250, yielding 125,remainder 0. The next division is 2 into 125, yielding 62, remainder 1.The process is repeated until the **residue** is 0. Now the remainders are arranged in reverse order. The result is the binary equivalent of the decimal number.

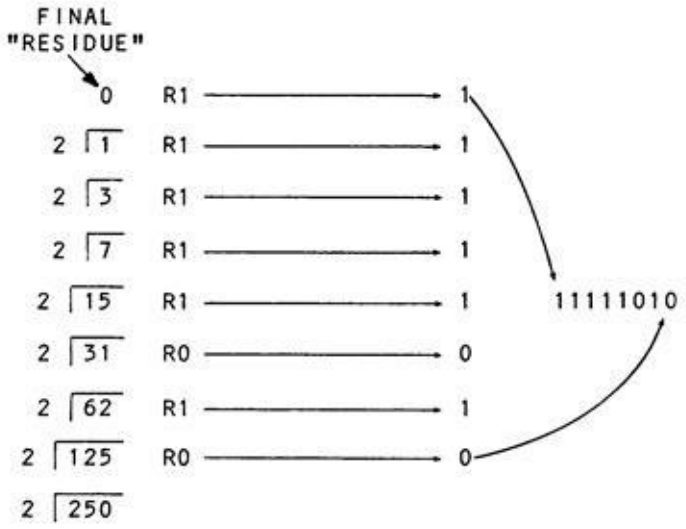


Figure 1-14. Decimal to Binary Conversion by “Divide and Save Remainders” Method

PADDING OUT TO EIGHT OR SIXTEEN BITS

Here’s a point that may not be apparent. Unused bit positions to the left are filled with 0 bits. This is simply a nicety used to "pad out" the binary number to eight or sixteen bits, whichever size we're working with at the time. It has some implications for **signed numbers**, however, so it's best to start using the practice early in the game.

ln the next chapter, we'll cover the shorthand notation of octal and hexadecimal numbers. In the meantime, try your hand at some self-test exercises for practice in conversion between decimal and binary numbers.

EXERCISES

1. List the binary equivalents of decimal 20 through 32.

2. Convert the following binary numbers to their decimal equivalents: 00110101, 00010000, 01010101, 11110000, 0011011101101001.

3. Convert the following decimal numbers to their binary form: 15, 26, 52, 105,255, 60000.

4. "Pad out" the following binary numbers to eight bits: 101, 110101, 010101.

5. What is the largest decimal number that can be held in four bits? Six bits? Eight bits? Sixteen bits? If n is the number of bits, what general statement can you make about the largest number that can be held in n bits? (The response, ”Some mighty big numbers!", is not considered acceptable.)

CHAPTER 2

Octal, Hexadecimal, and Other Number Bases

Hexadecimal and octal are variants of binary numbers. They are commonly used in microcomputer systems, especially hexadecimal. Data can be specified in hexadecimal notation in both higher-level languages and assembly language on many microcomputers. We’ll discuss hexadecimal and octal in this chapter, along with some other interesting number systems. Let's revisit Big Ed.

CHILI IS BIG ON REGULUS

"We don't get many of your type around here," said Big Ed, as he set down a bowl of Big Ed's Chili Surprise in front of a male customer with scaly green skin.

“I know I should say something like 'Yes, and at these prices, you'll get even fewer,' but I’ll tell you the truth. I’m just in from the United Nations to look over your semiconductor industry," said the visitor. "It's very impressive."

"I've done some work in it myself," said Big Ed, glancing at his call board. "Where you from, Buddy? I couldn't help noticing your eight-fingered hands."

"You've probably never heard of it - it's just a small star …er … place. These hands, by the way, are what bring me to Silicon Valley. Your recent microprocessor products are a natural for my people. Would you like to hear about it?"

Ed nodded.

"You see, my people are called Hexers. We base everything upon powers of sixteen. When our civilization was first developed, we counted on our hands. We found it pretty easy to count up to your value of sixteen by simply using our fingers. Later, we needed to express larger numbers. Eons ago, one of our kind discovered **positional notation**. As we have sixteen fingers, our numbers use a base of sixteen, just as your numbers use a base of ten. Each digit position represents a power of sixteen - 1, 16, 256, 4096, and so forth.”

"Exactly how does that work?" said Big Ed, anxiously eyeing the fresh tablecloth he had put on several days ago.

"Here’s a typical number," said the visitor, swiftly scratching on the tablecloth. “Our number A581 represents

$$A \times 16^3 + 5 \times 16^2 + B \times 16^1 + 1 \times 16^0$$

"But, what are the As and Bs?" asked Big Ed.

"Oh, l forgot. When we count on our fingers, we count 1, 2, 3, 4, 5,6, 7, 8, 9, A, B, C, D, E, and F. Actually, that's only fifteen fingers - our last represents the number after 'F' our '10,' which is your sixteen."

"Aha! So A represents our decimal 10, B represents decimal 11, and so forth," said Ed, drawing a chart as shown in Fig. 2-1.

DECIMAL	BASE 16
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

Figure 2-1. Base 16 Representation

Exactly right!" said the visitor. "Now you can see that our number A5B1, in our base 16, is equal to your number 42,417."

$A \times 16^3 + 5 \times 16^2 + B \times 16^1 + 1 \times 16^0 =$
$10 \times 4096 + 5 \times 256 + 11 \times 16 + 1 \times 1 =$
42,417

"Well, to make a long story short, your decimal number system is really terrible. We almost decided not to trade with you after we discovered that you used decimal numbers. Fortunately, however, we found that the number base you most frequently used on your computers was hexadecimal, which is our base 16."

"Wait a minute!” cried Big Ed. "We use **binary** on our computer systems!"

"Well, yes, of course, on a 'hardware' level. But you use hexadecimal as a kind of shorthand to represent data values and instructions in memory. After all, binary and hexadecimal are almost identical.” He went on after seeing Big Ed's puzzlement.

"Look, suppose that I have the number A1F5. I’ll represent it by holding up my hands (Fig. 2-2). The eight fingers of the hand on your right represent the two hexadecimal digits of F5. The eight fingers on your left represent the two hexadecimal digits of A1 . Each hexadecimal digit is represented by four fingers, which hold four bits, or the digit in binary. Get it?”

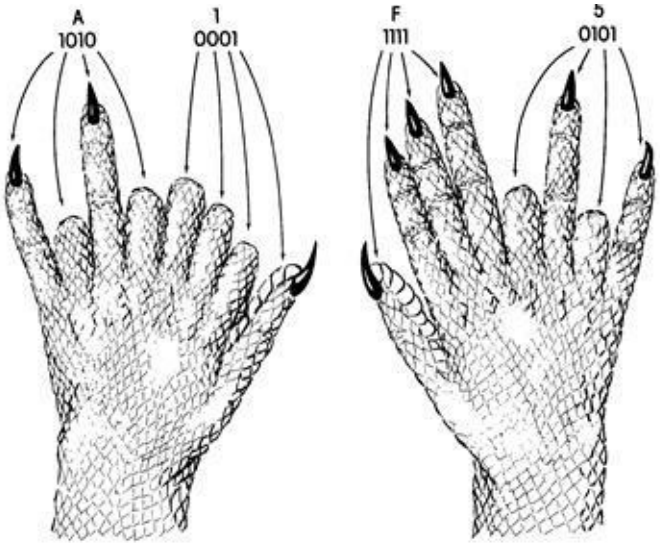


Figure 2-2. Hexadecimal Shorthand

Big Ed scratched his head. "Let's see, 5 in binary is 0101, and that's represented by those four fingers. The next group of four represents the F which is really 15, or binary 1111.The next group ..., Oh, l get it. instead of writing down 1010000111 1 10101 , you just write a short-hand notation of A1 F5.”

"Big Ed, you don't just serve the best chili this side of Altair, you're a mathematician to boot!" exclaimed the visitor, as he left with a scaly green eight-fingered wave.

Big Ed looked at the sixteen-sided copper piece that had been left as a tip and then started figuring on the tablecloth.

HEXADECIMAL

Ed's visitor was correct. Hexadecimal is used on microcomputers because it is a convenient way to shorten long strings of binary ones and zeroes. Each group of four bits can be converted to a hexadecimal(we'll use **hex** from now on) value of 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B,C, D, E, or F as shown in Table 2-1.

Binary	Decimal	Hex
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

Table 2-1. Binary, Decimal, and Hexadecimal Representation

It's a simple matter to convert from binary to hex. Starting from the rightmost bit (bit 0), divide the binary number into groups of four. If you do not have an integer multiple of four (4, 8, 12, etc.), you may have some bits left over on the left; in this case, simply pad out with zeroes. Now convert each group of four bits into a hex digit. The result is the hexadecimal representation of the binary value, which is one-fourth as long in terms of characters. An example is shown in Fig. 2-3.

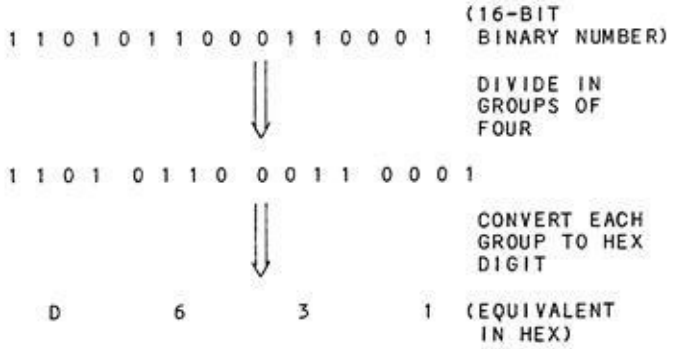


Figure 2-3. Convert from Binary to Hex

To convert from hex to binary, do the inverse. Take each hex digit and convert it into a 4-bit group. An example is shown in Fig. 2-4.

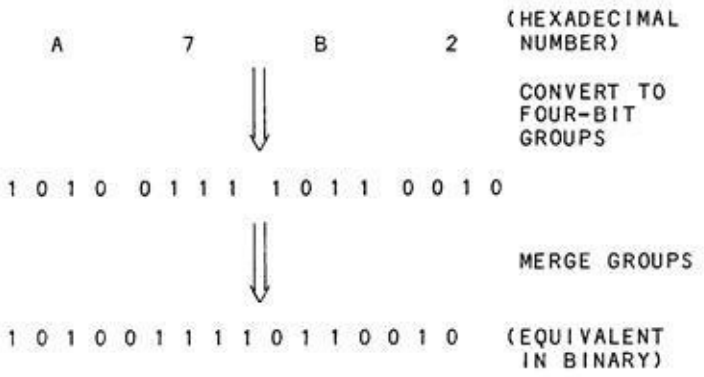


Figure 2-4. Hex to Binary Conversion

Hex notation is used for the early Z-80, 6502, and 6809 microprocessors and many contemporary microprocessors and microcontrollers.

Converting Between Hex and Decimal

Converting between binary and hex is easy. What about decimal and hex conversions? Many of the principles and techniques discussed in the first chapter still apply.

To convert from hex to decimal by the powers of sixteen method, take each hex digit and multiply by the appropriate power of sixteen, as Big Ed did. To convert 1F1E, for example, we'd have:

1 x 4096 + 15 x 256 + 1 x 16 + 14 x 1 = 7966

The double-dabble method can also be adapted to a **hexa-dabble**. (ln fact,

this scheme works for any number base.) Take the leftmost hex digit and multiply by 16. Add the next hex digit. Repeat the multiplication and addition process until the last (rightmost) hex digit has been used. This procedure is shown in Fig. 2-5.

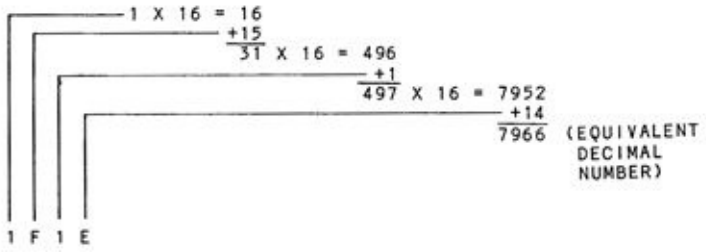


Figure 2-5. Hexa-dabble Hex to Decimal Conversion

You might try this procedure and compare the results with a double-dabble of the binary number. (Needless to say, the results had better be the same.)

Converting From Decimal to Hexadecimal

The method of successive subtraction of powers of sixteen doesn't work too well here, as you might have to do fifteen subtractions to get one hex digit! The analogous "divide by sixteen, save remainders "method, however, works quite well as shown in Fig. 2-6. Take the decimal value 48,555 as an example. Dividing by 16 yields a value of 3034 with a remainder of 11 (hex B). Then, dividing the 3034 by 16 yields 189 with a remainder of 10 (hex A). Dividing the 189 by 16 yields 11, with a remainder of 13 (hex D). Finally, dividing 11 by 16 gives 0 with a remainder of 11. The remainders in reverse order are the hex equivalent, BDAB.

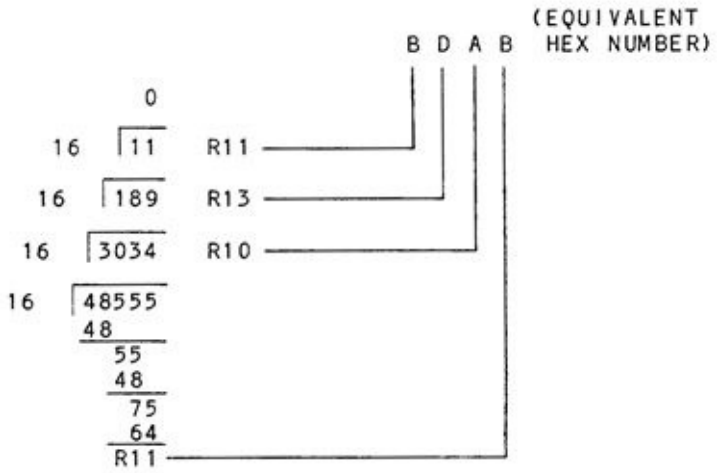


Figure 2-6. Decimal to Hex conversion

OCTAL

Hexadecimal is the most commonly used number base on microcomputers. However, octal, or base 8, is also sometimes used, especially for those microprocessors that use fields of three bits in their machine language architecture (early 8086 microprocessors, for example). Octal values use powers of 8; that is, position 1 is to the zero power(8^0), position 2 is to the first power (8^1), position 3 is to the second power (8^2), and so forth. Here, there is no problem with assigning names to new digits, as there was in the case of the hex digits A through F. The octal digits are 0, 1, 2, 3, 4, 5, 6, and 7. Each digit can be represented by three bits.

Converting Between Binary and Octal

Conversion between binary and octal numbers is similar to hex conversion. To convert from binary to octal, group the bits into 3-bit groups, starting at the right. If you are working with 8- or 16-bit numbers, you will have some bits left over. Pad these with zeroes. Now change each group of three bits into an octal digit. An example is given in Fig. 2-7. To convert from octal to binary, reverse the process.

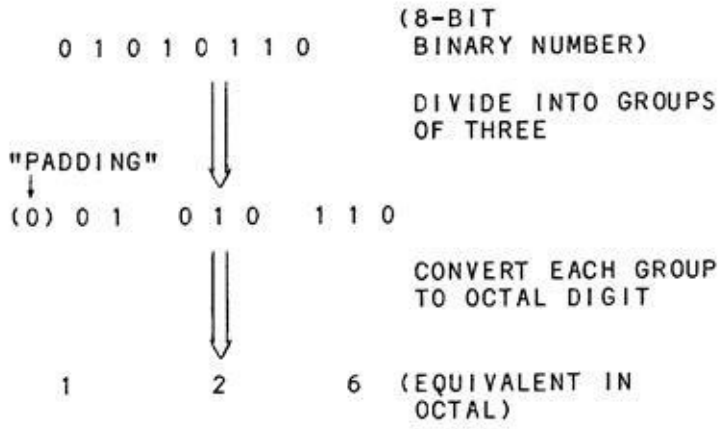


Figure 2-7. Binary to Octal Conversion

Converting Between Octal and Decimal

Conversion from octal to decimal may be handled by the powers of eight method or by **octal-dabble**. ln the first method, multiply the octal digit by the power of eight. To convert octal 360, for example, we'd have:

3 x 8^2 + 6 x 8^1 + 0 x 8^0 =
3 x 64 + 6 x 8 + 0 x 1 =

192 + 48 + 0 = 240

In using the octal-dabble method, go to the leftmost octal digit, and multiply it by eight. Add the result to the next octal digit. Multiply this result by eight, and add to the next octal digit. Repeat this process until the last (rightmost digit) has been added. This procedure is shown in Fig. 2-8.

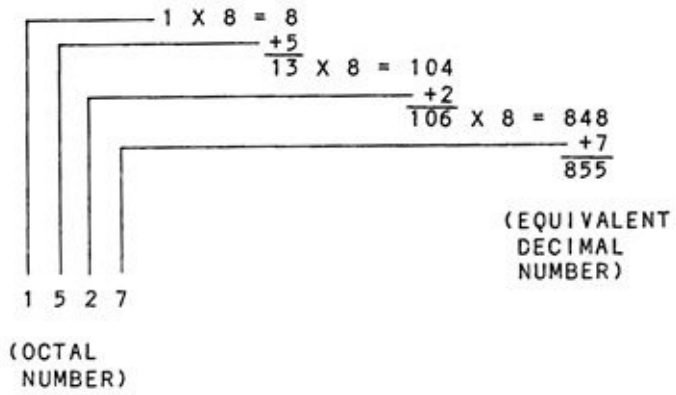


Figure 2-8. Conversion from Octal to Decimal using Octal-dabble

To convert from decimal to octal, the "divide and save remainders" technique is used again. Divide the octal number by eight and save the remainder. Divide the result again, and repeat until the remainder is less than eight. The remainders in reverse order are the equivalent octal number. An example is shown in Fig. 2-9.

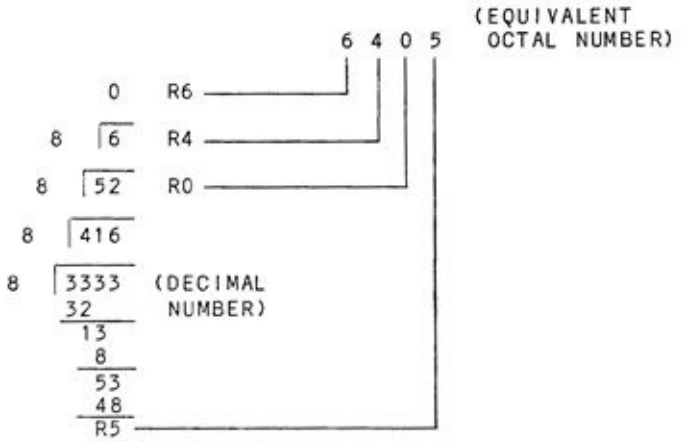


Figure 2-9. Decimal to Octal Conversion by “Divide and Save Remainders Method

WORKING IN OTHER NUMBER BASES

Although hexadecimal and octal are the most popular number base used in microcomputers, it's possible to work in any number base whether it is base 3, base 5, or base 126. Some software packages allow just about any base to be used for a large number of digits of precision.

Two examples of the use of such a base might be interesting. An old technique of compressing three characters into two bytes uses **base 40**. Each of the alphabetic characters A through Z, the digits 0 through 9, and four special characters (period, comma, question mark, and exclamation mark, or any four others) are assigned a code of 0 through 39 decimal. As the largest three-digit base-40 number is $39 \times 40^2 + 39 \times 40^1 + 39 \times 40^0$, or 63,999, the three base-40 digits can be nicely put into sixteen bits (two bytes). Normally, the three characters would have to be held in three bytes. This "compression" results in a 50% saving in memory space for text using only A through Z, 0 through 9, and four special characters.

A second example would be the special processing of a tic-tac-toe array. Each of the nine elements of a tic-tac-toe "square" holds a space, a "naught," or a "cross." As there are three characters, a base 3 representation (space = 0, naught = 1, cross = 2) may be used to advantage. The largest base 3 number for this encoding would be

$$2 \times 3^8 + 2 \times 3^7 + 2 \times 3^6 + 2 \times 3^5 + 2 \times 3^4 + 2 \times 3^3 + 2 \times 3^2 + 2 \times 3^1 + 2 \times 3^0, \text{ or } 19682,$$

which again can be held nicely in sixteen bits or two bytes.

Numbers in other bases may be converted to decimal numbers and back again by the "base-dabble" method and the "divide by base, save remainders" method, in a similar fashion to octal and hexadecimal numbers.

STANDARD CONVENTIONS

In the remainder of this book, we'll occasionally use the suffix "H" for hexadecimal numbers. The number "1234H," for example, will mean 1234 hexadecimal and not 1234 in decimal. (It will also be equivalent to &H1234 in certain higher-level languages.) Also, we'll express powers by using an up arrow(^) to represent exponentiation-raising a number to a power. The number 2^8 will be represented by 2 ^8, 10^5 by 10 ^ 5, and 10^{-7} (or $1/10^7$) by 10^ -7.

In the next chapter we'll consider **signed numbers**. In the mean time, work through the following exercises in hexadecimal, octal, and special bases.

EXERCISES

1. What does the hexadecimal number 9E2 represent in terms of powers of 16?

2. List the hexadecimal equivalents of decimal O through 20.

3. Convert the following binary numbers to hexadecimal: 0101, 1010, 10101010, 01001111, 1011011000111010.

4. Convert the following hexadecimal numbers to binary: AE3, 999, F232.

5. Convert the following hexadecimal numbers to decimal: E3, 52, AAAA.

6. Convert the following decimal numbers to hexadecimal: 13, 15, 28, 1000.

7. The greatest memory address in a 64K microcontroller is 65,535. What is this in hexadecimal?

8. Convert the following octal numbers to decimal: 111, 333.

9. Convert the following decimal numbers to octal: 7, 113, 200.

10. What can you say about the octal number 18? (Limit your answers to 1000 words or less, please.)

11. In a number system based on seven, what would be the decimal equivalent of 636, base 7?

CHAPTER 3

Signed Numbers and Two’s Complement Notation

Thus far we've talked about **unsigned binary numbers** that represented only positive values. ln this chapter, we'll learn how to represent positive *and* negative values in microcomputers.

BIG ED AND THE BINACUS

"Hi guy. What'll it be?" Big Ed asked a short customer with hair in a long braided queue and colorful brocaded coat.

"I'll have the chop suey," said the customer."

“What do you have there?" Big Ed asked, spying a strange looking device that resembled an abacus with only a few beads. "It looks like an abacus."

"No, it's a binacus!" said the customer. "It was to be my road to fame and fortune, but alas, he who looks for those two companions on the road of life will find only chuckholes. Would you care to hear about it?”

Ed threw up his hands, knowing he had no choice; it was a slow afternoon…

"This device is like an abacus, but it deals with binary numbers. That’s why it is sixteen columns wide and has only one bead per column(Fig. 3-1). l spent years developing it, and only recently tried to market it through some of the high-tech firms here. They all rejected it!"

"Why, pal?" said Ed, handing over the chop suey.

"It can represent any number from 0 through 65,535 and one can easily add or subtract binary numbers with it. But it has no way of representing negative numbers!"

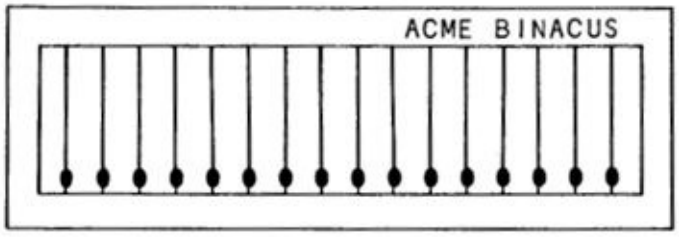


Figure 3-1. The Binacus

Just as he uttered those words, Bob Borrow, the lnlog computer engineer, walked in. "l couldn't help hearing your tale," he said. "l think I might be of some help.”

"You see, your problem is one that computer engineers faced many years

ago. Many times, it's sufficient to simply hold an absolute number. For example, most 16-bit microprocessors can address 65,536 separate addresses, numbered from 0 through 65,535. There’s no reason to have negative numbers in this case. On the other hand, microcomputers have the ability to perform arithmetic operations on **data** and must have the ability to hold both negative and positive values."

"There are different schemes for representing negative numbers. Look at this." Ed winced as Bob reached for the tablecloth.

"You could simply add an extra bead, a seventeenth, on the left end of the binacus. If the bead was up, the number represented would be a positive number; if the bead was down, the number would be a negative number. This scheme is called **sign/magnitude** representation." (See Fig. 3-2.)

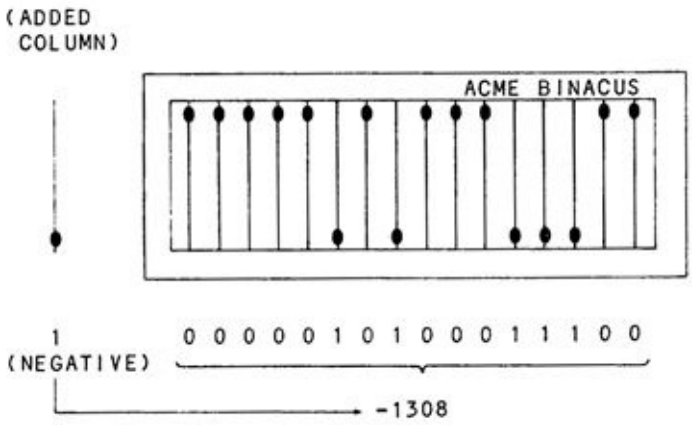


Figure 3-2. Sign/Magnitude Representation

"That's true," said the stranger. "Of course, it would mean some redesign," he mused, looking at the invention.

"Wait, I haven’t come to my point. I’ve got a scheme for you that won't require any redesign at all! It’s called two's complement notation, and it allows you to hold numbers from - 32,768 to + 32,767 without any modification."

"Oh, sir, if you could do that ..., " said the stranger.

"In this scheme, we'll let the sixteenth bead on the end, in bead position 15, represent the **sign bit**. It the bead is up or 0, the sign will be positive, and the remaining beads will hold the value of the number. Since we'll have only 15 beads, the number represented will be from 0 (000 0000 0000 0000) through

32,767 (111 1111 11111 1111)."

"What about negative numbers?" asked the stranger.

"I was getting to that. If the bead in bead position 15 is down, or 1,then the number represented is a negative number. ln this case, flip all of the beads that are up, or 0, down to a 1. Flip all of the beads that are down, or 1, up to a 0. Finally, add one."

"Thanks for your trouble, sir," said the stranger, frowning, as he picked up the device and started to walk out the door.

"No, wait, this system works!" cried Bob. "Here, let me show you. Suppose that you have the binary number 0101 1110 1111 0001."

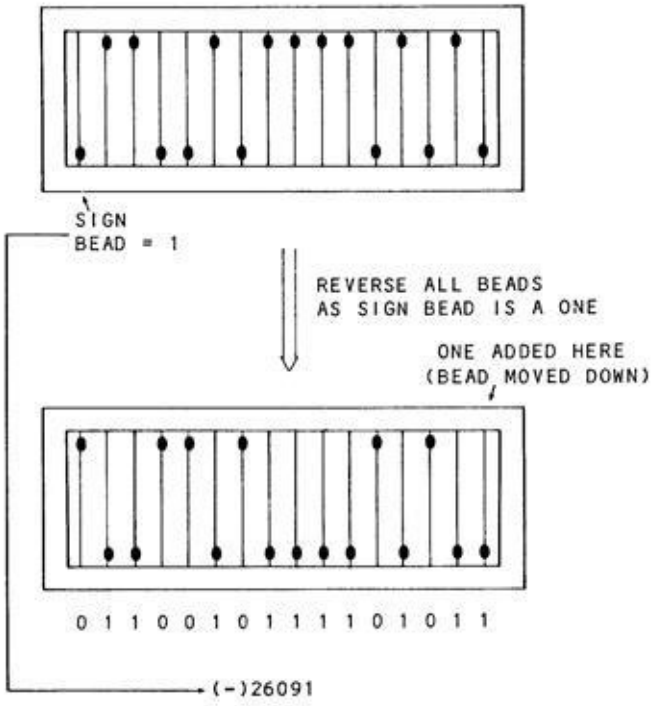


Figure 3-3. Two's Complement Notation

"The sign bead is a 0, so the number represents 101 1110 1111 0001 (hexadecimal 5EF1 or decimal 24,305). Now, suppose that the configuration is 1001 1010 0001 0101. The sign bead is a one, so the number is negative. Now, reverse all the beads and add one." (See Fig. 3-3.)

"The result is 0110 0101 1110 1011. We now convert the number to decimal 26,091; therefore, the number represented is a - 26,091 .This scheme is

the same one that microcomputers use. You'll easily be able to sell your binacus idea to a manufacturer if you tell them it uses this scheme of **two's complement** notation for negative numbers!"

The stranger looked unconvinced. "Let's see if I understand this. If the sign bead is a one, I reverse all the beads and add one? Let me try it with a few examples." He moved the binacus beads and calculated on the tablecloth. Ed grimaced. A chart of what he came up with is shown in Fig. 3-4.

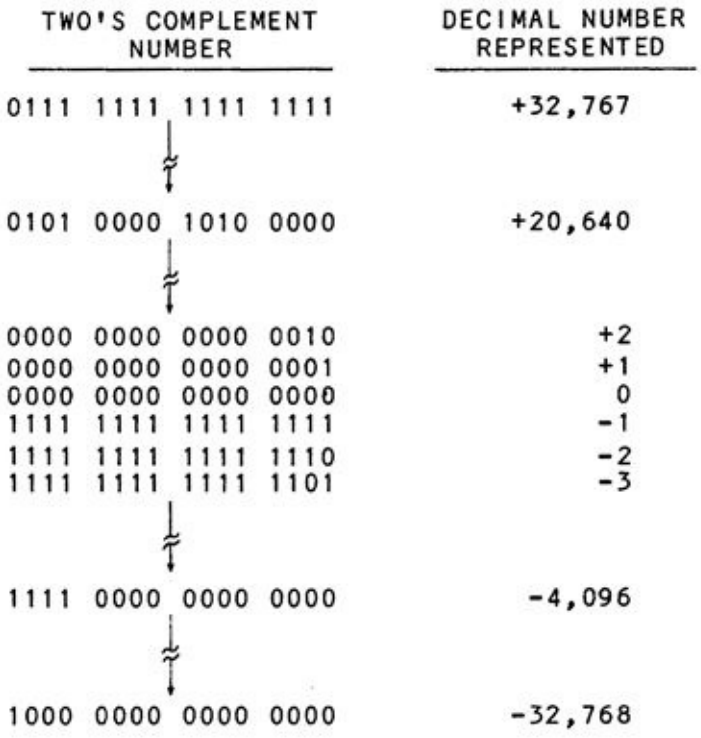


Figure 3-4. Two's Complement Range for Sixteen Bits

"It looks to me like negative numbers from -1 through -32,768 could be held using this scheme. But why such a complicated scheme?"

"To make it easier in hardware, friend," said Bob. "This method means that all numbers can be added or subtracted without first testing the sign of each operand. We just go ahead and add or subtract the numbers, and the result will have the correct sign. Suppose that we have the two numbers 0011 0101 0111 0100 and 1011 1111 0000 0000. These are +13,684 and -16,640, respectively. We add them as follows."

0011 0101 0111 0100 (+13,684)

1011 1111 0000 0000 (-16,640)
1111 0100 0111 0100 (-2,956)

"The result is 1111 0100 0111 0100, or -2956, just as it should be!"

"That’s amazing," said the stranger. "I will study this, sell my binacus, and return to my native land with a fortune!"

"Where might that be?" asked Big Ed.

"Brooklyn," said the stranger. "Goodbye, and thank you. Someday you'll be able to say that you knew Michael O'Donahue!"

ADDING AND SUBTRACTING BINARY NUMBERS

Before we look at the binacus scheme of representing signed numbers (which duplicates the scheme used on all current computers), let's look at the topic of adding and subtracting binary numbers in general.

Adding binary numbers is much easier than adding decimal numbers. Can you remember when you had to memorize your addition tables? Four and five make nine, four and six make ten, and so forth. In binary we also have addition tables, but they are much simpler: 0 and 0 make 0, 0 and 1 make 1, 1 and 0 make 1, 1 and 1 make 0, with a carry to the next higher bit position. If the next higher bit position has a 1 and 1, then the table has a fifth entry of 1 and 1 and 1 make 1,with a carry to the next higher bit position. The table is summarized in Fig. 3-5.

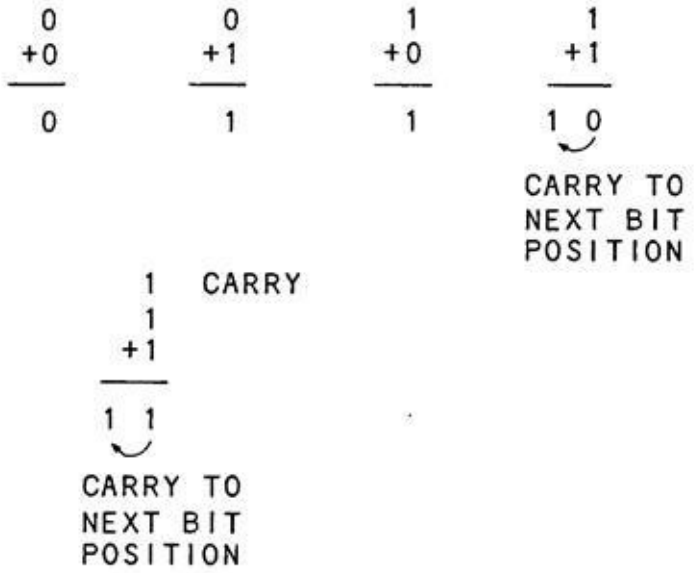


Fig. 3-5. Binary Addition

Let's try this with an example of two unsigned 8-bit numbers, the same ones we've been working with in previous chapters. Suppose we add 0011 0101 and 0011 0111 (53 and 55, respectively).

0110 1110 (Carries)
0011 0101 (53)
0011 0111 (55)
0110 1100 (108)

The 1s above the operands represent the carries to the next higher bit position. The result is 0110 1100, or 108, as we had hoped.

Subtraction is just about as easy. 0 from 0 is zero. 0 from 1 is 1. 1 from 1 is zero. And the toughie, 1 from 0, is 1 with a borrow from the next higher bit position, just as we borrow in decimal arithmetic. This table is summarized in Fig. 3-6.

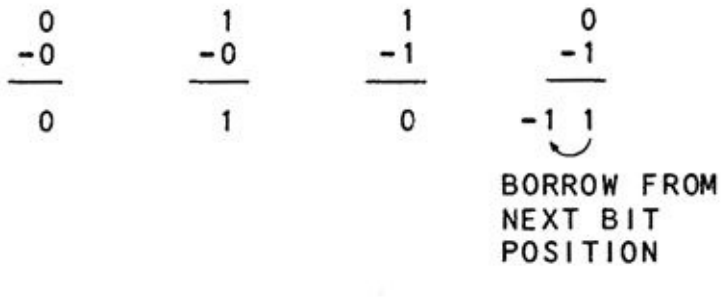


Fig 3-6. Binary Subtraction

Let's try it on some actual numbers. Subtract 0001 0001 from 0011 1010, or 17 from 58:

0000 0010 (Borrow)
0011 1010 (58)
0001 0001 -(17)
0010 1001 (41)

The one above the operand represents the borrow from the next higher bit position. The answer is 0010 1001, a 41, as we had expected.

Now let's try another example. Subtract 0011 1111 from 0010 1010, a subtract of 63 from 42:

1111 1110 (Borrows)
0010 1010 (42)
0011 1111 -(63)
1110 1011 (??)

The result is 1110 1011, or 235, a result we didn’t expect. But wait, could it be? Is it possible? If we apply the rules of two's complement representation

and consider 1110 1011 as a negative number, then we have 0001 0100 after changing all ones to zeroes and all zeroes to ones. We then add one to get 0001 0101. The result is -21 when we add the negative sign, which is exactly right. It seems as though we’re forced into this two's complement business whether we want to be or not!

TWO'S COMPLEMENT REPRESENTATION

We've pretty well covered all aspects of two's complement notation. If the leftmost bit of an 8- or 16-bit value is treated as a sign bit, then it is either a 0 (positive) or 1 (negative). The actual two's complement representation can then be found by applying the rules discussed earlier - looking at the sign bit, changing all ones to zeroes, all zeroes to ones, and adding one.

Numbers are stored in two's complement fashion either as 8-bit two's complement numbers or as 16-bit two's complement numbers. The sign bit is always in the leftmost bit position, and it is always set (1) for a negative number. The formats for both 8- and 16-bit two's complement representation are shown in Fig. 3-7.

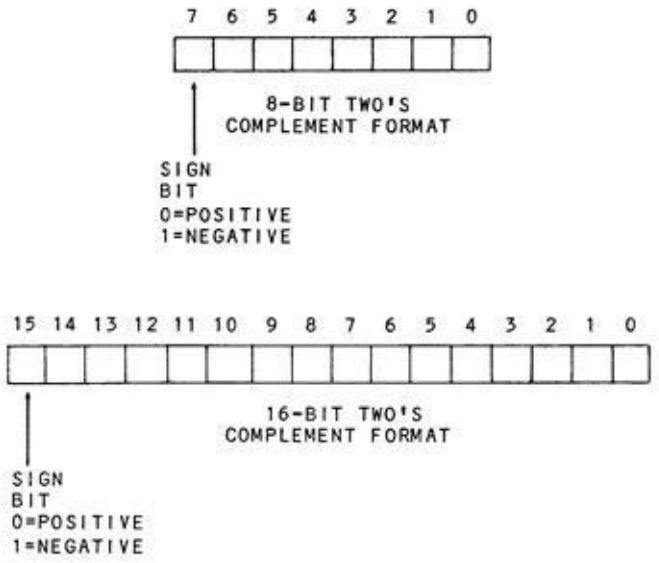


Fig 3-7. Two's Complement Formats

The 8-bit values are used for **displacements** in microprocessor instructions to modify the address of a memory operand. The 16-bit values are used as integer variables in programs. Either of the 8-bit or 16-bit values can, of course, be used by the assembly language or higher-level programmer to represent anything he chooses.

SIGN EXTENSION

It is possible to add or subtract an 8-bit two's complement number and a 16-bit two's complement number. When these operations are done, the sign of the smaller number must be extended to the left until the numbers are of equal length. If this is not done, the result will be incorrect. Take the case of adding the 8-bit value of 1111 1111(-1) to the 16-bit value of 0011 1111 1111 1111 (+16383). If the sign is not extended, we have:

0011 1111 1111 1111 (+16,683)
1111 1111 (-1)
0100 0000 1111 1110 (+16,638)

The result here is +16,638 - obviously incorrect. If the sign is correctly extended to occupy all of the bit positions to the left, however, we have:

0011 1111 1111 1111 (+16,3830
1111 1111 1111 1111 (-1)
0011 1111 1111 1110 (+16,382)

which is the correct result.

ADDING AND SUBTRACTING TWO'S COMPLEMENT NUMBERS

All microprocessors or microcontrollers used in current computers have an instruction to add two 8-bit signed numbers and another instruction to subtract two 8-bit signed numbers. ln addition, some microprocessors enable addition and subtraction of two 16-bit signed numbers or even 32-bit or 64-bit numbers.

The two operands that are involved in the addition or subtraction operation may be any configuration - two positive numbers, a positive and negative number, or two negative numbers. When a subtraction is done, it is convenient to look on the operation as follows. The number to be subtracted (**subtrahend**) is **negated** by taking its two's complement -changing all ones to zeroes, all zeroes to ones, and, then, adding a one. After this is done, the subtraction is identical to an addition of the negated number. The following is an example. Suppose that 1111 1110 (-2) is to be subtracted from 0111 0000 (+112). First the subtrahend of -2 is negated. The two's complement of 1111 1110 is taken to produce 0000 0010 (+2). Then, the addition is done.

0111 0000 (+112)
0000 0010 (+2)
0111 0010 (+114)

Bear in mind that the microprocessor does not perform this negate operation before the subtraction takes place. It is simply a convenient way to visualize what happens in the subtraction and it has some bearing on the operations we'll be talking about in the next chapter.

Before we discuss some of the idiosyncrasies of addition and subtraction, such as overflow and carries, and the **flags** of typical microprocessors, try your hand on a few of the following exercises to sharpen your skills in two's complement addition and subtraction.

EXERCISES

1. Add the unsigned numbers below. Show the results in both binary and decimal.

```
  01
 +11

  0111
 +1111

  10101
+101010
```

2. Subtract the unsigned numbers below. Show the results both in binary and in decimal.

```
  10
- 01


  0111
- 0101
  1100
- 0001
```

3. Take the two's complement of the following signed numbers, if necessary, to find the decimal numbers represented: 01101111, 10101010, 10000000

4. What is the 8-bit two's complement form of -1, -2, -3, -30, +5, and +127?

5. Sign extend the following 8-bit numbers to 16 bits. Show the numbers represented both before and after.

01111111, 10000000, 10101010

6. Add - 5 to -300. (No, no-in binary!)

7. Subtract -5 from -300 using binary.

CHAPTER 4

Carries, Overflow, and Flags

In this chapter, we'll expand some of the concepts presented in the previous chapter. There are some subtle, and some not too subtle, points to consider in working with signed numbers. We'll look at some of them here.

THIS RESTAURANT HOLDS +127 PEOPLE. AVOID OVERFLOW!

Big Ed was just sitting down to a cup of Big Ed's Famous Java after the engineering crowd had left when the restaurant door swung open and a uniformed man walked in.

"Yes, sir, what can I get you?"

"Make it a cuppa coffee and a Danish," the customer answered.

”l notice you're in uniform. Are you in the Air Force from Moffett Field?" asked Ed.

"Naw, I’m a plumber. l came to this area to answer an advertisement in the help wanted section about an 'Overflow Specialist’ at lnlog. I figured I fit the bill, but the HR man just laughed and showed me the door!”

"I think I see the problem, sir," said a slightly built customer who appeared to be about fourteen. An Arduino reference card stuck out of his shirt pocket. "I'm a programmer who has had a lot of experience with 'overflow.' "

"I didn’t think you guys worked wit' pipes."

”Well, we have many pipeline processors, and we do work with overflow quite a bit. Mind if I explain?" The plumber raised his eye-brows and waited.

… Several cups of Ed's coffee later, the programmer had covered the binary system and simple arithmetic operations.

"So you see, you can hold any value from -128 to +127 in eight bits or any value from -32,768 to +32,767 in sixteen bits. Understand?”

The plumber said, "Right, except for unsigned numbers, in which case the range is 0 through 255, or 0 through 65,535, or 2 ^ (N - 1),where N is the size of the register in bits."

"My, you are a fast learner," said the programmer. ”Now, to continue …. If we perform an addition or subtraction in which the result is greater than +32,767 or less than -32,768 (or is greater than +127 or less than - 128), what happens?”

"Well, I guess you get an incorrect result," the plumber said.

"That's true, you definitely get an incorrect result. The result is too great in

a positive sense or too great in a negative sense to be held in eight or sixteen bits. In this case, you get **overflow**, because the result overflows the value that can be held in eight or sixteen bits. However, we've designed an **overflow flag** as part of the microprocessor. This flag can be examined by an assembly language programmer to see if overflow occurred after an addition or subtraction."

"Not only do we have an overflow flag, but we have a carry flag, a sign flag, a zero flag…"

"Wait a minute, you mean that guy in HR wanted a computer engineer that specialized in microprocessor arithmetic?" the plumber said with a grin. "I guess the joke's on me! My PhD is in physics!"

"You have a PhD in physics?" sputtered Big Ed, as most of a mouthful of Big Ed's Java went spewing out across the restaurant floor.

"Yes, I'm only in plumbing to make a living," said the plumber as he walked out the door, shaking his head at the afternoon's embarrassment.

OVERFLOW

As we saw in the preceding example, overflow is possible at any time that an addition or subtraction is done and the result is too large to be put into the number of bits allocated for the result. ln the case of most current computers, this would mean when the result is larger than could be held in eight or sixteen bits. Suppose that we are working with an 8-bit register in a microcontroller. A typical add instruction would add the contents of an 8-bit register, called the accumulator, with the contents of another register or memory location. Both operands would be signed two's complement operands. What conditions would cause overflow? Any result that was larger than +127 or more negative than -128 would produce an overflow. Examples are:

```
  1111 0000 (-16)
+1000 1100 (-116)
  0111 1100 (+124no!)

  0111 1111 (-127)
 +0100 0000 (+64)
  1011 1111 (-80 no!)
```

Notice that in both overflow cases, the result was obviously wrong, as the sign was opposite from its true sense. Overflow may only occur when the operation is adding two positive numbers, adding two negative numbers, subtracting a negative number from a positive number, or subtracting a positive number from a negative number.

The result of an addition or subtraction in most microprocessors either sets (1) or resets (0) a **flag** bit in the microprocessor. This flag bit can be tested or used in a **conditional jump** - a jump if overflow, or a jump **if not** overflow. The testing is done at a machine-language level only, and not in higher-level languages.

CARRY

Another condition that is important is a carry condition. We've seen how carries are used in additions of binary numbers, and how **borrows** (a form of carry) are used in subtractions. The carry that is produced by an addition or subtraction is the carry that is **propagated** off the most significant bit of the result. Here's an example. Suppose that we add the two numbers below:

```
1 1111 111 (Carries)
 0111 1111 (+127)
 1111 1111 (-1)
 0111 1110 (+126)
```

The ones above the operands represent the carries to the next bit position. The leftmost carry, however, "falls off the end" into the bit bucket, an imaginary container. Actually, it doesn't fall into the bit bucket, it normally sets the carry flag in the microprocessor involved. Is this carry useful? Marginally, in this example. There will be a carry as long as the result does not go negative. When the result goes from 0000 0000 to 1111 1111, no carry is produced. As the carry flag may be tested by a conditional "jump if carry" or "jump if no carry" on a machine-language level, the state of the carry is sometimes useful. It is also used for results in shifting, which we'll cover in a later chapter.

OTHER FLAGS

The results of arithmetic operations, such as addition and subtraction, generally set two other flags in the microprocessor. One flag is the “zero" flag. It is set when the result is zero, and reset when the result is not zero. it would be set (1) for an addition of -23 and +23, and reset (0) for an addition of - 23 and +22. As we are constantly comparing values in machine-language programs forth microprocessor, the zero flag is very handy indeed.

Another flag that is generally used is the “sign” flag. The sign flag is set or reset according to the result of the operation. It echoes the bit value in the most significant bit position.

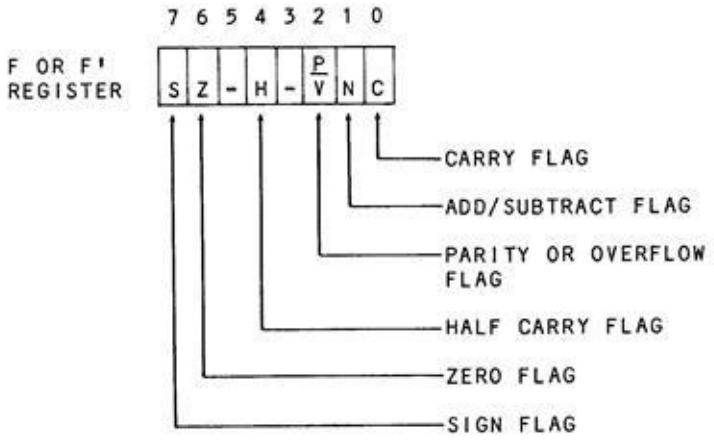


Fig 4-1. Flags in Z-80 Microcomputers

Both the zero flag and the sign flag can be tested in machine-language programs by conditional jumps, such as "jump if zero," "jump if nonzero," "jump if positive," and "jump if negative." Note that a condition of positive includes the case where the result is zero. Zero is a positive number in two's complement notation.

FLAGS IN TYPICAL MICROCOMPUTERS

The flags in the Z-80 microprocessor are representative of the flags in all microprocessors. They are shown in Fig. 4-1. The flags in the ATMega 128 microprocessor (used in some Arduinos) are a second example of flags in a current microprocessor, and are shown in Fig. 4-2. In the next section, we'll examine logical and shifting operations, both of which also affect the flags, in detail. Before we start that chapter, however, here are some exercises in the use of overflow, carries, and flags.

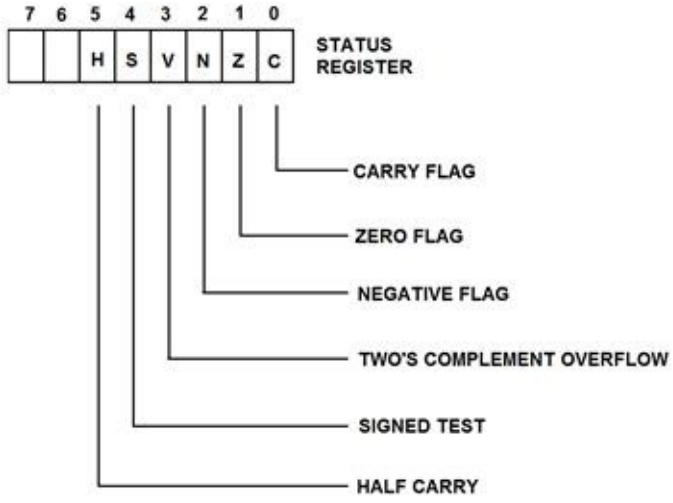


Fig 4-2. Flags in an ATMega 128 Microprocessor (Arduino)

EXERCISES

l. Which of the following operations will result in overflow? Show all values.

```
  01111111
+00000001

  01111111
 -00000001

  10101111
+11111111
```

2. Which of the following operations will produce a carry "off the end" that sets the carry flag?

```
  01111111
+00000001

  11111111
+00000001
```

3. Suppose that we have a Z(ero) flag and an S(ign) flag in the microprocessor. What are the "states" (0 for reset, l for set) of the Z and S flags after the following operations:

```
  01111111
+10000001

 11011111
-10101010
```

CHAPTER 5

Logical Operations and Shifting

Logical operations in microcomputers are used to manipulate data on a bit or **field** basis. Shifting also may be used to process bits in binary numbers, or to implement simple multiplication or division.

THE BRITISH ENIGMA

Things were slow at Big Ed's Restaurant in Silicon Valley. Ed was just getting ready to open the local newspaper when he heard a screeching of brakes, and a large red double-decker bus pulled up in front. The doors of the bus opened and a dozen or more people tumbled out. All seemed dressed in tweeds, bowler hats, and other British attire. One even carried a black bumbershoot and kept peering anxiously at the sky.

"Hello, there. Can I help you?"

"Ratherrr. Can you accommodate a group of 17 British computer engineers and scientists?"

"l think so," said Big Ed. "If you don't mind two tables close together, we can put eight of you at Table A and eight of you at Table B. One of you will have to sit in a chair next to Table A. It's my daughter Carrie's chair, but l think it'll do. That'll accumulate ... er ... accommodate all of you."

"Jolly good," said the spokesman, as the group filled up the two tables and the extra chair (Fig. 5-1).

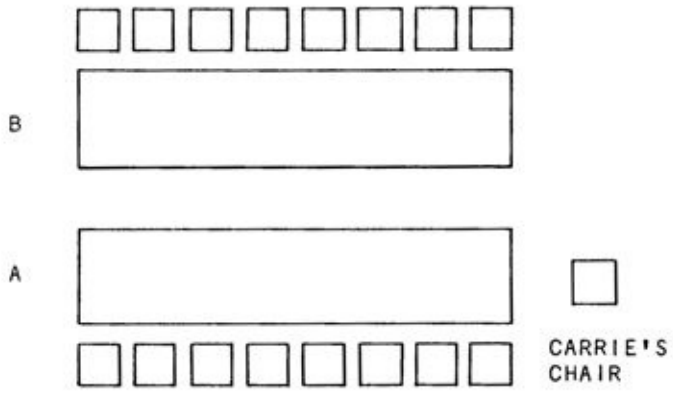


Figure 5-1. The seating Arrangement

"Wow, I never realized there were so many computer scientists in Britain," said Ed.

"Just like you Yanks," said the spokesman. "Haven't you heard of Turing, the Colossus Project to break Enigma, or Williams Tubes?"

Big Ed shook his head. "Sorry, no slight intended."

”That's all right, Yank. Let's see, we've got to have lunch and, then attend a special conference on microprocessors for cricket applications. Gentlemen, may l have your attention, please?"

“As you know, the ministry has given us limited funds for this trip. Therefore, we must put certain constraints on lunch today. I’ve been looking over the menu, and reached the following conclusions.”

"One. You may have either coffee (ugh) OR tea, but NOT both.”

”Two. You may have soup OR salad or both."

“Three. You may have a sandwich OR a businessman's lunch OR BigEd's Surprise."

“Four. If you have dessert AND an after-lunch drink, you must pay the additional charge. Any questions?"

One of the older scientists spoke. ”Let me see. As I understand it, we have an Exclusive-OR of the coffee and tea, an Inclusive-OR of the soup and salad, an Inclusive-OR of the luncheon, and an AND of the dessert and drink for the additional charge. Correct?"

"`That’s correct, Geoffrey. All right chaps, chow down!"

"l say, this air conditioner is giving me quite a stiff neck," said one of the young engineers.

"What we must do, then, is to rotate Table A through the extra chair, so that everyone gets a turn at the cool air," said the spokesman. “Every time I say rotate, we'll rotate one chair position."

"Is it really necessary to go through my chair?" asked the engineer seated in the extra chair.

"Well, we have the option of rotating through the chair or not, but I think we'd best do it to be fair."

At regularly spaced intervals, as the lunch progressed, the spokesman would shout out "Rotate!” and the group at Table A and the extra chair would move as shown in Fig. 5-2. Finally the entire group at Table A got up, paid their bills, and left.

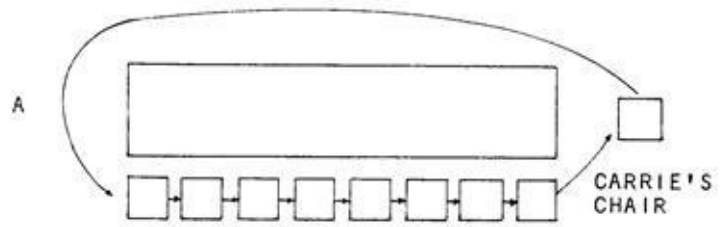


Fig 5-2. The Group Rotates

"l think we're just about done here, sir," said the group at Table B.

"All right, then, shift left logically into the empty chair so I can look at your bill," said the spokesman. The scientist nearest to the cash register got up and slid into the empty chair. The spokesman examined the bill. "Next shift!" he shouted. The person in the extra chair went to the cash register, and the next person slipped into the extra chair. This process continued until Table B was empty (see Fig. 5-3).

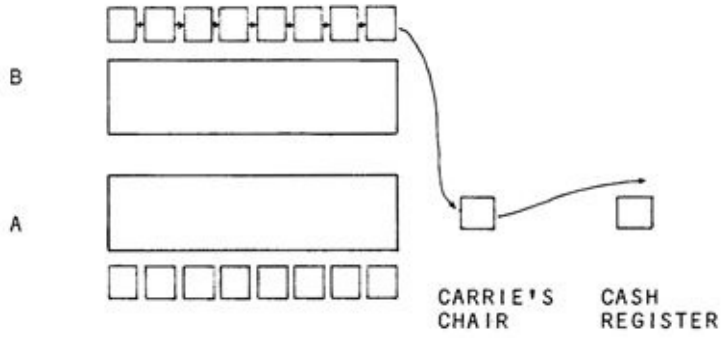


Fig 5-3. The Group Shifts

"Jolly good lunch," said the spokesman to Big Ed as he left.

"It was nice having you," said Ed. "I hope your cricket seminar goes well."

“We’re not too worried about that. It’s the arachnid problem that concerns us," said the spokesman as he unfurled his umbrella and strode out the door into the sunny San Jose skies. "Cheerio!"

LOGICAL OPERATIONS

All microcomputers have the ability to perform the **logical** operations of ANDing, ORing, and Exclusive-ORing on a machine-language level. Also, most higher-level languages allow ANDing and ORing.

All logical operations operate on a "bit position" basis. There are no carries to other bit positions. On a machine-language level, logical operations operate on one byte of data; higher level languages allow the logical operations to take place with two-byte or larger operands. There are always two operands involved, and one result.

ORS

The OR operation is shown in Fig. 5-4. Its **truth table** states that there will be a 1 bit in the result if either operand has a 1 bit or both operands have a 1 bit.

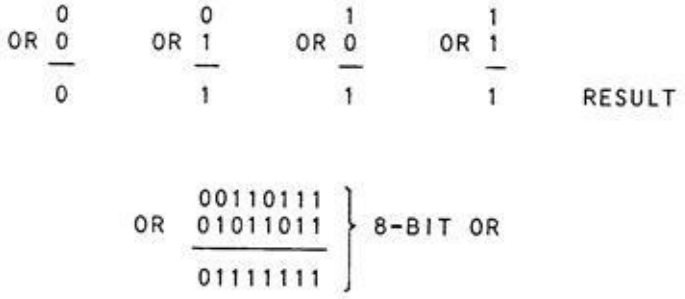


Fig 5-4. OR Operation

The OR operation is used on a machine-language level to **set** a bit in a value. A typical application might use the eight bits of a memory location as eight **flags** for various conditions, as shown in Fig. 5-5.

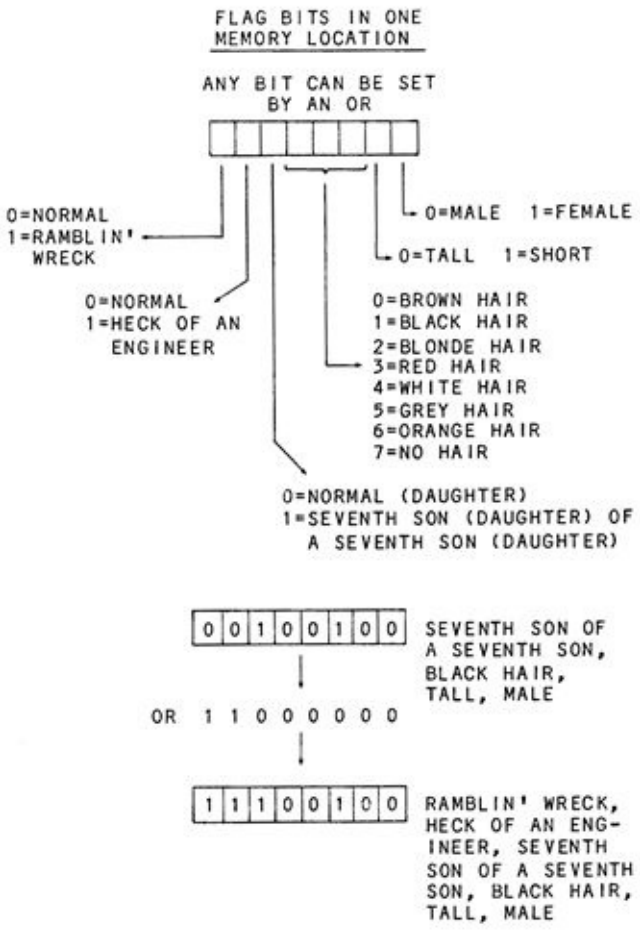


Fig 5-5. Using an OR Operation

The OR would not be as widely used in high-level languages, but it comes up occasionally, as for example, when setting the bits of a video-display byte to lower case, as shown in Fig. 5-6.

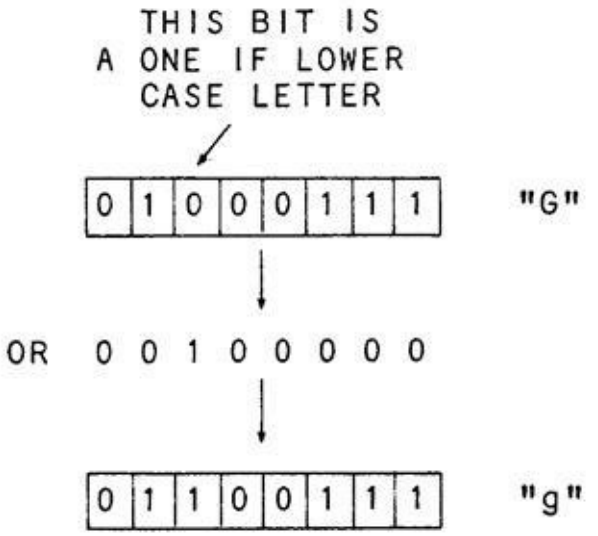


Fig 5-6. ORing Operation

ANDS

The AND operation is shown in Fig. 5-7. It also operates only on a bit-wise level with no carries to other bit positions. The result of the AND is a one if both operands have a 1 bit; if either has a O, the result is zero.

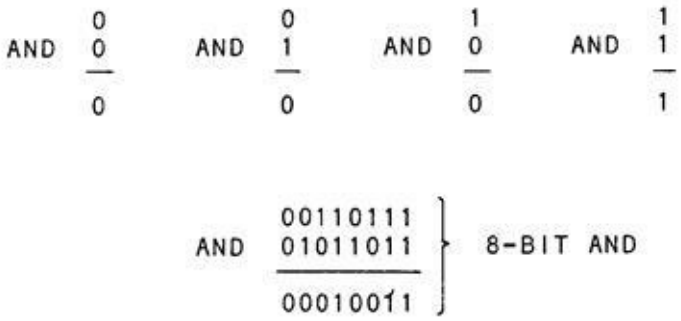


Fig 5-7. AND Operation

The AND is used on an assembly-language level primarily to **reset** a bit, or to **mask out** certain parts of an 8-bit word, as shown in Fig.5-8. ln higher-level languages, the AND operation would have more limited applications. An example is shown in Fig. 5-9; it checks for multiples of 32 fora line count of 32 lines per page.

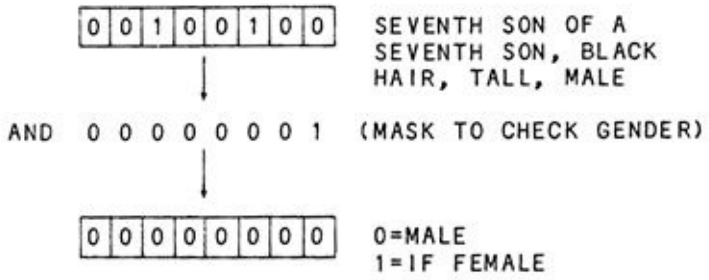


Fig 5-8. ANDing Example

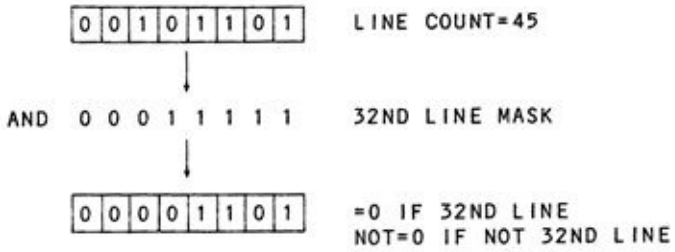


Fig 5-9. Another ANDing Example

EXCLUSIVE ORS

The Exclusive-OR is shown in Fig. 5-10. Its rules state that the result is a 1 if one bit OR the other bit, but not both, are 1s. In other words, if both bits are 1s, the result is 0.

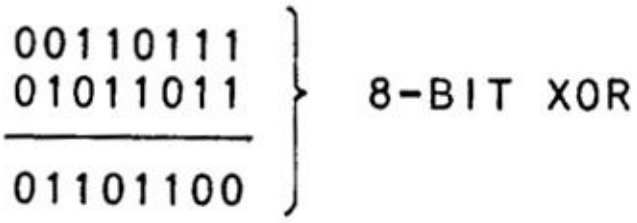


Fig 5-10. Exclusive OR (XOR) Example

The Exclusive-OR is used infrequently in both machine language and in higher-level languages. One application is shown in Fig. 5-11 where the least significant bit is used as a "toggle" bit to indicate the number of the pass -odd or even.

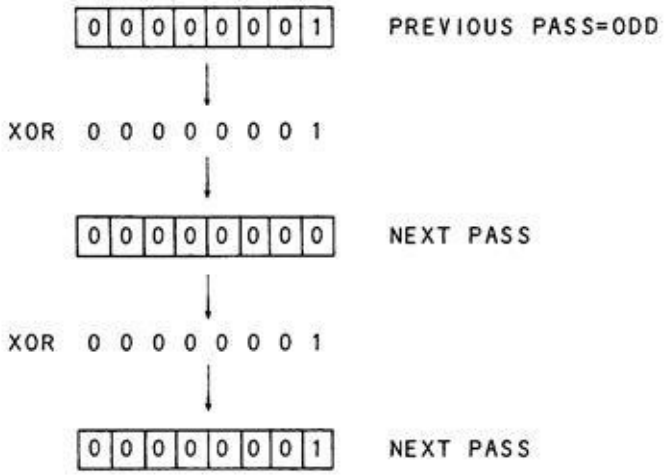


Fig 5-11. XOR Example

OTHER LOGICAL OPERATIONS

There are a number of other logical operations that are implemented in higher-level languages and in machine language. One of these is the NOT operation. NOT is similar to the **negate** operation discussed in Chapter 4, except that it forms the **one's complement**. The one's complement of a number is formed by changing all the ones to zeroes and all the zeroes to ones, *without* adding a one. What effect does this have? Let's look at an example of a signed number.

Suppose that we have the value 0101 0101. If we NOT the number, we get 1010 1010. The original value was +85. The result is a negative number which, when reconverted by the two's complement rules, comes out to 0101 0101 + 1 = 0101 0110 = -86.You might say therefore, that the NOT adds one to the number and, then, negates it. The machine-language version of the BASIC NOT operation is normally called CPL - for (One's) Complement.

The NOT operation can be used to test logical conditions in a higher-level program, as shown in Fig. 5-12. The machine-language CPL is used somewhat infrequently.

```
1010    IF NOT (PRINTER) THEN
        PRINT "NO PRINTER-BUY ONE-I'LL WAIT"
        ELSE LPRINT "RESULT=";A
```

Fig 5-12. NOT Operation

SHIFT OPERATIONS

The British restaurant party illustrated two types of shifts commonly used on microcomputers - **rotates** and **logical** shifts. These are available on a machine-language level but not generally in higher-level languages, and generally operate on eight bits of data. They are tied in with the carry flag discussed in the last chapter.

Rotates

A rotate shift was first illustrated in Fig. 5-2 with Table A in the restaurant. Now, let's look at Fig. 5-13, where the data is rotated right or left, one bit position at a time. Although larger computers will allow any number of shifts with one instruction, some microprocessors or mic rocontrollers allow only one shift with each instruction.

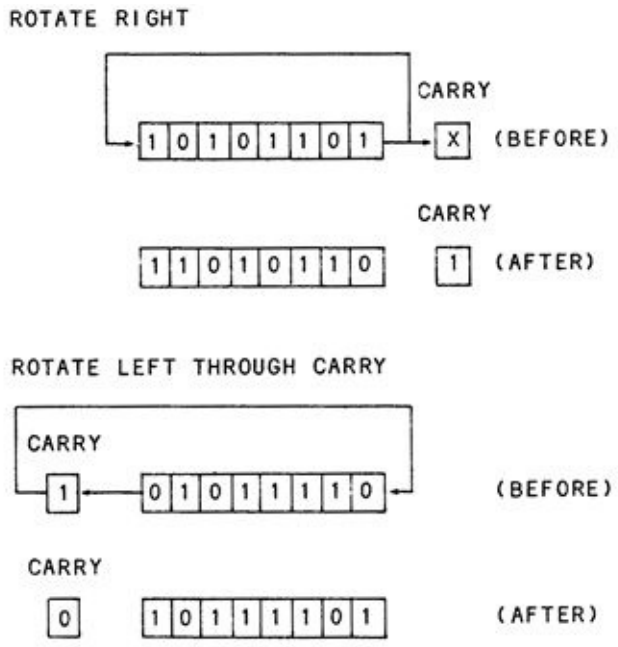


Fig 5-13, Two Rotate Shifts

As the data is rotated out the end of the microprocessor register or memory location, it either goes back into the opposite end of the register or memory location, or goes into the carry flag. If the data goes through the carry, it is really a **9-bit rotate**. If the data bypasses the carry, it is an **8-bit rotate**. In *either case*, the bit shifted out always goes into the carry as shown in Fig. 5-13.

The carry flag can be tested by a conditional jump instruction on a machine-language level to effectively test whether a one bit or zero bit was

shifted out. The rotate shift is used to test a bit at a time for such operations as multiplication (see next chapter) or the alignment of data for **masking** with an AND.

Logical Shifts

The second type of shift that was illustrated in the restaurant anecdote was a **logical** shift. The logical shift is not a rotate. Data falls off the end into the bit bucket, just as the scientist left the restaurant. As each bit is shifted out, though, it does go into the carry so that the carry always holds the result of the shift. The opposite end of the register or memory location is filled with zeroes as each bit is shifted. Here, again, one bit position at a time is shifted. The logical shift operation is shown in Fig. 5-14.

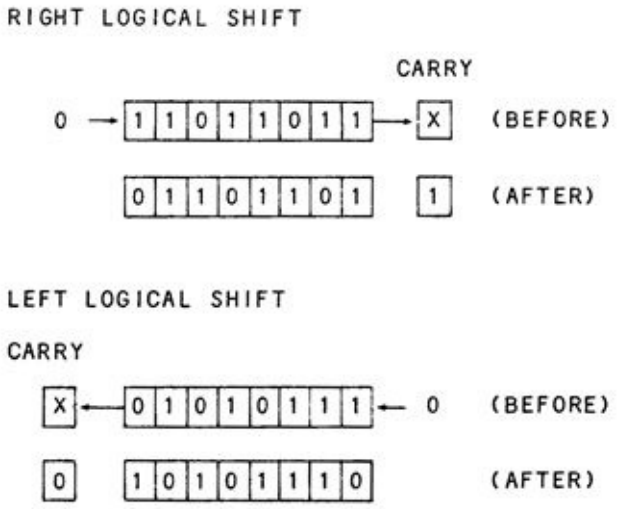


Fig 5-14. Two Logical Shifts

We couldn't really say much about the rotate in regard to what happened to the contents arithmetically. This was because the data recirculated back into the register and, in an arithmetic sense, the results were not predictable.

In the case of the logical shift right or the logical shift left, however, the results are very predictable. Let's look at some examples. Suppose that we have the value 0111 1111 with the carry set to some value. We'll express the register and carry by nine bits with the carry on the right, as in 0111 1111 x. After a shift-right logical operation, we have 0011 1111 1. The original value was + 127. After the shift, the value is +63 with the carry set. It appears that a logical right shift divides by two and puts the remainder of 0 or 1 into the carry! This is true, and the shift-right logical operation can be used any time that a divide by 2, 4, 8,

16, or other power of two is called for.

How about a shift left? Absolutely right! A shift-left logical operation multiplies by 2. As an example, consider x 0001 1111, where x is the state of the carry flag. After a logical shift left, the result is 0 0011 1110. The original number was +31 and the result is +62, with the carry reset by the most significant bit. The logical shift left can be used any time that a number is to be multiplied by 2, 4, 8, or any other power of two.

Arithmetic Shifts

There is a problem that arises when a logical shift is used with signed numbers. Consider the case of the number 1100 1111. This is a value of -49. When the number is shifted right in a logical shift, the result is 0110 0111, representing the value + 103. Obviously, the shift did not divide the signed number of -49 by 2 to produce a result of - 24.

To solve this problem of shifting arithmetic data, an **arithmetic shift** is often implemented in microcomputers. The arithmetic shift **extends** the sign as it shifts right, so that the shift is (almost) arithmetically correct. If an arithmetic shift were used in the preceding example, the result would be as shown in Fig. 5-15.

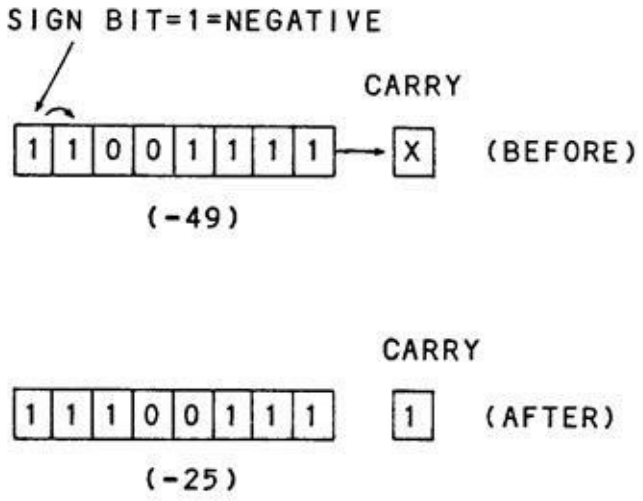


Fig 5-15. Arithmetic Right Shift

What about arithmetic left shifts? In some microcomputers, the arithmetic left shift retains the sign bit and shifts the next most significant bit out of the register and into the carry as shown in Fig. 5-16. In other microcomputers, there

is no true left shift. ln the next chapter, we'll see how shifting can be used to implement many different types of multiplication and division **algorithms**. In the meantime, try your hand with the following test questions.

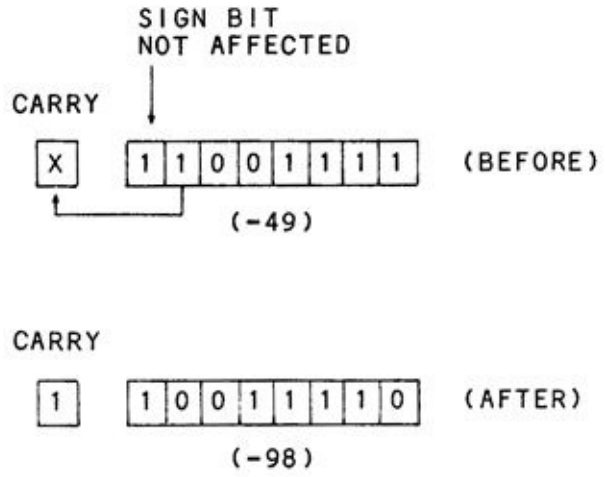


Fig 5-16. Arithmetic Left Shift

EXERCISES

1. OR the following sets of 8-bit binary operands.

```
  10101010
OR 00001111
-
  10110111
OR 01100000
```

2. Exclusive-OR the following sets of 8-bit binary operands.

```
  10101010
XOR 00001111
-
  10110111
XOR 01100000
```

3. Bits 3 and 4, of a location in memory, hold a code as follows: 00 = BROWN HAIR, 01 = BLACK HAIR, 10 = BLONDE HAIR, 11 = NO HAIR. Using an AND operation, show how these bits could be put into an 8-bit result alone. The location holds XXXYYXXX, where X = unknown bit and Y = code bit.

4. Negate the following signed operands. Show their decimal equivalents:

00011111, 0101, 10101010

5. Perform a left rotate on these operands:

00101111, 10000000

6. Perform a right rotate on these operands:

00101111, 10000000

7. Perform a left rotate with carry on these operands and carry:

C =1 00101111, C = O 10000000

8. Perform a right rotate with carry on these operands and carry:

00101111 C = 0, 10000000 C = 1

9. Perform a right logical shift of the following operands. Show the carry after the shift and the decimal values of the operands before and after the shifts:O1111111, 01011010, 10000101, 10000000

10. Perform a left logical shift of the following operands. Show the carry after the shift and the decimal values of the operands before and after the shifts:

01111111, 01011010, 10000101, 10000000

11. Perform an arithmetic right shift on the following operands and show the decimal values before and after the shift:

01111111, 10000101, 10000000

CHAPTER 6

Multiplication and Division

Many simple microprocessors do not have "built-in" multiply and divide instructions. Even some more advanced microprocessors or microcontrollers have limited multiply and divide instructions. As a result, multiplication and division often have to be done in *software* routines, in machine language. In this chapter, we'll look at some of the ways that multiplication and division can be implemented in software.

ZELDA LEARNS HOW T0 SHIFT FOR HERSELF

An engineer from lnlog had just appeared at the cash register.

"Hi, Don. How was the lunch?" asked Zelda, the serving person.

“Fine, just fine. Well, the meat was a little tough .... "

"That always happens when it sits around for so long," said Zelda, as she took the bill. "Let's see, a cup of coffee … one leftover meatloaf sandwich … and twelve lo-cal desserts .... "

As Zelda read off each item she performed some type of operation out of sight beneath the counter. On the last item, twelve lo-cal desserts, she spent a great deal of time.

”Zelda, what are you doing down there?" asked the engineer.

"Well, Don, Big Ed wants us to get used to working in binary, what with this restaurant being in the center of computer chip manufacturers and all. He wants us to figure out all of the checks in binary to gives practice. I'm okay when it comes to adding and subtracting in binary, but multiplying gives me some trouble.”

"What method are you using, Zelda? Maybe I can help."

"Every time I multiply, l use successive addition. Like in this case, where you had the twelve lo-cal desserts. Now, they cost 65 cents each, so I add 1100, which is twelve in binary, 65 times."

"Well that certainly will work, all right," said Don, as he suppressed a giggle. "That successive addition method is accurate, but it takes a good deal of time. Let me show you a faster method; one called **shift and add**."

He quickly snapped the tablecloth off of a nearby table, leaving the silverware and accessories relatively unmolested. He drew out an engineer's mechanical pencil. "Pity we don't have any quadrille paper, but these small checks in the pattern will have to do."

"Let's take that 65 cents per lo-ca1 dessert for twelve items example. First of all, we'll draw two registers. **The partial product** register is sixteen bits wide like so." (See Fig. 6-1 .) "The **multiplicand** register is eight bits wide like so.

Now, into the upper eight bits of the partial product register, we'll put the twelve, the **multiplier**. We've padded it out to eight bits to make it 00001 100. The remainder of the register we'll zero out. Next, we'll put the multiplicand into the multiplicand register.

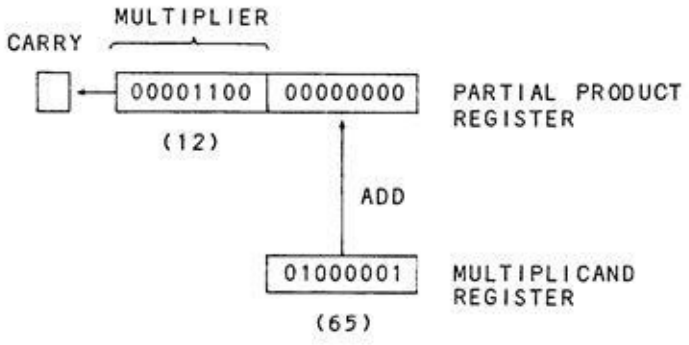


Fig 6-1. Shift and Add Multiplication

"Now, we're all set to do the multiplication. We'll have eight steps for this. We software engineers call them *iterations*. For each iteration, we'll shift the multiplier one bit position to the left. The bit shifted out will go into the carry. If the bit in the carry is a 1, then we'll add the multiplicand to the partial product register. If the bit in the carry is a 0, we won't do the addition. At the end of eight iterations, we'll be done."

"But, won't the add to the partial product register *clobber* the multiplier in the upper eight bits?" asked Zelda, obviously proud that she had picked up some computer jargon.

"No, it won't. Remember that data is being shifted out to make room for the possibly expanding partial product. After eight iterations, it will have shifted out entirely, and the partial product register will now have the final product of the multiplication. Look, I drew out all the iterations for this case." (See Fig. 6-2.)

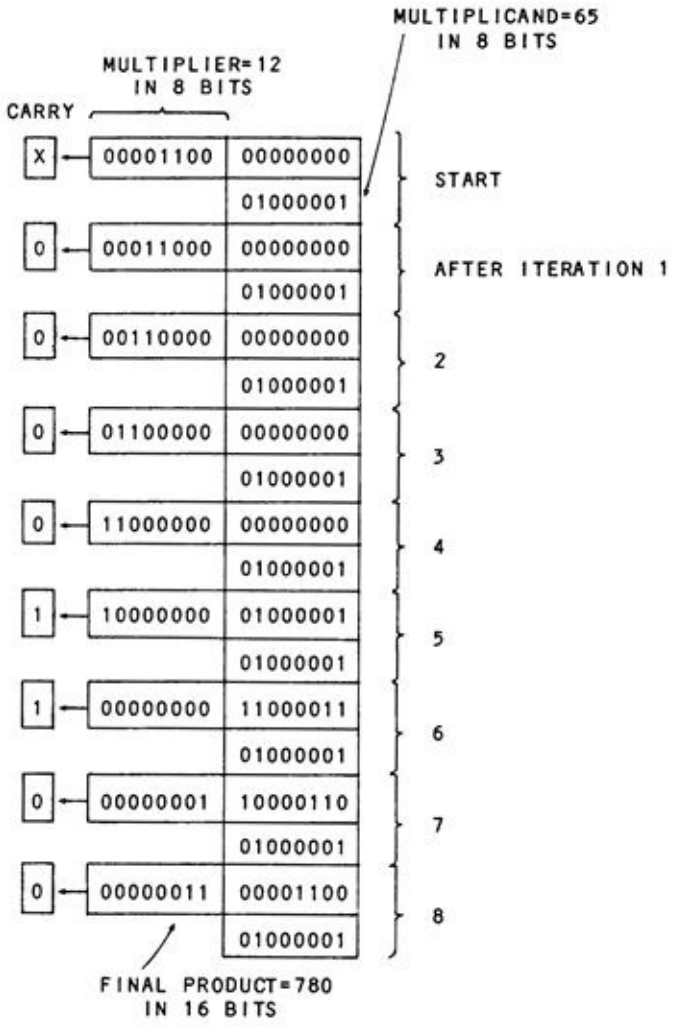


Fig 6-2. Shift and Add Multiplication Example

"Oh, yeah. Gee thanks, Don. I'll use this for sure. Now, let me figure out the rest of your bill. The total is 15.63 plus sales tax. That's 1563 cents divided by 100 times six. Let's see 01100100 from 011000011011 leaves 010110110111 - that's once. 01100100 from 010110110111 leaves 010101010011- that's twice. 01100100 from ...."

MULTIPLICATION ALGORITHMS

Successive Addition

Zelda was using a straightforward method of multiplication called **successive addition** (Fig. 6-3). In it, the multiplicand (the number to be multiplied) is multiplied by the multiplier. The process is done by zeroing a result, called the partial product, and adding the multiplicand to the partial product for the number of times that is equal to the multiplier. The preceding example was 65 times 12, which Zelda solved by adding 12 to the partial product 65 times.

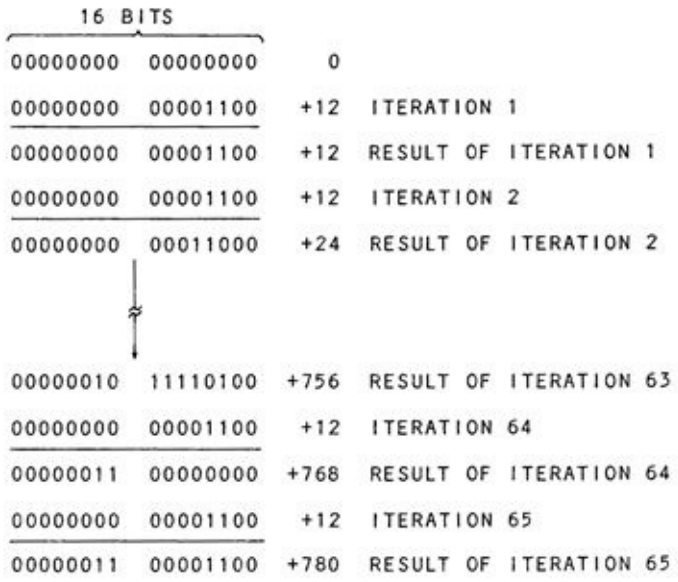


Fig 6-3. Successive Addition Multiplication Example

Although this method is straightforward, it is *very* time-consuming in the average case. Suppose that we're working with an "eight by eight" multiplication. An 8-bit by 8-bit multiplication produces a 16-bit product, maximum. The average multiplier is 127, if an unsigned multiplication is done. This means that in the average case, 127 separate additions to the partial product would have to be done.

Contrast this with Don's **shift and add** technique of eight iterations! In the worst case, the successive addition will involve 255 additions and, in the best case, one addition. The successive addition multiplication is best left for cases where the multiplier is fixed at some low constant value, below 15 or so, or where infrequent multiplications are required.

Successive Addition of Powers of Two

The powers of two successive addition method is a multiplication method that is often used when the multiplier will be a fixed value. Suppose that we must always multiply an amount by ten. Ten can be broken up into a number of sums. One combination of these is 8 + 2. As we saw in the last chapter, logically shifting to the left multiplies a value by two. To multiply by ten, a combination of shifts and additions can be done to effect the multiply. The operation goes like this:

1. Shift the multiplicand by two to get two times X.

2., Save this value as "TWOX."

3. Shift the multiplicand by two to get four times X.

4. Shift the multiplicand by two to get eight times X.

5. Add in "TWOX" to the multiplicand to get ten times X.

This procedure is shown in Fig. 6-4, for a multiplier of 10. It can be used any time the multiplier is fixed. Multiplying by 35, for example, could be done by five shifts of the multiplicand to get 32X, and an addition of 2X and 1X - 32 + 2 + 1 = 35.

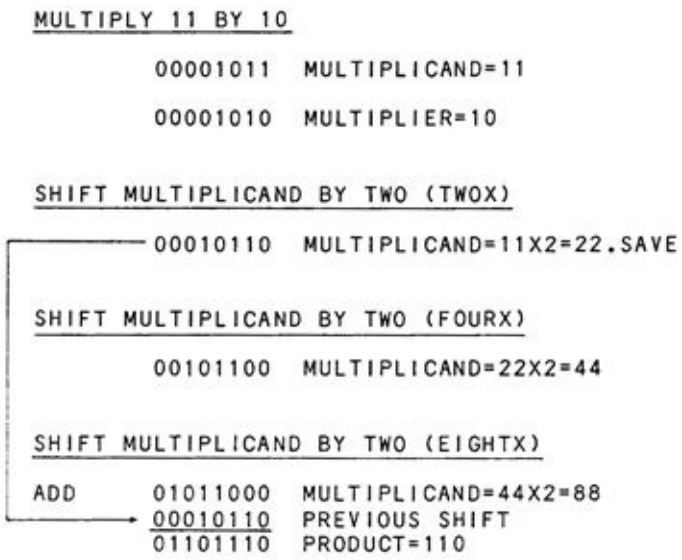


Fig. 6-4. Successive Addition Using Powers of Two

Shift and Add Multiplication

The shift and add multiplication that Don showed Zelda is the most

commonly used method for computers that do not have a built-in *hardware* multiplication capability. It emulates a paper and pencil technique that closely follows the normal decimal multiplication method that we use. For example, Fig. 6-5 shows a paper and pencil binary multiplication that is similar to a decimal multiplication. The only real difference between the paper and pencil binary method and the shift and add binary method is in the shifting. With paper and pencil, the multiplicand is shifted to the left and then added. With shift and add, the multiplicand is stationary and the partial product is shifted, as shown in Fig. 6-2.

```
    01000001    MULTIPLICAND (65)
    00001100    MULTIPLIER (12)
  ------------
   0100000100
  01000001
  ------------
  01100001100   PRODUCT (780)
```

Fig 6-5. Paper and Pencil Binary Multiplication

This shift and add technique can be used for any size multiplier and multiplicand. The rule for the size of the product is this: *The product of a binary multiplication can never be larger than the total number of bits in the multiplier and multiplicand.* If the multiplier and multiplicand are eight bits and eight bits, allow sixteen bits for the product. If the multiplier and multiplicand are twelve bits and eight bits, allow twenty bits for the product, and so forth. Another interesting fact about shift and add multiplication is that the shifting can be in reverse. We can work with multiplier bits starting from the **least significant** end! In this case, we can stop the process when the multiplier is zero, which means that we only have to perform as many iterations as there are significant bits in the multiplier. This reduces the average time of multiplication to one-half the maximum multiplier value. This efficient method is shown in Fig. 6-6.

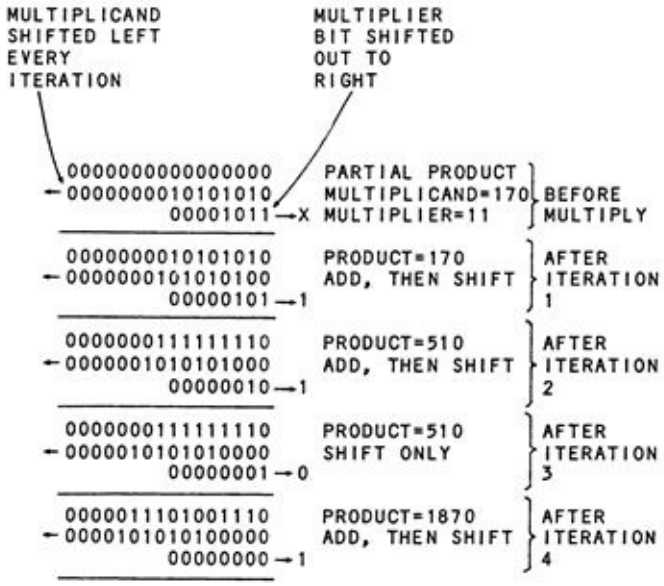


Fig 6-6. Example of the “Multiply until Zero Multiplier” Method

SIGNED VS UNSIGNED MULTIPLICATION

ln all of the preceding examples, we've been considering *unsigned* multipliers and multiplicands. The products formed in these cases are absolute numbers without a sign bit. For example, multiplying 1111 1111 (255) by 1011 1011 (187) produces a product of 1011 1010 0100 0101 (47,685) where the most significant bit is 2 ^ 15, and not a sign bit.

What about *signed* multiplication, where the multiplier and multiplicand are signed two's complement numbers? In this case, the numbers can be converted to their **absolute values**, the multiplication can proceed, and the product can then be converted to the proper sign. Of course, a plus quantity times a plus quantity is a plus, a minus times a plus is a minus, and a minus times a minus is a plus quantity, just as in decimal arithmetic.

Here's a good example of one of our logical operators, the Exclusive-OR. If an Exclusive-OR is taken of the multiplicand and multiplier, then the most significant (sign) bit of the result will be the sign of the product! For example,

1011 1010 (-70)
0000 1010 (+10)
1011 0000 (XOR)

produces an XOR whose most significant bit is a 1; therefore, the product will be negative.

The algorithm for a signed multiplication is this:

1. XOR the multiplier and multiplicand. Save the result.

2. Take the absolute value of the multiplicand by changing a negative number to positive, if necessary.

3. Take the absolute value of the multiplier by changing a negative number to positive, if necessary.

4. Multiply the two numbers by the standard shift and add method.

5. If the result of the XOR has a 1 bit in the sign, change the product to a negative number by the two's complement method of complementing it (negate operation).

There is one minor problem in the preceding method. Overflow was not possible in the unsigned method, but it is possible in the signed method, for one case only. When both the multiplier and multiplicand are maximum negative values, the product will overflow. For example, if the multiply is eight by eight and the multiplier and multiplicand are both 1000 0000 (-256), then the product will be +65,536, which is too large to be held in sixteen bits. This condition can be tested before the multiplication takes place.

DIVISION ALGORITHMS

Software division algorithms are somewhat harder to implement than multiplication algorithms. One of the methods of division is successive subtraction, the one that Zelda was using as we left her. A second is a bit-by-bit "restoring division."

Successive Subtraction

Successive subtraction is shown in Fig. 6-7. In this method, the **quotient** is cleared to zero initially. The **divisor** (the operand that “goes into" the **dividend**) is subtracted from the dividend successively until the dividend "goes negative." Each time the divisor is subtracted, the quotient count is incremented by one. When the dividend goes negative, the divisor is added once to the **residue** to restore the remainder. (The residue is the amount of dividend remaining.) The count is the quotient of the division while the remainder is in the residue, as shown in Fig. 6-7.

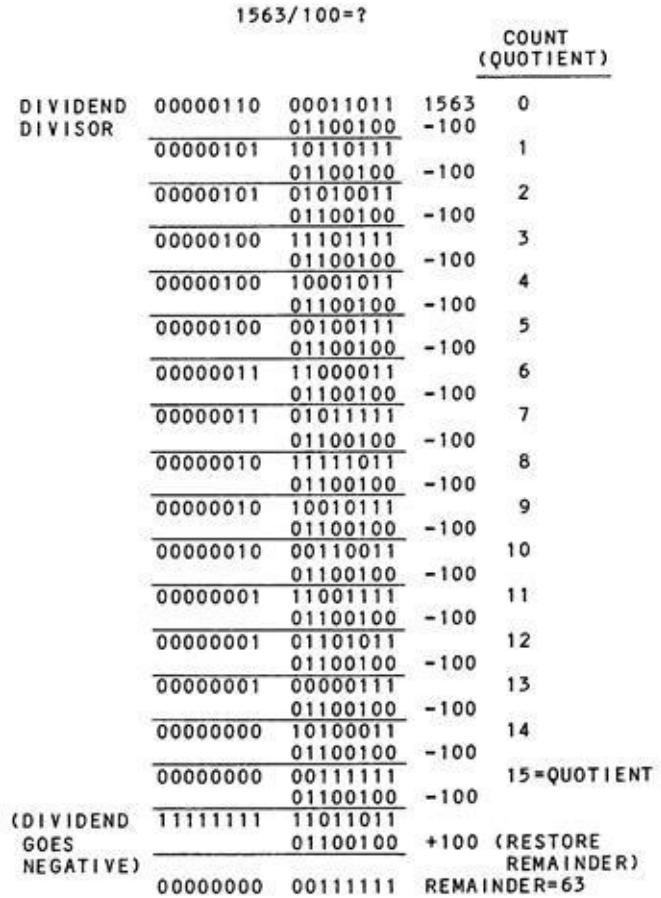


Fig 6-7. Successive Subtraction Method of Division

As in the case of successive addition for multiplication, the successive subtraction is very slow for the average case. If we are working with a 16-bit dividend and an 8-bit divisor, the average quotient is 255. A total of 255 subtractions is just as intolerable as in the multiplication case. Successive subtraction for a division is fine where the size of the divisor is large compared to the size of the dividend; for example, if the divisor were 50 and the dividend was a maximum of 255, then only five subtractions would have to be done in the worst case, which would make this an efficient division.

Restoring Division

The answer to a relatively fast general-purpose division technique lies in a bit-by-bit divide. The restoring division method emulates the way we divide by paper and pencil. Fig. 6-8 shows the scheme. We are dividing a 16-bit dividend with an 8-bit divisor. The general rule for the size of the quotient, by the way, is that it must be equal to the number of bits in the dividend, as the value 1 is a legitimate divisor.

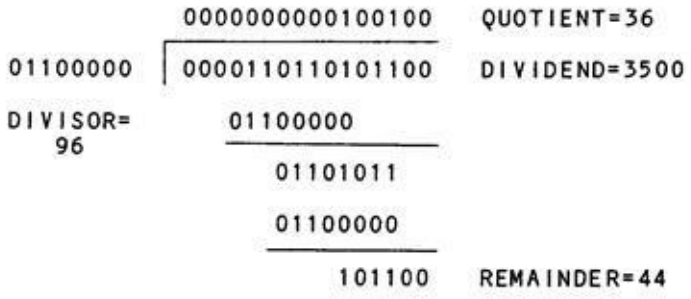


Fig 6-8. Bit by Bit Division Implementation

We start with the two absolute values (positive numbers) of + 96 for the divisor and +3500 for the dividend. First of all, 0110 0000 (96 is subtracted from the first 0 of the dividend of 0000 1101 1010 1111. Of course, this subtraction is impossible, and the result is negative. If the result is negative after any subtraction, we *restore* the previous residue by adding back the divisor, and we do so in this case. We continue this subtraction, test, and the restore/nonrestore for each of the sixteen bits in the dividend. Any time the result is negative, we restore by adding back the divisor and put a 0 in the quotient bit. Any time the result is positive, we do not restore and put a 1 bit in the quotient bit. At the end of sixteen iterations, the quotient holds the final value, and the residue is the remainder of the operation. The residue may have to be restored by one final add

to obtain the true remainder.

This paper and pencil technique can be almost exactly duplicated in the microcomputer. The dividend is put into a 24-bit register (three 8-bit registers). The divisor is put into an 8-bit register. The two registers are aligned as shown in Fig. 6-9. The divisor is subtracted from the upper eight bits of the dividend. A restore is done by adding back the divisor if necessary.

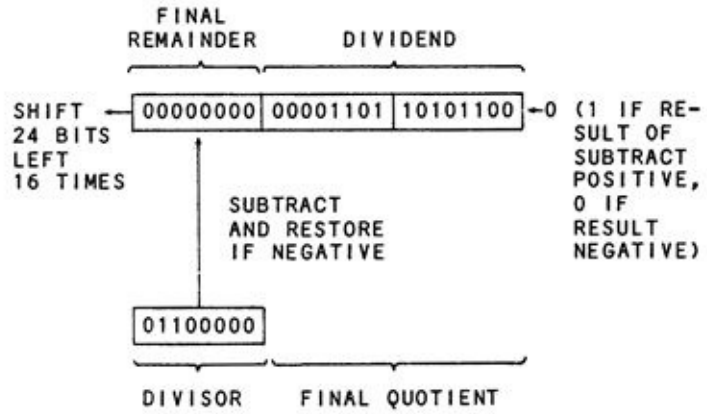


Fig 6-9. Bit by Bit Division Implementation

After the subtraction and possible restore are done, the dividend is shifted left one bit position. At the same time that the shift occurs, the quotient bit of 0 or 1 is shifted left into the dividend register from the right end. At the end of sixteen iterations, sixteen quotient bits will have been shifted into the dividend register, and they will be in the lower sixteen bits. The upper eight bits will hold an 8-bit remainder, provided that any final restore has been done.

Signed Vs. Unsigned Divisions

All of the preceding divisions were of unsigned operands. As in the case of multiplication, the easiest route for a signed division is to find the *sense* (positive or negative) of the final product, take the absolute value of the dividend and divisor, and then perform an unsigned division, changing the quotient to a negative value if a plus quantity was divided by a minus quantity, or a minus was divided by a plus.

There is one minor problem. When -32,768 is divided by -1, the result of +32,768 will overflow the quotient. A check can easily be made of this condition before the division.

In the next chapter, we'll consider **multiple-precision** arithmetic

operations that will allow us to work with greater values than can be held in sixteen bits. Before we get on to this topic, however, the following exercises are in search of readers who want to sharpen their skills in multiplication and division algorithms and calculations.

EXERCISES

1. What is the result of the unsigned eight-by-eight multiplication of 11111111 and 11111111? What are the equivalent decimal values?

2. What is the result of the unsigned division of 1010101011011101 by 00000000? What is the result of the unsigned division of 1111111111111111 by 00000001? What conditions cannot be allowed in computer division?

3. If all operands are signed numbers, what will be the sign of the result of these operations:

10111011 x 00111000 =? 1011011100000000/10000000 = ?

CHAPTER 7

Multiple Precision

Many times it is necessary to work beyond the precision offered by eight bits or sixteen bits. Real-world quantities such as physical constants, accounting data, and other numeric values often exceed the+32,767 to -32,768 range of even 16-bit quantities. Larger values can be expressed by using a number of bytes to hold each operand. In this chapter, we'll see how this can be done by using 8-bit bytes.

IS THE FIBONACCI SERIES RELATED TO ITALIANSOCCER?

" 'Scusa, me. Is this-a Big Ed's Ristorante?"

Big Ed turned around to see a smiling man with a large notebook just entering his restaurant. "Yes, sir, it is. What can I get you?"

"I'd like-a some lasagna and Chianti, if-a you please."

"Certainly, sir. By the way may I ask you to dispense with the Italian accent? It makes the writer very nervous. He's not too good with foreign accents . .”

"Oh, I'm sorry. I have to use it on lecture tours. I guess I’ve fallen into bad habits."

"Lecture tours?"

"Yes, one of my distant relatives was Leonardo of Pisa, also known as Fibonacci, the great Italian mathematician. I'm carrying on his work. That’s why I'm here in Silicon Valley. I want to use a microcontroller to help me analyze the Fibonacci Series. I was a little dismayed to find out that they normally don't have enough precision to do this."

"What do you mean by that? I thought computers could handle any size number."

"Well, they can handle *approximately* any size number by **floating-point** operations, but that doesn't give you exact precision. For example, the value 122,234,728,956 would probably only be kept in a higher-level language as 122,235,000,000 with the rest of the digits lost. I need exact precision for my work on the Fibonacci Series."

"I really don't follow soccer .... "

"No, you-a see, sorry .... You see, the Fibonacci Series goes like this: If you take the numbers 1 and 1 and add them, you get 2. That's the third term in the series. Now take the 1 and 2 and add them to get 3.That's the fourth term. Now add 2 and 3 to get 5. That's the fifth term. Well, if you keep on going like that, you'll have the simple series of 1,1, 2, 3, 5, 8, 13, 21, 34, 55, and so forth."

"Well, that seems simple enough. Is it good for anything?”

"It has certain applications in mathematical modeling and in nature, but the main interest about the series seems to be with a hard core of people who delight in mathematical games. There are literally tens of thousands of people around the world investigating the properties of the Fibonacci Series. Why, in my logo lecture alone, 134 engineers, programmers, and scientists showed up, in addition to one gentleman demanding the lecture hall rental fee!"

“To make a long story short, though, my main problem in working with the series is that the terms get very big, very fast. The twenty-fifth term is 75.025, too much for sixteen bits to handle. I had to go to multiple precision to process the larger terms of the series."

"Multiple precision? How does that work?"

"Well, as you probably know from associating with all of the computer manufacturing people around here, eight bits will hold values up to 255 and sixteen bits will hold values up to 65,535, if the numbers are unsigned, of course. I wanted to hold numbers up to 18,446,745,073,709,548,616 - that would cover all Fibonacci numbers up to almost the one-hundredth term."

"You’d need hundreds of bytes to do that!" exclaimed Ed.

"Not really-four bytes hold 4,294,967,296, six bytes hold 281,474,976,710,656, and eight bytes hold 18,446,745,073,709,548,616. So you can see that if I had a program capable of dealing with only eight bytes, I’d have more than enough precision. All that program has to do is to work with eight-byte operands for addition and subtraction. I finally found a software house that specialized in programs for computing large numbers in microcomputers .... "

"Yes, I'm all ears .... "

"Alas, alack, they had gotten a huge federal contract to process federal budget deficits. They're now in Washington and I'm here, looking for another consulting company .... “

ADDITION AND SUBTRACTION USING MULTIPLE PRECISION

Multiple precision is not generally used to handle large numbers (floating-point, as covered in Chapter 10, is used instead), but it does come in handy for certain types of processing, such as Fibonacci's problem, high-speed operations, or high-precision operations.

In multiple-precision addition and subtraction, two operands that are a number of bytes long are added or subtracted together. What is the appearance of the operands? Suppose we have determined that we want to represent numbers up to a size of one half of 18,446,745,073,709,548,616. We know from the Fibonacci conversation that this magnitude could be held in a signed number of eight bytes, or 64 bits. The value of + 9,223,372,536,354,779,313 would be represented by

```
01111111  11111111  11111111  11111111
11111111  11111111  11111111  11111111
```

The bits would be numbered in our standard representation of powers of two, with bit 0 on the right-hand side and bit 63 on the left-hand side as a **sign bit**. The value of -9,223,372,536,354,779,314 (note that this is one more in magnitude than the positive number)would be represented by

```
10000000  00000000  00000000  00000000
00000000  00000000  00000000  00000000
```

Other values would be between these two limits. Note that there is only one sign bit and that the number is treated as a single group of 64 bits, even though it is *physically separated* into eight bytes.

The addition of two operands in this eight-byte precision involves taking the bytes of each of the operands from right (least significant) to left (more significant) and adding them together. Any carry from the addition of the last, less significant byte must be added in.

While carries **propagate** to the next bit position inside each byte as a normal result of the addition, any carry to the next bit position affects the next higher byte, and the carry must be saved and added in. The carry flag is used to

record any carry from the previous addition, and an *add with carry* instruction is used to add-in the previous carry.

For an eight-byte operand, the first addition is a simple "add" (there is no previous carry), while the next seven additions are add with carry" instructions. Fig. 7-1 shows this operation for a sample addition of two 8-bit operands.

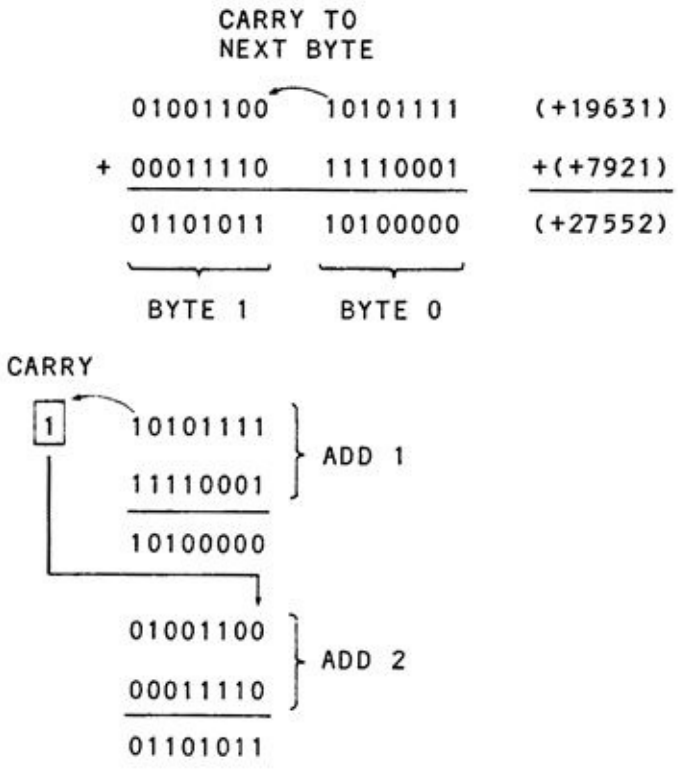


Fig 7-1. Two-byte Multiple-Precision Addition

A multiple-precision subtraction works in much the same way except that a **borrow** from the next higher byte is used instead of a carry. The first subtraction is a plain subtraction, while the remaining subtractions are *subtract with borrow* operations, commonly called subtract with carry operations. Fig. 7-2 shows the subtraction operation for two eight-bit operands.

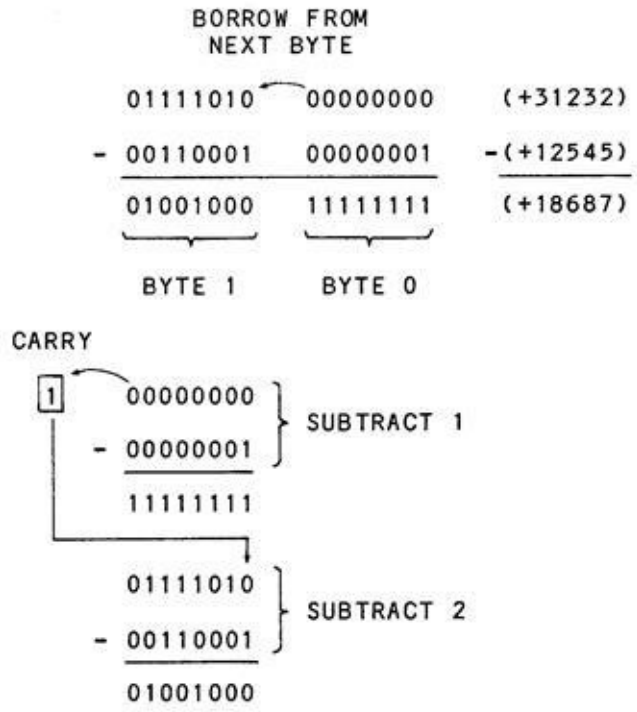


Fig 7-2. Two-Byte Multiple-Precision Subtraction

The preceding addition and subtraction operations work fine with any mixture of two's complement numbers; the operands do not have to be absolute.

When multiple-precision numbers are stored in memory, one cautionary note should be added. ln older Z-80 based microcomputers, the normal memory storage of sixteen bits of data, including memory addresses and two-byte values, is the least significant byte, followed by the most significant byte, as shown in Fig. 7-3. If a multiple-precision number is stored in the most significant byte to the least significant byte fashion, there is no problem if the data is accessed one byte at a time to: (1)load the data into a register, (2) add or subtract the second operand, and, then, (3) store the result byte back into memory. If the data is loaded into a 16-bit register, however, the microprocessor will load the first bite into the lower eight bits of the register and the second byte into the higher eight bits of the register. In this case, the data must be arranged in two-byte groups ordered as the least significant byte and the most significant byte, as shown in Fig. 7-3.

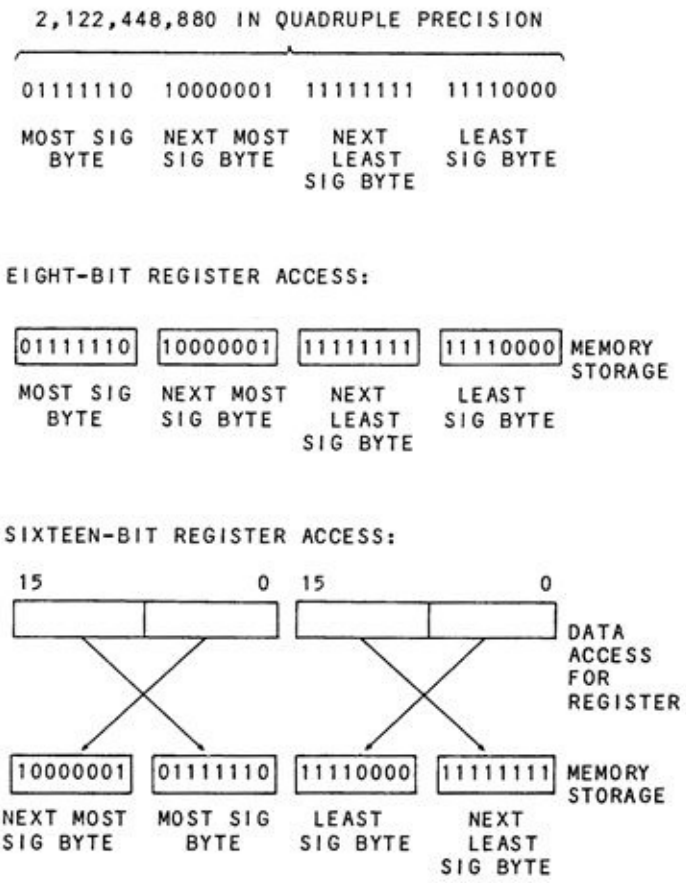


Fig 7-3. Memory Storage with Multiple Precision

MULTIPLE-PRECISION MULTIPLICATION

Multiplication using many bytes is very hard to implement. One of the reasons for this is that most microprocessors do not have wide enough registers to work with for such multiplications. About the best that can be done is to multiply a two-byte multiplicand by a two-byte multiplier to yield a four-byte product.

Another method of achieving multiple-precision multiplication is to take advantage of the expansion of (A + B) x (C + D). The expansion of (A+B) x (C+D)is A x C + B x C + B x D + A x D.

Suppose that we want to multiply the two unsigned numbers of 778 and 1066. Both of these can be held in sixteen bits each, as we know. The numbers appear as shown in Fig. 7-4. Note an interesting thing. Each number can be split into two parts - a high-order eight bits, and a low-order eight bits. The high-order eight bits of 778, for example, equal 3 x 256, and the low-order eight bits equals 10. The high-order eight bits of 1066 equal 4 x 256, and the low-order eight bits equal 42. We could express 778 x 1066 as:

778 x 1066 = ([3 x 256] + 10) x ([4 x 256] + 42)

The expansion of this is:

([3 x 256] x [4 x 256]) + (10 x [4 x 256]) +

([3 x 256] x 42)+ (10 x 42)

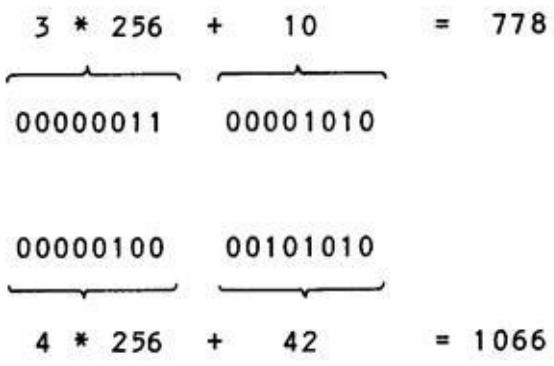


Fig 7-4. Multiple-Precision Expansion

The first term, 3 x 256 x 4 x 256, is the same as 3 x 4 shifted left sixteen bits. The second term is the same as 10 x 4 shifted left eight bits. The third term

is the same as 42 x 3 shifted left eight bits. The fourth term is a simple multiplication of 10 x 42. In fact, to compute the product, all we must do is:

1. Clear a 32-bit (four-byte) product.

2. Multiply 3 x 4 and add the result to the first two bytes of the product.

3. Multiply 10 x 4 and add the result to the second and third bytes of the product.

4. Multiply 42 x 3 and add the result to the second and third bytes of the product.

5. Multiply 10 x 42 and add the result to the third and fourth bytes of the product.

This procedure is shown in Fig. 7-5. It works, of course, not only for this example but for any 16-bit multiplier and multiplicand, and also for wider operands. Split the operands into as many parts as necessary, perform four multiplications, align the results and add, and you will have the product.

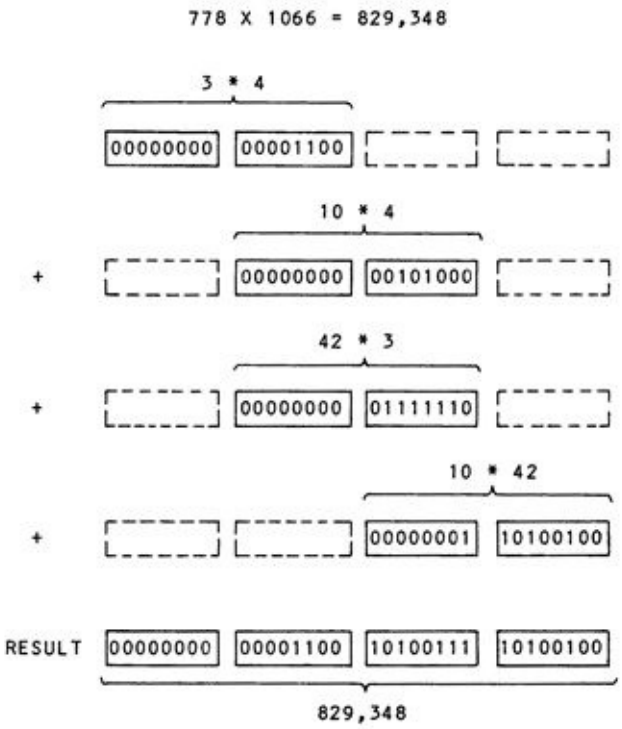


Fig 7-5. Multiple-Precision Multiplication

Unfortunately, multiple-precision divisions cannot be as easily factored.

Here again, there are usually not enough registers in the microprocessor and they are not wide enough to handle the divide of many bytes effectively. We'll look at division of larger numbers, with some loss of precision, by the floating-point method in Chapter 10. In the meantime, try the following exercises to reinforce what you have learned in this chapter.

EXERCISES

1. What is the largest value that can be held in 24 bits?

2. Add the four-byte multiple-precision operands given below:

  00010000 11111111 01011011 00111111
+00010001 00000000 11111111 00000001

3. Subtract the four-byte multiple-precision operands given below:

  00010000 11111111 01011011 00000000
-00010000 00000000 10000001 00000001

-

4. Take the two's complement of the following four-byte multiple-precision operand:

00111111 10101000 10111011 01111111

5. Perform a logical shift right of the following four-byte multiple-precision operand:

00111111 10101011 10111011 01111111

CHAPTER 8

Fractions and Scaling

We've done a lot of talking about binary numbers in the course of this book, but all of the numbers have been integer values. In this chapter, we'll see how fractions can be represented in binary notation, and how numbers can be scaled to represent mixed numbers.

BIG ED WEIGHS THE NUMBERS

"Are you the owner?" asked a chunky individual in a green tie, red and white checked jacket, beige pants, and scuffed black shoes.

"Yes, sir, I am," said Big Ed, shaking his head over the lack of a handkerchief in the vest pocket of the stranger's coat. Certainly not dressed for success, he thought to himself.

"Well, hi. I'm John Upchuck of Acme Sales. l have a sample of our new Binary Scale Mark III here which is specifically geared to restaurants like your own. This little gadget is not a slicer, not a dicer, not a cutter… er … I'm sorry, I got off on the wrong pitch. What I have here is a scale that you can use to accurately weigh your patron's meat portions. I see that you advertise "twelve ounces of utility beef" for your BigEdburger. Well, this little gadget will make certain that every Big Ed-burger is exactly twelve ounces - no more, no less."

How does it work?" asked Ed intrigued by the catchword "binary."

"Let me show you. You see, there are eight weights. The first weight is 8 ounces. The next is 4 ounces. The next is 2 ounces. The next is 1 ounce." (See Fig. 8-1 .)

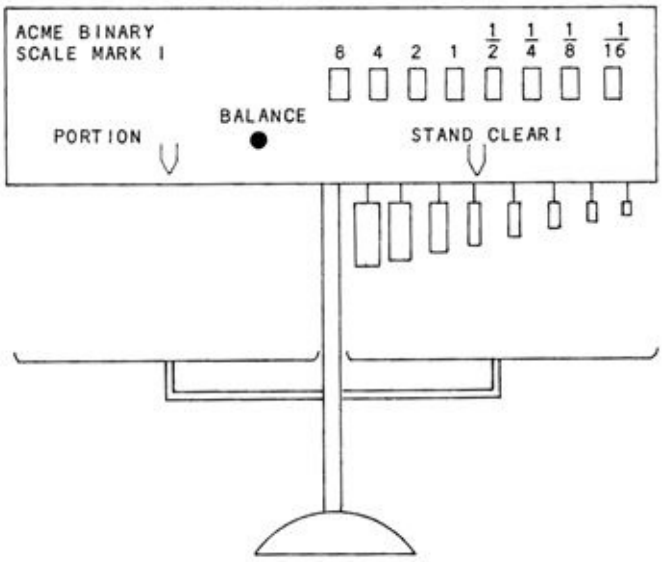


Fig 8-1.The Binary Scale

"That seems vaguely reminiscent of something ..., " mused Ed.

"Well, continuing, the next weight is 1/2 ounce, the next weight is ¼ ounce, the next is 1/8 ounce, and the last weight is 1/16 ounce. A total of eight weights, allowing you to weigh any portion of meat or vegetable between 1/16 ounce and 15 and 15/16 ounces."

”Now, the operation is simple. You put the meat or portion into the pan on the left side of the scale. You now press one or more of the buttons on the front. You can see that they are labeled 8, 4, 2, 1, 1/1, ¼, 1/8, and 1/16. Each time a button is pushed, the weight corresponding to the button is dropped onto the right-hand scale pan and a light illuminates the button. Push the same button again, and the weight is removed from the pan and the light goes out. Here, suppose that you want to weigh out 12 and 1/4 ounces of your meatloaf. Put the meat-loaf here…and, now, press the 8, 4, and 1/4 buttons.”

As he did so, three weights labeled 8, 4, and 1/4 dropped onto the pan, and the lights above the three buttons illuminated. The salesman trimmed the meat until a light marked "BALANCE" came on in the middle of the panel.

"Well, that's very nice," said Ed. "By the way, I’ve been meaning to ask - what was the Mark l model like?"

"The Mark I model was an early design. It was calibrated in units of 1/16 of an ounce. The panel was marked like this .... " He grabbed a piece of nearby tablecloth, and started drawing furiously (Fig. 8-2).

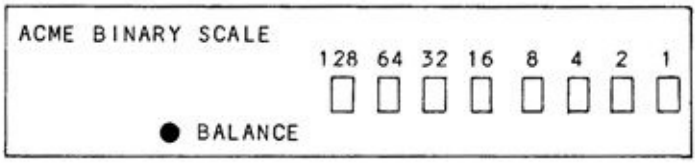


Fig 8-2. The Earlier Model

"The front panel, as you can see, was labeled 128, 64, 32, 16, 8, 4,2, 1, representing 128/16ths of an ounce, 64/16ths of an ounce, 32/16ths ounce, 16/16ths ounce, 8/16ths ounce, 4/16ths ounce, and 1/16ths ounce. However, it proved to be too complicated for nontechnical people to use, as they had to multiply the number of ounces required by 16 and, then, had to punch the button combinations. People that used the machine started referring to this process as *scaling up* in kind of a derisive way. The Model II is much easier to use. Well, would you like me to leave one and bill you later? . .

"No, not right now. Thanks much for the demonstration, though. Here, take along one of my meatloaf sandwiches - it’ll match your green tie.”

FRACTIONS IN BINARY

Binary fractions have a format similar to decimal fractions. There is a binary point instead of a decimal point, which separates the integer portion of the binary number from the fractional part (Fig. 8-3). The bit position immediately to the right of the binary point represents a weight of 1/2, the next bit position is 1/4, the next bit position is 1/8, the next 1/16, and so forth. Whereas the integer bit positions are powers of two, the fractional bit positions are reciprocals of powers of two-1/2^1, ½^2, ½^3, and so forth.

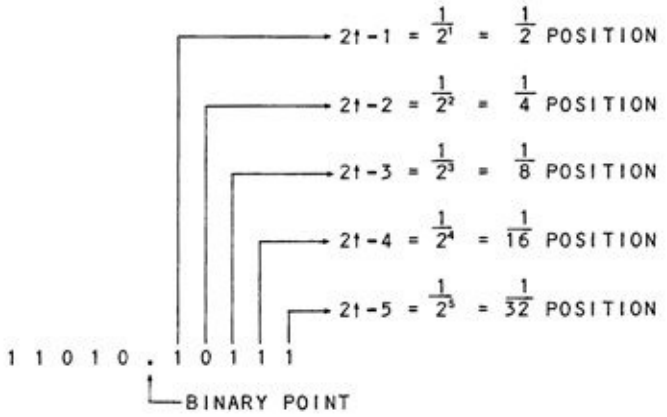


Fig 8-3. Binary Mixed Number Representation

Let's consider some more examples. The binary mixed number 0101.1010 is made up of the integer 0101, which is a decimal 5, and the fractional portion .1010. This represents 1/2 + 1/8 or 4/8 + 1/8 which equals 5/8. The total number, then, is equivalent to the decimal number 5 5/8 or 5.625.

The binary mixed number 0111.0101011 is made up of the integer 0111, or decimal 7, and the fractional part .0101011. The fractional part is 1/4 + 1/16 + 1/64 + 1/128, or 32/128 + 8/128 + 2/128 + 1/128 = 43/128.The total mixed number is, therefore, 7.3359375. To convert from a fractional binary number to decimal, convert the integer portion by any of the methods discussed previously. Next, convert the fractional portion as if it were an integer number. For example, convert 0101011 to 43 as if it were an integer. Now, count the number of bit positions in the fraction and raise the number 2 to that power. Here, the number of bit positions is 7 and 2 raised to the seventh power, or 2 ^ 7, is 128. Divide 128 into 43. Dividing 43 by 128 gives us 43/128, the same value that we obtained by adding the separate fractions. This process is shown in Fig. 8-4.

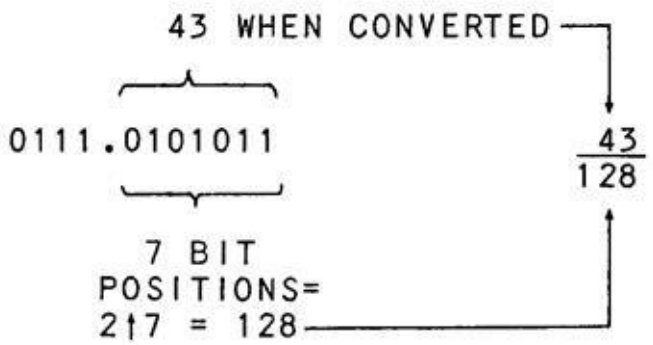


Fig 8-4. Conversion of a Binary Fraction

Working with Fractions in Binary

There are several different ways to process mixed numbers containing fractions in microcomputers. Higher-level languages, of course, use floating-point number routines, which automatically process mixed numbers, but we're primarily talking about a machine-language level here.

Keeping a Separate Fraction

The first way to handle fractions is to keep the fractional part and integer part of a mixed number separate. This scheme is shown in Fig. 8-5, where two bytes in memory hold the integer portion, and an additional one byte holds the fractional part. The binary point is permanently fixed between the integer and fractional part of the number. The integer part and fractional part are processed separately.

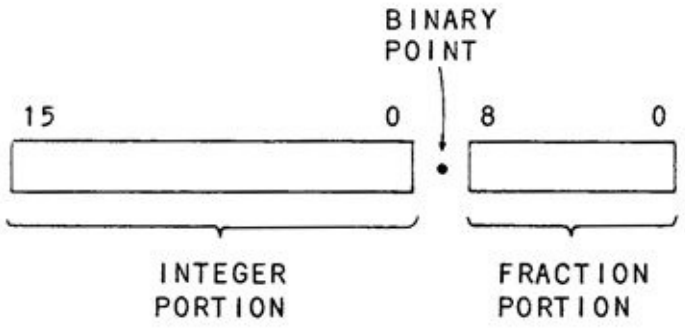


Fig 8-5. Handling Binary Mixed Numbers

The maximum positive number that can be held in this scheme is+32,767.9960…, represented by 01111111 11111111 in the integer portion and 11111111 in the fractional part. The dots behind the number indicate that the number has additional digits. The maximum negative number that can be

represented is -32,768, and is represented by 10000000 00000000 in the integer portion and 00000000 in the fractional portion. To find the magnitude of a negative number in this form, use the two's complement rule discussed earlier. Change all the zeroes to ones, all the ones to zeroes, and add one. In this case, add one to the least significant bit of the fraction And add any carry from the fraction to the next higher bit position of the integer portion.

The advantage of this method is that the fractional portion of the number is immediately available for such things as rounding to the nearest cent, if dollar amounts are being handled. In fact, the number can be treated as a 24-bit number in two parts when it comes to additions and subtractions. The fractional part is computed first, and any carry (addition) or borrow (subtraction) is carried over to the integer portion. Fig. 8-6 shows an example of an addition and subtraction using this approach.

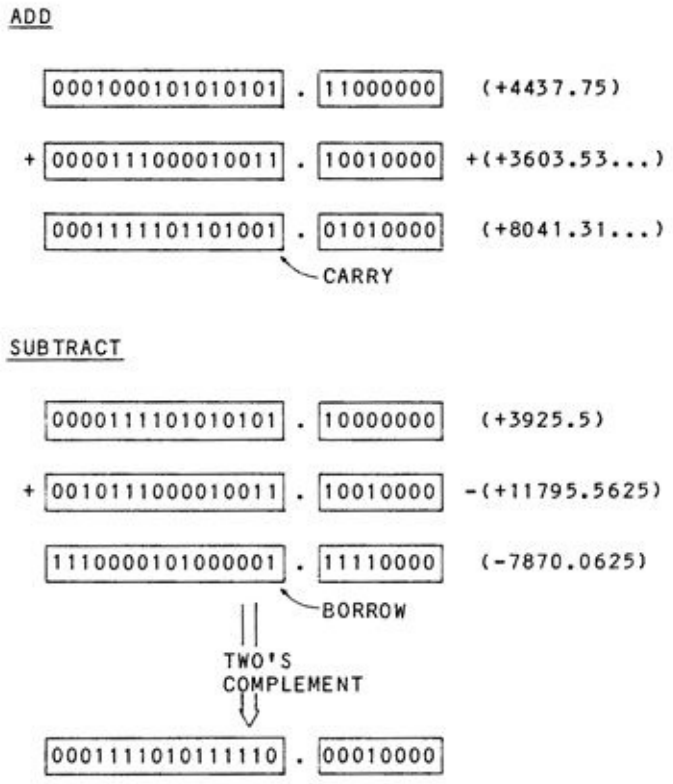


Fig 8-6. Addition and Subtraction of Mixed Numbers

Scaling

Another scheme uses a less obvious fixed point. This scheme is similar to the one used by the Acme Binary Scale Mark I. Each number is scaled up by a certain amount. The binary point, though not as well defined as when it was on a

byte boundary, is there in the programmer's mind.

Here's an example. Suppose that we are using 16-bit numbers. We decide that we want to have four fractional bits. This decision is based upon our requirements for **resolution**. We know that four fractional bits will enable us to get within 1/16 of the actual value, and this is adequate for our needs.

The layout of our "scaled up by 16" numbers will appear as shown in Fig. 8-7. There are twelve bits of integer, an invisible binary point between bits 4 and 3, and four bits of fraction. The maximum positive number that we can handle is 01111111 11111111, which is+2047.9375; the maximum negative number we can handle is-2048.0000, which is 10000000 00000000. Before we can process any numbers from the outside world, we must scale them up to this fixed-point representation. We do this by entering a valid number in the range of -2048.0000 to -2047.9375 and, then, multiplying by 16. Entering 1000.55, for example, results in 16008.8. We truncate the .8 (drop it), and enter the number 16008 as a 16-bit binary number. (The number reconverted is 1600816 = 1000.5, thus losing some accuracy, as we expected.) Other numbers are scaled up in the same fashion.

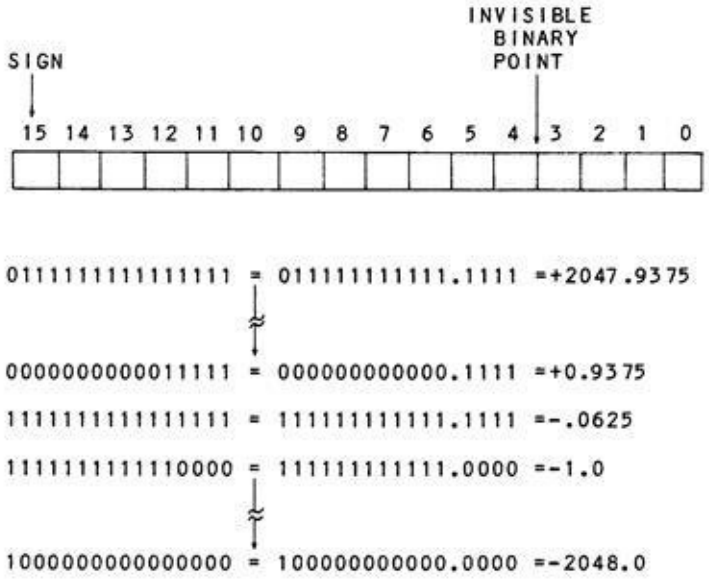


Fig 8-7. Scaling Example

We then go ahead and add, subtract, multiply, and divide those numbers in any operation that we want. Now comes the rub. If we add and subtract numbers

scaled in this fashion, the binary point remains fixed at its position with no problem. However, multiplication and division move the binary point! Examples are shown in Fig.8-8. We must keep track of the point for multiplication and division. Each multiplication moves the point of the result two more positions to the left. Each division moves the point two positions to the right.

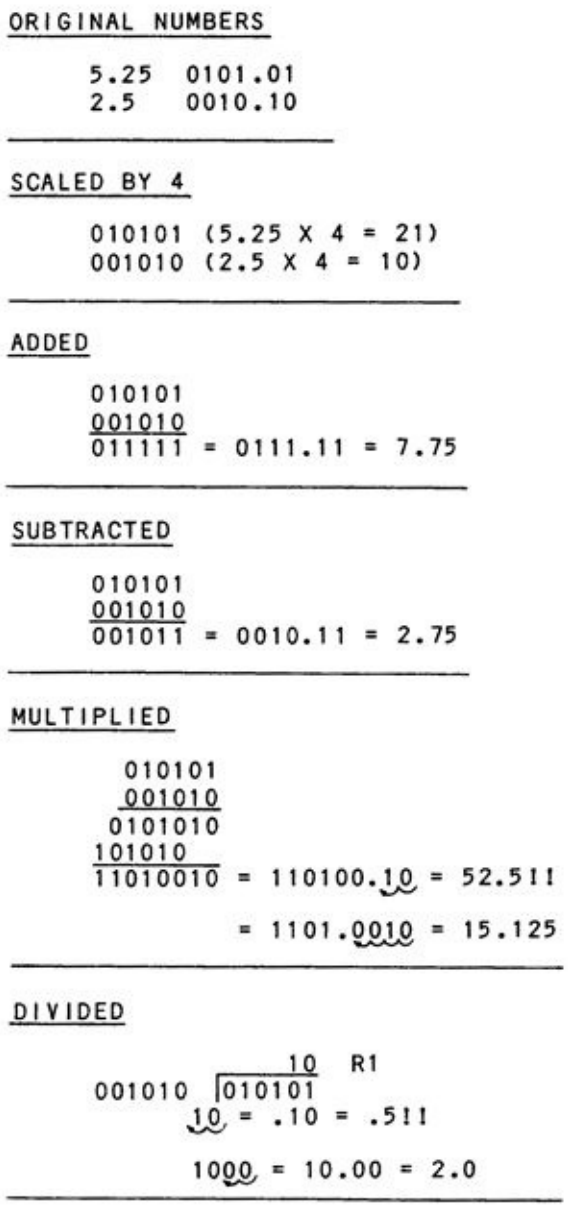


Fig 8-8. Operations Involving Scaling

At the end of the processing, after the point location has been adjusted for multiplication and division, we reconvert the numbers back into the actual values

by dividing by 16. Since multiplying by 16 and dividing by 16 can easily be handled using four one-bit shifts, the conversion and reconversion are easy. Typical processing for numbers in this format is shown in Fig. 8-9.

Fig 8-9. Processing Scaled Numbers

Although having to keep track of the binary point is tedious, the advantage of this method is that we can handle both the fractional and integer portions of mixed numbers at the same time without having to process the fractional portion separately and propagate the carry to the integer portion. Use this approach for any processing that requires a fixed set of operations that are few in number.

CONVERSION OF DATA

If you have read this chapter carefully, you'll probably have a significant question at this point. How do we convert the data in character format from the outside world into binary form, scaled or not, and then reconvert it back to a form suitable for display on a video display or printer? We’ll answer that question in the next chapter. ln the mean-time, to keep you honest, we've included some self-testing exercises on fractions and scaling.

EXERCISES

1. What are the equivalent decimal fractions of:

.10111, .1, .1111, .0000000000000001

2. Convert the following decimal fractions to binary values:

.365, .3125, .777

3. What are the equivalent decimal numbers for the following signed binary mixed numbers:

00101110.1011, 10110111.1111

4. Scale up these numbers by 256 and show the result as 16-bit signed binary numbers:

100, -99

5. The following signed numbers are scaled up by eight. What decimal mixed numbers are represented?

001011100, 1010110101101110

CHAPTER 9

ASCII Conversions

So far we've done a lot of talking about processing binary data in various formats. How, though, does the data get into the computer? Certainly, some of it is stored in the program as constant data, but other data must be input from the real world from a keyboard or terminal and, then, output back for display or printing. ln this chapter, we’ll discuss how data is converted between its real world form and its computer representation.

BIG ED AND THE INVENTOR

Big Ed had just opened the restaurant. He was busy getting ready for the onslaught of hungry engineers, programmers, marketers, and computer scientists from the many microcomputer and semiconductor manufacturers around his restaurant.

"Am l late?" puffed a small pudgy man with a chalk-stained suit and a small bow tie.

"Oh, hi. Late for what?"

”Late for lunch. l wanted to get here in time for the lunch crowd from the microcomputer companies."

"No, you're just in time. They should be arriving any minute. Are you meeting someone?"

"No, I wanted to try to strike up some friendships with some of the engineers. My name is Anton Slivovitz. I'm an inventor, and I wanted to get some support for my new invention."

"Well, l can introduce you to some of our regulars. What kinds of inventions do you invent?"

"Mainly high-tech stuff. l invented a saser."

"A saser? Is that like a laser?"

"Well, kind of. It's sound amplification by stimulated emission of radiation. It produces coherent sound waves of one frequency. I thought possibly I could develop it into a death ray of some type, but when I tried it out in a crowded shopping center, not a soul was affected. I also invented a binacus."

"Oh, oh…uh ..., what's your latest invention?"

"The one I'm going to try to push today is a uniform code for microcomputer peripheral devices. You see, if all the manufacturers of video displays terminals, keyboards, printers, plotters, and other character-oriented devices used the same codes, then you could easily use *any* peripheral device with any other one, or with any microcomputer system. I’ve developed what I call my Anton Slivovitz Code for Internal Information, or ASCII, for short. It

represents all the alphabetic characters (upper and lower case, of course), numeric digits, and special characters, such as the pound sign, the dollar sign, and the percent sign. It even has provision for special codes."

As Anton was describing his code, Bob Borrow, an engineer from In log, walked in. "Did you say you developed the ASCII code? May I shake your hand, Mr.... er .... "

"Slivovitz."

Slivovitz. And could you autograph this pocket reference card for me?”

"Certainly, I ..., but ..., I don't understand. This card says 'ASCII Codes and they're the same as mine!"

“Mr. Slivovitz, these are the ASCII codes!"

"You mean that someone else already uses these?"

"Just about everybody but IBM. And you know that old joke about where a big gorilla sleeps. Hee, hee."

"Oh, my goodness. Well, back to the saser .... "

So saying, the dejected inventor made a hurried retreat out the door.

ASCII CODES

As you might have surmised from the above, ASCII codes are as standard set of 7-bit codes for character data that are used with peripheral devices for computers, including microcomputers. The ASCII codes are shown in Fig. 9-1. The most significant bit for all codes is not used, and is usually set to 0. The ones that we're primarily concerned with here are the codes that represent the numerals 0 through 9, the code for a plus sign, the code for a minus sign, and the code for a period. The codes for 0 through 9 are 3OH through 39H,respectively while the code for a plus sign is 2BH, the code for a minus sign is 2DH, and the code for a period is 2EH.

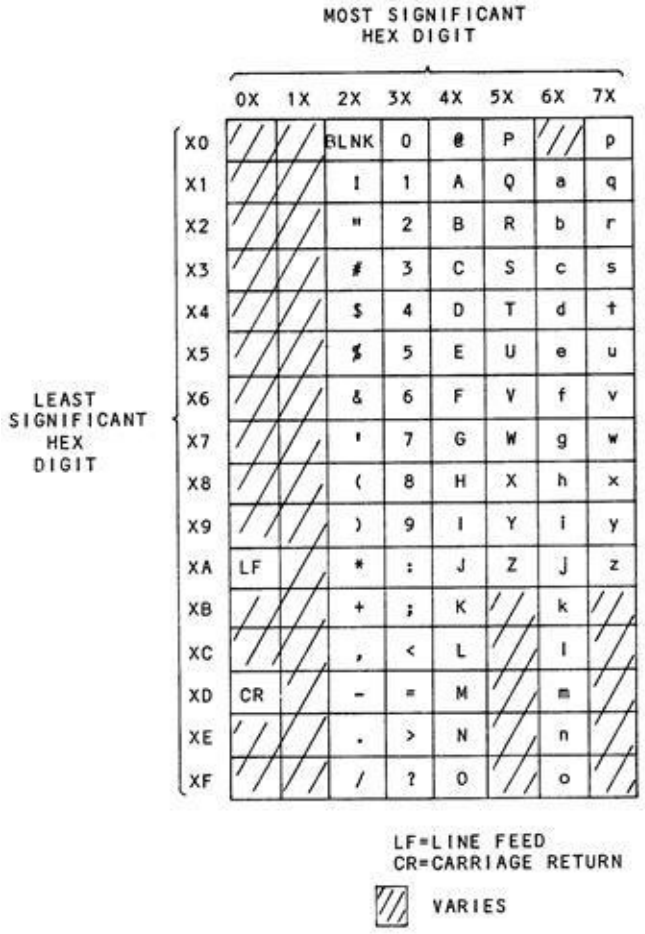


Fig 9-1. ASCII Codes

As data is input or output from the microcomputer, a **character string** of these codes is handled. For example, inputting a string of data from a keyboard might result in the codes shown in Fig. 9-2 being stored in a memory **buffer** that

is dedicated to the keyboard input. Outputting data to a line printer might take place from a line of character codes in a print buffer, as shown in the same figure. The program has to convert the input character codes to binary before processing can occur. After processing is done, the binary results are converted back to character codes for display or printing.

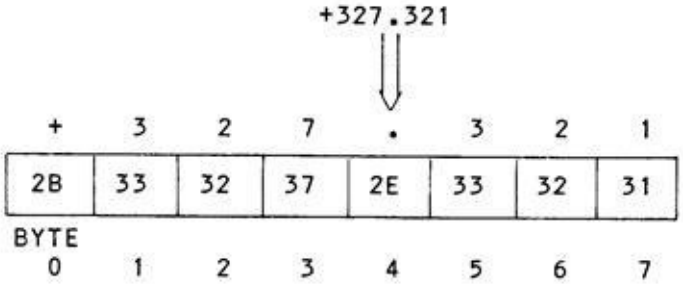


Fig 9-2. Storage of ASCII input in Memory Buffer

CONVERSION FROM ASCII TO BINARY INTEGERS

Let's first consider a conversion from ASCII coding to an integer binary value. Suppose that we have typed a value of five decimal digits from the keyboard. The keyboard driver program has put those characters in a buffer, somewhere in memory.

Before we do the conversion from ASCII characters (representing the digits O through 9) into a binary value, we have to define some conditions about the input value. First, we have to define the size of the data that we'll be working with. If we are to be working with 16-bit binary integers, for example, then we'll have to restrict the input value from- 32,768 to + 32,767. Anything larger would be an invalid input. We'll also have to restrict the input to integer values with no fractions. Finally, we'll have to accept only the characters 0 through 9, and a prefix of "+" or "-". The conversion would go something like this:

1. Clear a partial product.

2. Multiply the partial product by ten. On the first pass, this would produce zero.

3. Starting from the most significant ASCII character, fetch it and subtract 30H from it.

4. Add the result to the partial product.

5. If this was not the last ASCII character, go to step 2.6. If there was a prefix of the partial product now holds the16-bit binary value. If there was a prefix of " –" negate the 16-bit partial product to get the negative value.

Of course, there's a lot left unsaid about setting up pointers to the input buffer, getting the character, checking it for a valid value between 30H and 39H, bypassing any sign prefix, and so forth, but this is the general conversion algorithm. A sample of this conversion is shown in Fig. 9-3.

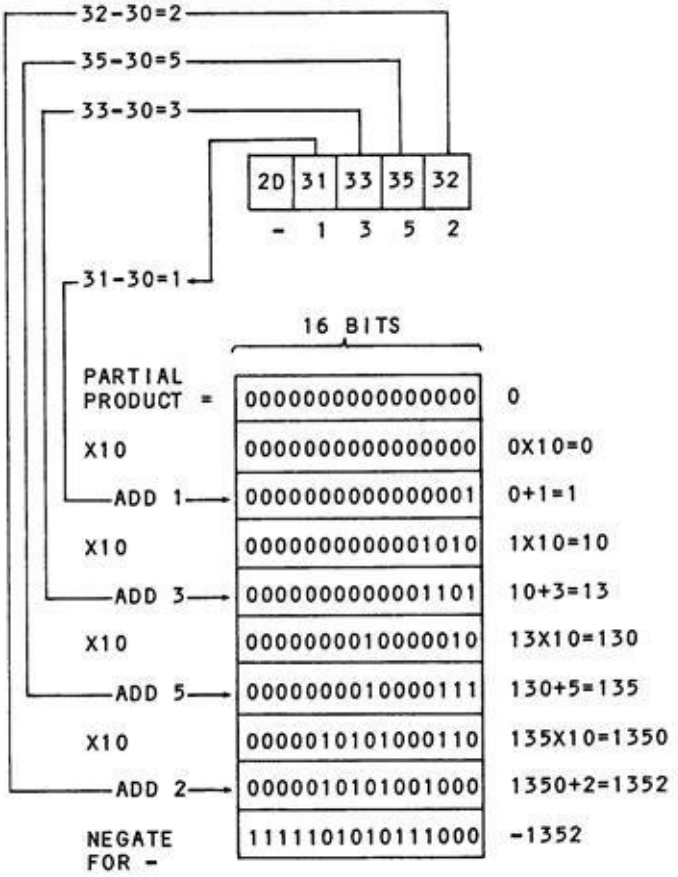


Fig 9-3. ASCII to Binary Integer Conversion

This conversion scheme can be used for any integer input value, although larger values will pose some problems in the holding of the entire partial product in the registers at one time.

CONVERSION FROM ASCII TO BINARY FRACTIONS

When the input string represents a mixed number with an integer, decimal point, and a fraction, the conversion problem is more difficult. Let's take the example shown in Fig. 9-4. The integer portion from the first character to the decimal point is converted as in the preceding scheme, and the result is saved.

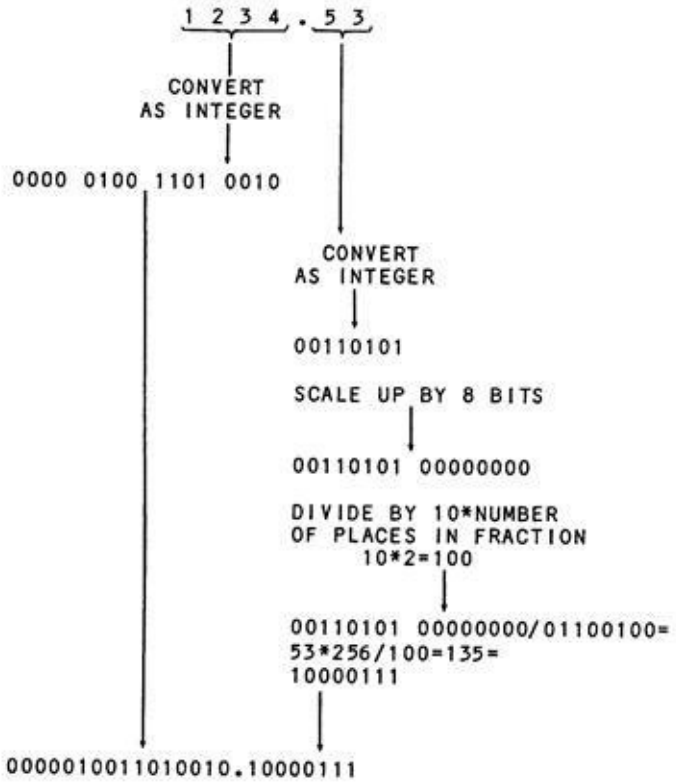


Fig 9-4. ASCII to Binary Fraction Conversion

The fraction portion is now converted as an integer value (from the first character after the decimal point to the last character). Now, determine how many bits the fraction is to occupy. Scale up by that amount. For example, if the fraction is to occupy eight bits, multiply the result of the conversion by 256; this can be done by simply shifting, or better yet, by adding a byte of zeroes to the result.

Now take the scaled result and perform a division of ten times the number of decimal places in the fraction. If, as in this case, there are four places, divide the scaled-up fraction by 10000. The quotient of the division is the binary fraction to be used, and the binary point is aligned after the original conversion

value as shown in Fig. 9-4. If the input value is a negative value, negate both the integer portion and the fraction.

This scheme can be used to convert a mixed number either to the integer/fraction format or to the "implied binary point" format as described in the previous chapter.

CONVERSION FROM BINARY INTEGERS TO ASCII

After processing has been done in the program, the result must be converted to an ASCII string of decimal digits, including a possible sign and decimal point.

Let's first consider conversion of an integer value of eight, sixteen, or some other number of bits. The general approach is to use the divide by ten and save remainders algorithm to get a string of values. Each value can be from zero to nine. As the *last* value represents the first character to be printed, the entire conversion must be done before the translation to ASCII. A sample conversion is shown in Fig. 9-5.

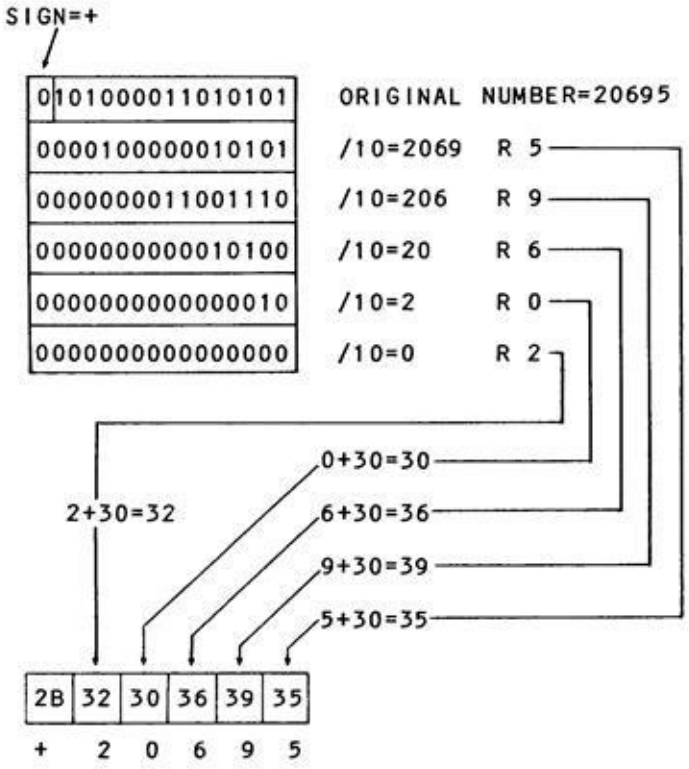


Fig 9-5. Binary to ASCII Conversion

The integer value to be converted is in a I6-bit register in the micro-processor. A test is first made of the sign. If the sign is negative, a negate is done to take the absolute value. The sign of the result is saved.

Next, a successive division by ten is done, with the remainders going to a print buffer in reverse order. When the quotient is zero, the division is completed

and all of the remainders are in the buffer.

Now, each remainder is changed to ASCII by adding 30H to it. The result is an ASCII digit from 0 to 9. After the last digit has been converted, a minus or plus sign is stored at the beginning of the print buffer, depending upon the initial value. The string of ASCII characters can be printed by calling a software print driver.

CONVERSION FROM BINARY FRACTIONS TO ASCII

The number to be output is first tested for sign. If the sign is negative, a flag is set, and the number is *negated* to its absolute value for conversion. The number is then separated into an integer and fractional part. The number may already be divided in this fashion if a separate integer and fraction are maintained. If the number has been scaled and has an implied fixed binary point, shifting will separate the fraction and integer portion into two component parts.

The integer portion may now be converted by the divide by ten, save remainders scheme as shown in Fig. 9-6. The remainders are put in reverse order into a print buffer. The ASCII code for a decimal point is now stored in the print buffer after the last remainder. Now convert each remainder to ASCII by adding 30H.

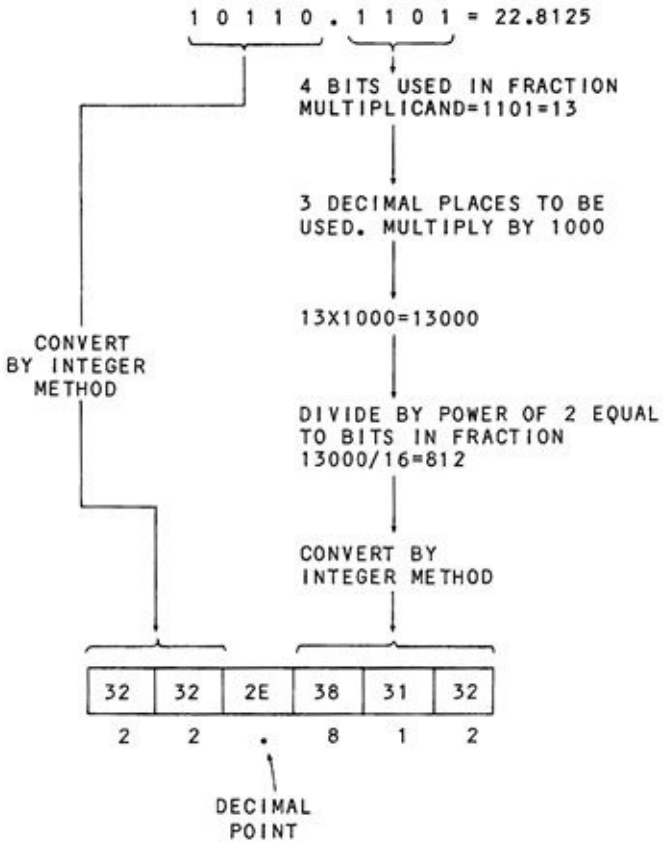


Fig 9-6. Binary to ASCII Fraction Conversion

Finally, the fractional part can be converted. The number of bits in the fraction is found. This is really the number of bits that was maintained while

processing of the binary mixed number was taking place. Align these bits in a multiplicand. Now determine the number of decimal places to be output. For each decimal place to be output, multiply by a power of ten. In this case, three decimal places are to be used, so the fractional portion is multiplied by 1000.

Next, divide the result of the multiplication by the power of two equal to the number of bits used in the fraction. In this case, four bits are used, so sixteen would be divided into the result of the multiplication. This division may be done by shifting. Discard the bits shifted out. The resulting number can now be converted to ASCII by the method used for integers that was described above - divide by ten and save remainders in reverse order and, then, add 30H to convert to ASCII. Add a negative sign in ASCII at the beginning of the print buffer and use a print driver to display the final result.

If you are wondering why floating point is used, the preceding conversion algorithms might shed some light upon why a generic way to handle a wide range of numbers in standard format might be required. Floating-point operations are described in the next chapter. They are much less tedious than a program that performs computations for a variety of scaled numbers like the preceding programs! The following exercises will help in your appreciation of the next chapter.

EXERCISES

1. A buffer holds the ASCII values shown below. What decimal numbers are represented?

2B, 30, 31, 32, 33, 39, 31
2D, 30, 39, 38
2D, 30, 31, 2E, 35, 32
2B, 2E, 37, 36, 38

2. Convert the following numbers into ASCII representation (with sign):

+958.2, - 1011.59

CHAPTER 10

Floating-Point Numbers

Floating-point numbers in computers express numbers in a form of scientific notation, where each number is made up of mantissa and a power of two. In this chapter, we'll see how floating-point operations work in general.

… AND 3000 COMBINATION PLATES FOR THE MOTHER SHIP . . .

It was early evening, and Big Ed was just getting ready to close down the restaurant. Half a dozen computer engineers were whisked out by Ed as he locked the door to the building and escorted them out into the clear California night.

"Well, g'night boys, see .... What the heck is that!"

All heads turned in the direction Ed was pointing. A large object with pulsating orange lights was hanging almost directly over Big Ed's. A faint humming sound could be heard.

"It's a UFO! I've finally seen one!" said one of the engineers, excitedly.

The crowd's eyes bulged as the object hung there.

"Don't just stand there! Let's try to photograph it, or communicate with it, or *something*!"

"l don't know - remember what happened in *Independence Day*!"

"I've got a flashlight in the car. Hang on, I'll get it," said one of the more adventurous engineers. He ran to his car and came back with a long powerful flashlight.

"You'd better hope they don't think it's some kind of laser, Frank, "warned one.

"Listen, we've got to try this. I've been following Neil Degrasse Tyson. I know what to do. I'll send it a number and see if it responds!"

He pointed the flashlight directly at the object and flicked the button on and off. "Nine flashes - let's see if it comes back with the same number!"

To everyone's excitement, a bright, directional light beam shot back a long flash, a short flash, a short flash, a long flash, and stopped.

"That's not nine!" said one of the engineers. "Yes it is--it's nine in binary. They're testing us to see if we've progressed beyond punched-card equipment! Let's try something else. Let's try a Fibonacci series ....

"He sent a "1, 1, 2, 3, 5, 8, 13, 21, 34" and then waited. Almost

immediately, a "long, long, short, long, long, long" came back. “That’s 110111, for the next term of 55 in the series!" shouted the engineer with the flashlight.

"How about pi," suggested one of the group.

"Yeah, let's try that. Three flashes, one flash, four flashes, one flash… , there, that's the first four digits…”

All watched as the object responded with a series of flashes. “Take these down, Roy," said the engineer with the flashlight. The UFO sent a burst of flashes, and then the sequence stopped. “Boy this doesn't make any sense at all!" said the engineer who was recording the response. What he had recorded is shown in Fig. 10-1.

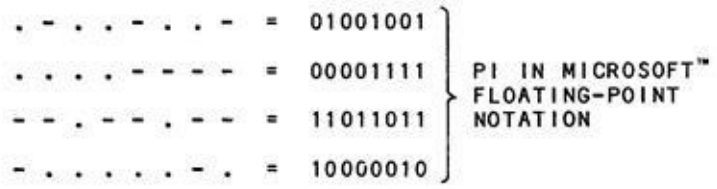


Fig 10-1. The UFO’s Response

"Wait! I’ve got it! This is pi in floating-point notation!"

The engineers grouped around the message excitedly. In the course of the next ten minutes, several other series were sent and received.

"What’s it sending now?"

"I don't know, it's different from the rest. Hey, wait! This is ASCII. It reads--- uh ..., 'CAN YOU READ THIS?' Signal back 'YES'."

The engineer with the flashlight answered with a "YES" in ASCII.

"Now what's it sending?" asked one of the engineers, as the other converted the incoming message to text."

“It looks like a take-out order," cried Ed, glancing over the shoulder of the engineer who was decoding the text. " ‘FORTY REUBEN SANDWICHES, EIGHTY ORDERS FRIES, FORTY COKES.' No wonder they stopped here. Well, a customer is a customer .... "

Ed turned back to the restaurant, and started unlocking the door ....

SCIENTIFIC NOTATION VS. FLOATING POINT

Floating point is really just a binary version of scientific notation. Scientific notation can express very large and very small numbers easily, in a *normalized* format that makes computations simpler. ln scientific notation, a number is represented by a mixed number and a power of ten. The number is adjusted so that it has a one-digit integer portion and a fraction.

Take the example of the number of square inches in a square mile. Admittedly, this isn't something that you would normally be too concerned with, unless you're subdividing tracts for ant colonies, but it shows how easy scientific notation is to work with. There are 12 inches in a foot. Within one square foot, therefore, there are 12 x 12 or 144 square inches. Converting to scientific notation goes like this: 144 x 10 ^0 = 144, as any number to the 0 power is one. Normalizing the mixed number portion, we move the decimal point two digits over, so that it is between the 1 and 4. For every shift to the left we add one to the exponent, or power of ten, so we have:

144= 144 x 10^O = 14.4 x10 ^ 1=1.44 x 10^2

The last figure is the normalized, or standard, form for scientific notation. Now, we want to find the number of square feet in a square mile. We know there are 5280 feet per mile, so there must be 5280 x 5280 square feet in a mile. Let's convert 5280 to scientific notation before we proceed.

5280 = 5280 x 10^0 = 528.0 x 10^1 = 528.0 x 10^1 =
52.80 x 10^2 = 5.280 x 10^3

To find the final answer, the number of square inches in a square mile, we can say:

number of sq. inches/sq. mile =

5.280 x 10^3 x 5.280 x 10^3 x 1.44 x 10^2

Now for the profound part. The powers of identical bases are added for multiplication of numbers, and subtracted for division. Since all the numbers use a power of 10, we can simply add the 3 from 10 T 3, the3 from 10^3, and the 2 from 10 ^2. We now have

number of sq. inches/sq. mile =
5.280 x 5.280 x 1.44 x 10^8

which turns out to be 40.14 x 10^8. To express this as a number without an exponent, shift the decimal point to the right, and subtract one from the exponent as you do so. Add zeroes if necessary.

number of sq. inches/sq. mile =

40.14 x 10^8 = 401.4 x 10^7 =
4014 x 10^6 = 40140 x10^5 =
401400 x 10^4 = 4014000 x 10^3 =
40140000 x 10^2 = 401400000 x 10^1=
4014000000 x 10^0 = 4,014,000,000

Exponents may also be used in scientific notation to express very small numbers. Let's calculate how many angels can dance on the head of a pin. We'll use the-standard angelic dimensions of one angel per 1/11,000 square inch, or 0.0000909. The size of an ordinary common pinhead (according to the American Association of Common Pin Manufacturers) is 0.000965 square inch. Expressing both numbers in scientific notation, we have:

area for one angel = .0000909 =
.0000909 x 10^0 = 0.000909 x 10^-1 =
0.00909 x 10^-2 = 0.0909 x 101-3 =
0.909 x 10^-4 = 9.09 x 10^-5

and

area for head of pin = .000965 =
.000965 x 10^0 = 0.00965 x 10^ - 1 =
0.0965 x 10^-2 = 0.965 x 101-3 =
9.65 x 10^ -4

In the above conversions, we subtracted one from the exponent every time we shifted the decimal point to the right. The negative power of 10 lets us represent reciprocal powers of ten – 1/10, 1/100, 1/1000, and so forth.

Now to find the number of angels doing the microprocessor waltz on the

head of a pin, we divide the size of the pin by the size of an angel:

number of angels on the head =
(9.65 x 10^-4)/(9.09 x 10^-5)

Here we apply the rule that if the bases are identical, then we can subtract the exponents for the division:

number of angels on the head =
9.65/9.09 X 10^(-4 + 5) =
9.65/9.09 x10^ l= 1.06 x10^1=
10.6 x10^0 = 10.6 angels

Using standard scientific notation has made the calculations much easier, as all of the values have been in standard base-10 format, and we can add exponents when multiplying and subtract them when dividing.

It turns out that adding and subtracting two numbers in scientific notation is more complicated in some respects than multiplying them. Consider the two numbers 3/32 and 6/60. In scientific notation, these are

3/32 = 0.09375 = 0.09375 x 10^0 =
.9375 x 10^-1 = 9.375 x 10^-2

and

6/60 = 0.1 = 0.1 x 10^0 = 1.0 x 10^-1

Now in order to add or subtract two numbers in scientific notation, their exponents must be the same. One or the other of the numbers, therefore, must be adjusted to make both exponents equal. We can do this either by moving the decimal point of 9.375 x 10 ^ - 2 to the left and adding one to make .9375 x 10^ - 1, or by moving the decimal point of 1.0 x 10^ -1 to the right and subtracting one to make 10.0 X 10^ -2. Once the exponents are equal, we can add or subtract.

3/32 + 6/60 = 9.375 x 10^-2
<u>+ 10.0 X 10^-2</u>
19.375 x 10^-2=
1.9375 x 10^-1=

$$0.19375 \times 10^0 = 0.19375$$

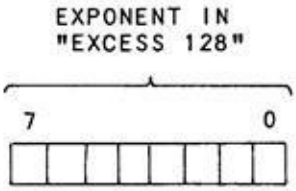
Right about now, you're probably asking yourself, "Why didn’t he just add the messy things in 'regular' notation?" With many computations, it makes more sense to convert to scientific notation; it's easier to keep track of the decimal point, for one thing.

USING POWERS OF TWO IN PLACE OF POWERS OF TEN

It turns out that the rules for adding, subtracting, multiplying, and dividing numbers in the same base work for any base. We could have just as easily used base-8, base-11, or (he said, triumphantly) base-2!

Let's define a standard base-2 format for numbers. This will be the same format that is used in many microcomputers. To begin with weave to choose a range of numbers. If we use base-2 format (some larger machines use base 16), we will then need a place to keep an exponent. We can allocate some convenient number of bits for the exponent. Since everything in microcomputers seems to be built around bytes, let's use one byte for the exponent. This will give us eight bits, allowing us a range from 2 ^ 0 to 2 ^ 8- 1, or from 0 to 2 ^255.This would allow us to represent numbers that would be equal to about 5.79 x 10 ^ 76.

One moment, though. We need negative powers of two, also, as we need to represent small values. We'll make the eight bits of an exponent an **excess 128** code. This means that we'll add 128 to the exponent value to obtain the number stored in the exponent byte, and subtract it from any result to get the true exponent. The excess 128code is done to simplify handling the floating-point number as a single entity. Fig. 10-2 shows the exponent format and some sample values. We now have a range of exponents from 2^-128 to 2^ +127 (3.4 x 10^-38 to 1.7 x 10^38).



VALUE	AFTER ADJUSTMENT BY -128 SUBTRACTION	POWER OF 2
11111111	01111111	+127
11010000	01010000	+80
10000101	00000101	+5
10000000	00000000	0
01111111	11111111	-1
01111110	11111110	-2
00001111	10001111	-112
00000000	10000000	-128

Fig 10-2. Exponent Format

Now for the **mantissa**, the number that is multiplied by the power of two. Rather than defining a range, the mantissa defines a **precision**. We know that two bytes, or sixteen bits, gives us values from 0 to 65,535 and about 4 1/2 decimal digits. This doesn't seem quite large enough for many problems. To keep things in multiples of bytes, we'll have to go to three bytes, or 24 bits. That will give us from 0 to 16,777,215, or about 7 decimal digits of precision, which is probably a good compromise between storage requirements and precision.

What we now have as an approximate format is shown in Fig.10-3-three bytes of mantissa, and one byte for the exponent. What can we say about **normalization**? The guiding rule here is that we would like to retain as much precision as possible. We do that by keeping as many significant bits as possible. Since we have a limited number of bits to hold the mantissa, we must get rid of all extraneous bits and just keep the significant ones. We do that by normalizing the mantissa so that the first "1" bit is immediately to the right of an assumed binary point just in front of the first mantissa bit, as shown in Fig. 10-4. That way, we are assured of maximum precision as there are no extraneous zeroes in

front of the first 1 bit and no truncated portion of the mantissa at the end. The first bit in the mantissa, therefore, is always a 1.

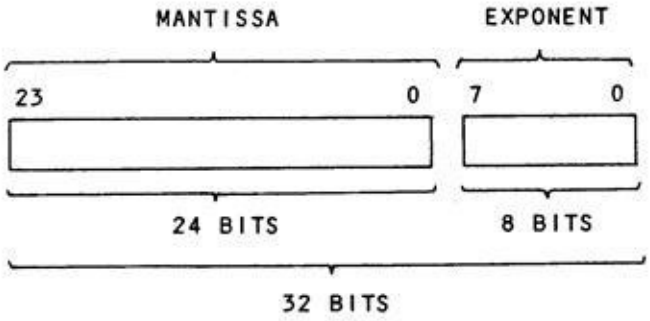


Fig 10-3. Floating-Point Format, First Try

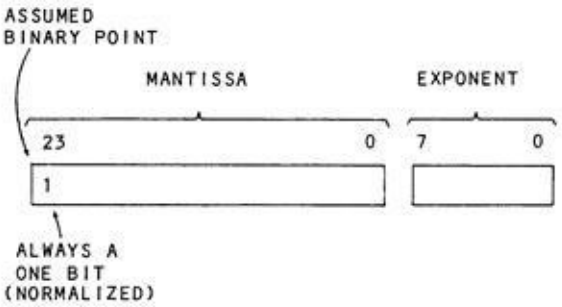


Fig 10-4. Floating-Point Format, Second Try

Let’s see. We now have one byte for the exponent and a three-byte normalized mantissa. What have we forgotten? It appears that we can express positive numbers, but no signed numbers. The sign bit has disappeared with normalization. We definitely need a sign bit, unless we choose to limit our BASIC users to positive numbers only …

One solution goes like this: As every floating-point number is going to be normalized anyway why not let the first mantissa bit represent the sign bit? The remainder of the number will be held as a two’s complement number. We’ll just throw away the first 1 bit as we know what it is anyway. The final format for floating-point calculation now has the form shown in Fig. 10-5.

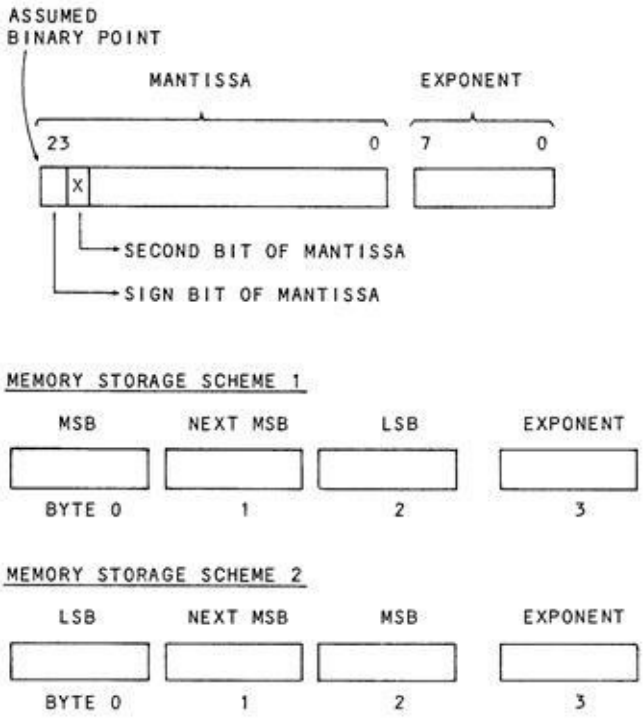
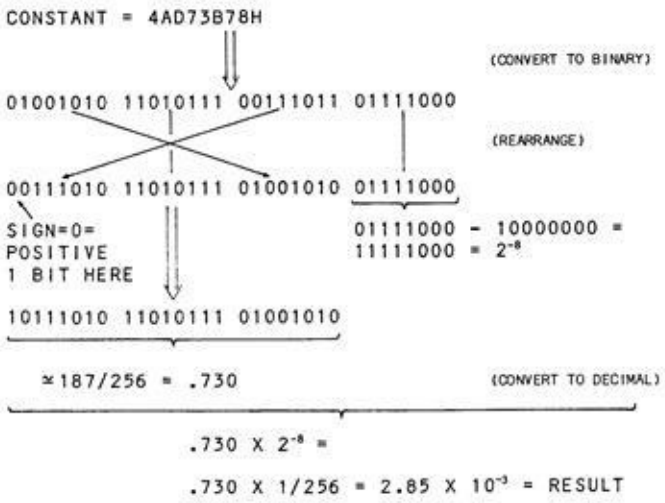


Fig 10-5. Floating-Point Format, Second Version

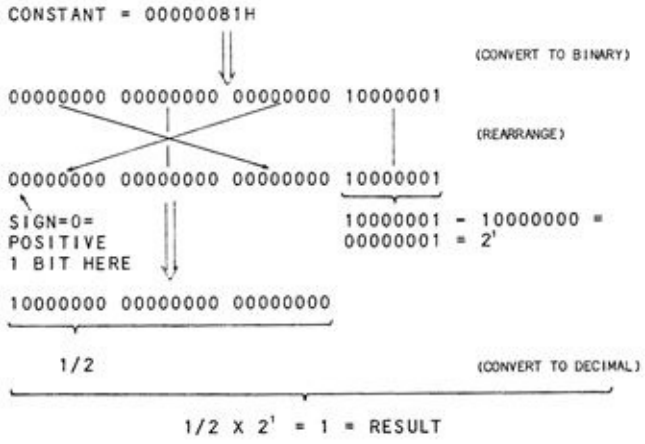
You'll note in Fig. 10-5 that there are two schemes for storing floating-point numbers. This is due to the way different microprocessors store 16-bit values. If data is stored in memory by two 16-bit stores (Z-80 microprocessor and others), then the format has the least significant byte first, followed by the most significant byte. This means that the floating-point number is stored as follows: least significant byte of the mantissa, next most significant byte of the mantissa, most significant byte of the mantissa, and exponent byte.

Fig. 10-6 and Fig 10-7 show three examples of constants stored in memory. Study these to better learn how to decode floating-point formats.

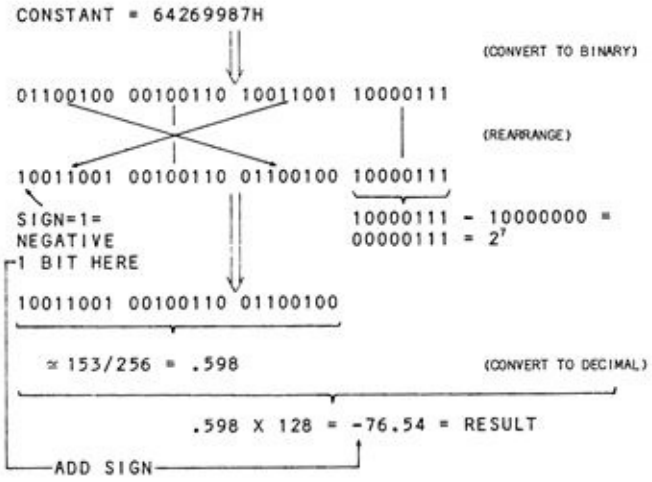


A. First example.

Fig 10-6. Floating Point Examples 1



B. Second example.



C. Third example.

Fig 10-7. Floating Point Examples 2

DOUBLE-PRECISION FLOATING-POINT NUMBERS

The range of exponents is probably more than adequate for most processing. Few quantities are greater than 1 to the 39th power or smaller than one part in 1 to the 39th power. The number of digits of precision, however, might be increased if the cost in storage is not too high. Another decimal digit is added for approximately every 31/2 bits(3 bits scaled up by 8 and 4 bits scaled up by 16). Therefore, for each two bytes added to the mantissa, about 5 decimal digits are added.

A double-precision floating-point format found in some higher-level languages adds four more bytes to the mantissa, to make the total precision about 17 decimal digits while retaining the same range of numbers. This scheme is shown in Fig. 10-8.

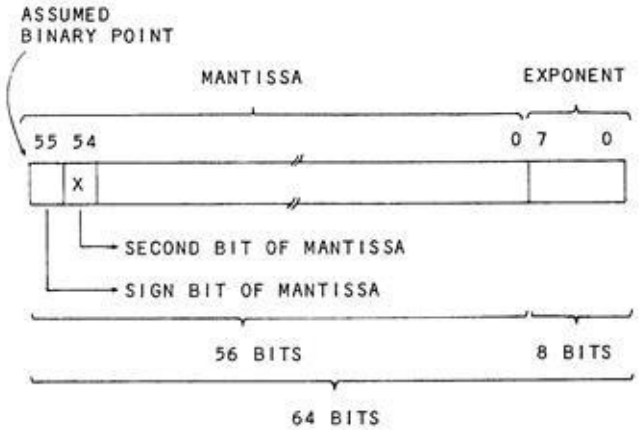


Fig 10-8. Double-Precision Floating-Point Format

CALCULATIONS USING BINARY FLOATING-POINT NUMBERS

Operations using binary floating-point numbers emulate the scientific notation operations. Two normalized floating-point numbers may be multiplied or divided by performing an integer multiplication on the mantissa, and adding or subtracting the exponents. Additions or subtractions of two floating-point numbers have to be performed with their exponents adjusted to equal values. As the format of the mantissa allows only fractions less than one, the number with the smaller exponent is adjusted by shifting the mantissa right and adding one to the exponent value for each shift until the exponents are equal.

The algorithms for floating-point operations are rather complicated, and the actual code for floating-point operations is a sizable chunk in higher-level language compiler. Although we can't go into detail here, we hope that the material in the past two chapters has provided some insight into the general approach to handling fractions, mixed numbers, and floating-point operations.

EXERCISES

1. Convert the following numbers to scientific notation:

3.141600, 93,000,000, -186,000, 00000135

2. Perform the following operations using scientific notation:

3.14 X .000152 X 200,000 = ?

3,100,000/.000015 = ?

3. Express these powers of two as 8-bit exponent values with excess 128 coding:

2^0, 2^1, 2^-5, 2^35, 2^-40, 2^128, 2^-129

4. Convert these floating-point numbers to decimal values. The mantissa is the most significant byte to the least significant byte, left to right. The exponent is the byte on the extreme right.

00010000 00000000 00000000 10000111=?
10010011 10000000 00000000 10000111=?

APPENDIX A

Answers to Exercises

CHAPTER 1

1. 10100, 10101, 10110, 10111, 11000, 11001, 11010, 11011, 11100, 11101, 11110, 11111, 100000.

2. 53, 16, 85, 240, 14185.

3. 1111, 11010, 110100, 01101001, 11111111, 1110101001100000.

4. 00000101, 00110101, 00010101.

5. 15, 63, 255, 65535 .(2^n)-1.

CHAPTER 2

1. 9 x 16^2 + 14 x 16^1 + 2 x 16^0..

2. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10, 11, 12, 13, 14.

3. 5, AH, AAH, 4FH, B63AH.

4. 101011100011,100110011001,1111001000110010.

5. 227, 82, 43690.

6. DH, FH, 1CH, 3E8H.

7. FFFFH.

8. 73, 219.

9. 7, 161, 310.

10. It is impossible.

11. 321.

CHAPTER 3

1. 1+3=4 (100), 7+15=22 (10110), 21+ 42=63 (111111).

2. 2-1=1 (1), 7-5=2 (10), 12-1=11 (1011).

3. Not necessary 111, necessary -86, necessary - 128.

4. 11111111, 11111110, 11111101, 11100010, 00000101, 01111111.

5. 00000000011111111 (+127, +127), 1111111110000000 (-128, -128), 1111111110101010 (-85, -85).

6. 1111111011010100 (-300 + 1111111111111011 (-5) =

1111111011001111 (-305).

7. 1111111011010100 (-300) – 1111111111111011 (-5) =

1111111011011001 (-295).

CHAPTER 4

1. +127+1= +128 (10000000, overflow), + 127-1 = +126 (01111110, no overflow), -81+(-1)= -82 (10101110, no overflow).

2. Result = 10000000 (overflow, but no carry), result = 00000000 (no overflow, but carry).

3. Result = 00000000(Z=1, S=0), result = 00110101(Z=0, S=0).

CHAPTER 5

1. 10101111, 11110111.

2. 10100101, 11010111.

3. XXXYYXXX AND 00011000 = 000YY000.

4. 11100001(-31), 1011(-5), 01010110(+86).

5. 01011110, 00000001.

6. 10010111, 01000000

7. C=0 01011111, C=1 00000000.

8. 00010111 C=1, 11000000 C=0.

9. 00111111 C=1 (before = +127 after = +63), 00101101 C=O (before= +90 after = +45), 01000010 C=1 (before = -123 after = +66), 01000000 C=0 (before = -128 after = +64).

10. 11111110 C=0 (before = +127 after = -2), 10110100 C=0 (before =+90 after = -76), 00001010 C=1 (before = -123 after = +10), 00000000 C=1 (before = -128 after = 0).

11. 00111111 (before = +127 after = +63), 11000010 (before = -123 after= -62), 11000000 (before = -128 after = -64).

CHAPTER 6

1. 1111111000000001 (255 x 255 = 65025).

2. 1111111111111111, 1111111111111111, division by zero .

3. 1 = negative, 0 = positive by Exclusive OR of signs of operands.

CHAPTER 7

1. 11111111 11111111 11111111 or 16,777,215.

2. 00100010 00000000 01011010 01000000.

3. 00000000 11111110 11011001 11111111.

4. 11000000 01010111 01000100 10000001.

5. 00011111 11010101 11011101 10111111 C=1.

CHAPTER 8

1. .71875, .5, .9375, .00012207.

2. .01011101 ..., .0101, .1100011.

3. 46.6875, -73.0625.

4. 0110010000000000, 1001110100000000.

5. 11.5, 2642.25.

CHAPTER 9

1. +012391, -098, -01.52, +.768.

2. 2B 39 35 38 2E 32, 2D 31 30 31 31 2E 35 39.

CHAPTER 10

1. 3.1416 x 10^0, 9.3 x 10^7, -1.86 x 10^5, 1.35 x 10^-5.

2. 3.14 x 1.52 x 10^-4 x 2.0 x 10^5 = 9.5 x 10^1

3.1 x 10^6 / 1.5 x 10^-5 = 2.0 x 10^11.

4. 72.0, -73.75.

APPENDIX B

Glossary of Terms

accumulator - The main register(s) in a microprocessor used for arithmetic, shifting, logical, and other operations. The Z - 80 microprocessor has one (A register), while the 6809E microprocessor has two (A and B registers).

add with carry - A machine - language instruction in which one operand is added to another, along with a possible carry from the previous (lower - order) add.

algorithm - A step - by - step process for performing a task.

AND - A bit - by - bit logical operation which produces a 1 in the result bit only if both operand bits are ones..

arithmetic shift - A type of shift in which an operand is shifted right or left with the sign bit being extended (right shift) or maintained (left shift).

ASCII - A standard code for representation of character data in computers and computer peripheral equipment.

assembly language - A symbolic computer language that is translated by an ”assembler program" into machine language, the numeric codes that are equivalent to microprocessor instructions.

base - The "starting point" for representation of a number in written form, where numbers are expressed as multiples of powers of the base value.

binary - Representation of numbers in "base two," where all numbers are expressed by combinations of the binary digits 0 and 1.

binary digit - The two digits (0 and 1) used in binary notation. Often shortened to “bit."

binary point - The point, analogous to a decimal point, that separates the integer and fractional portions of a binary mixed number.

bit - Contraction of "binary digit."

bit position - The position of a binary digit within a byte or larger group of binary digits. Bit positions in most microcomputers are numbered from right to left, zero through N. This number corresponds to the power of two represented.

borrow - A 1 bit that is subtracted from the next higher bit position.

buffer - A portion of memory dedicated to hold characters (or other data) as they are read in, or used to store characters (or other data) for output.

byte - A collection of eight binary digits. Each memory location in most microcomputers is one byte "wide."

carry - A one bit added to the next higher bit position or to the ”carry flag."

carry flag - A bit in the microprocessor used to record the carry ”off the end" as result of a machine - language instruction.

clobber - (computerese) To destroy the contents of memory or a register.

Colossus- - A British computer used to crack German Enigma codes during World War ll.

conditional jump - A machine - language instruction that causes a jump to another instruction if a specified flag (or flags) is set or reset.

data - A generic term for numbers, operands, program instructions, flags, or any representation of information using binary ones and zeroes.

displacement - A signed value in machine language used in defining a memory address.

dividend -The number that is divided by the divisor. ln A/B, A is the dividend.

divisor - The number that ”goes into” the dividend in a divide operation. In A/B, B is the divisor.

double - dabble - A method of converting from binary to decimal representation by doubling a leftmost bit, adding the next bit, and continuing until the rightmost bit has been processed.

eight - by - eight multiply - The multiplication of eight bits by eight bits to produce a 16»bit product.

Enigma - A German cipher machine (World War II).

excess 128 code - A standard way to bias the exponent in Microsoft" BASIC. The value 128 is added to the actual power of two and then stored as an exponent value in floating - point representation.

exclusive OR - A bit - by - bit logical operation that produces a 1 bit in the result only if one or the other (but not both) operand bits is a 1.

exponent - In this book, usually the power of two of a binary floating - point number.

Fibonacci series - The sequence of numbers 1, 1, 2, 3, 5, 8, 13, 21, 34 …where each term is computed by the addition of the two previous terms.

floating - point number - A standard way of representing a number of any size in computers. Floating - point numbers contain a fractional portion (mantissa) and power of two (exponent) in a form similar to scientific notation.

H - A suffix for hexadecimal numbers.

hex - Abbreviation of hexadecimal.

hexa - dabble - The conversion from hexadecimal to decimal by multiplying each hex digit by sixteen and adding the next hex digit until the last (rightmost) hex digit has been reached.

hexadecimal - The representation of numbers in a "base - sixteen" notation by use of the hexadecimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

inclusive OR - A bit - by - bit logical operation which produces a 1 - bit result if one or the other operand bits, or both, is a 1.

integer variable - A variable type. Can hold values of - 32,768 through + 32,767 in a two - byte "two's complement" notation.

iteration - One pass through a given set of instructions.

least significant bit - The rightmost bit in a binary value, representing 2 ^0.

logical shift - A type of shift in which an operand is shifted right or left, with a zero filling the vacated bit position.

machine language – One or more bytes that make up microprocessor native

instructions. These values are produced by an "assembler program" from assembly language code.

mantissa - The fractional portion of a floating - point number.

minuend - The number from which the subtrahend is subtracted. In 5 - 3 = 2, 5 is the minuend.

mixed number - A number that consists of an integer and a fraction as, for example, 4.35 or (binary) 1010.101 1.

most significant bit - The leftmost bit in a binary value, representing the highest - order power of two. In two's complement notation, this bit is the sign bit.

most significant byte - The highest - order byte. In the multiple - precision numberA13EF122H, the two hex digits A and 1 make up the most significant byte.

multiple - precision numbers - Multiple - byte numbers that allow extended precision.

multiplicand - The number to be multiplied by the multiplier. The number "on the top."

multiplicand register - The register used to hold the multiplicand during a machine - language multiplication.

multiplier - The number that is multiplied against the multiplicand. The number “on the bottom.”

negation - Changing a negative value to a positive value, or vice versa. Taking the two’s complement by changing all the ones to zeroes, all zeroes to ones, and, then, adding a one.

normalization - Converting data to a standard format for processing. In floating - point format, converting a number so that a "significant" bit (or hex digit) is in the first bit (or four bits) of the fraction.

NOT - A logical operation that takes the "inverse" or one's complement.

octal - The representation of numbers using a "base - eight” notation which uses the octal digits 0, 1, 2, 3, 4, 5, 6, and 7.

octal - dabble - The conversion of an octal number to a decimal number by multiplying by eight and adding the next octal digit; continuing until the last (rightmost)digit has been added.

operands - The numeric values used in an add, subtract, or other operation.

OR - See Inclusive - OR.

overflow - A condition that exists when the result of an addition, subtraction, or other arithmetic operation is too large to be held in the number of bits allotted.

overflow flag - A bit in the microprocessor used to record an overflow condition for machine - language operations.

padding - Filling bit positions to the left of a value with zeroes to make a total of eight or sixteen bits.

partial product - The intermediate results of a multiplication. At the end, the partial product becomes the whole product.

partial - product register - The register(s) used to hold the partial results of machine - language multiplication.

peripheral devices - A generic term for equipment attached to a computer, such as keyboards, disk drives, printers, plotters, speech synthesizers, and so forth.

permutation - Arrangements of things in a definite order. Two binary digits have four permutations - 00, 01, 10, arid 11.

positional notation - Representation of a number where each digit position rep - resents an increasingly higher power of the base.

precision - The number of significant digits that a variable or number format may contain.

print buffer - A portion of memory dedicated to holding the string of characters tu be printed.

product - The result of a multiplication.

propagation - The manner in which a carry or borrow travels to the next higher bit position.

punched - card equipment – One hundred-year old peripheral devices that

permit the punching or reading of paper punched cards used to hold character or binary data.

quotient - The result of a divide operation.

register - A fast - access memory location in the microprocessor of a microcomputer. Used for holding intermediate results and, also, used for computation in machine language.

remainder - The amount of dividend remaining after a division has been completed.

residue - The amount of dividend remaining part way through a division.

restoring division - A division in which the divisor is restored if the operation “does not go" for any iteration. A common microcomputer division technique.

rotate - A type of shift in which data is recirculated right or left back into the operand from the opposite end.

rounding - The process of truncating bits to the right of a bit position and adding a zero or a one to the next higher bit position based on the value to the right. Rounding the binary fraction 101 1.1011 to two fractional bits, for example, results in 1011.11.

scaling - Multiplying a number by a fixed amount so that a fraction can be processed as an integer value.

scaling up - Refers to a number that has been multiplied by a scale factor for processing.

scientific notation - A standard form for representing any size number by a mantissa and a power of ten.

shift and add - A method in which multiplication is achieved by the shifting and addition of the multiplicand.

sign bit - The leftmost bit (bit position 15 or 7) of a two's complement number. If a zero, the sign of the number is positive. If it is a one, the sign of the number is negative.

sign extension - Extending the sign bit of a two's complement number to the left by duplication.

sign flag - A bit in the microprocessor used to record the sign of the result of machine - language operation.

sign magnitude - A nonstandard way of representing positive and negative numbers in microcomputers.

signed numbers - Numbers that may be either positive or negative.

significant bits - The number of bits in a binary value after the leading zeroes have been removed.

subtract with carry - A machine - language instruction in which one operand is subtracted from another, along with a possible borrow from the next lower byte.

subtrahend - The number that is subtracted from the minuend. In 5 - 3 = 2, 3 is the subtrahend.

successive addition - A multiplication method in which the multiplicand is added a number of times equal to the multiplier to find the product.

truncation - The process of dropping bits to the right of a bit position. Truncating the binary mixed number 1011.1011 to a number with fraction of two bits, for example, results in 101.10.

truth table - A table defining the results for several different variables and containing all possible states of the variables.

Turing - An early (1940's) British computer scientist and mathematician.

two's complement - A standard way of representing positive and negative numbers in microcomputers.

unsigned numbers - Numbers that may be only positive. Absolute numbers.

Williams tube - An early type of computer memory based upon the storage of data on the face of a cathode - ray tube; designed by E C. Williams, Manchester University.

word - A collection of sixteen binary digits. Two bytes.

zero flag - A bit in the microprocessor used to record the zero/nonzero status of the result of a machine - language instruction.

APPENDIX C

Binary, Decimal, Octal, Hexadecimal Table

Binary	Dec	Oct	Hx
00000000	0	000	00
00000001	1	001	01
00000010	2	002	02
00000011	3	003	03
00000100	4	004	04
00000101	5	005	05
00000110	6	006	06
00000111	7	007	07
00001000	8	010	08
00001001	9	011	09
00001010	10	012	0A
00001011	11	013	0B
00001100	12	014	0C
00001101	13	015	0D
00001110	14	016	0E
00001111	15	017	0F
00010000	16	020	10
00010001	17	021	11
00010010	18	022	12
00010011	19	023	13
00010100	20	024	14
00010101	21	025	15
00010110	22	026	16
00010111	23	027	17
00011000	24	030	18
00011001	25	031	19
00011010	26	032	1A
00011011	27	033	1B
00011100	28	034	1C
00011101	29	035	1D
00011110	30	036	1E
00011111	31	037	1F
00100000	32	040	20
00100001	33	041	21
00100010	34	042	22
00100011	35	043	23
00100100	36	044	24
00100101	37	045	25
00100110	38	046	26
00100111	39	047	27

00101000 40 050 28
00101001 41 051 29
00101010 42 052 2A
00101011 43 053 2B
00101100 44 054 2C
00101101 45 055 2D
00101110 46 056 2E
00101111 47 057 2F
00110000 48 060 30
00110001 49 061 31
00110010 50 062 32
00110011 51 063 33
00110100 52 064 34
00110101 53 065 35
00110110 54 066 36
00110111 55 067 37
00111000 56 070 38
00111001 57 071 39
00111010 58 072 3A
00111011 59 073 3B
00111100 60 074 3C
00111101 61 075 3D
00111110 62 076 3E
00111111 63 077 3F
01000000 64 100 40
01000001 65 101 41
01000010 66 102 42
01000011 67 103 43
01000100 68 104 44
01000101 69 105 45
01000110 70 106 46
01000111 71 107 47
01001000 72 110 48
01001001 73 111 49
01001010 74 112 4A
01001011 75 113 4B
01001100 76 114 4C
01001101 77 115 4D
01001110 78 116 4E
01001111 79 117 4F
01010000 80 120 50
01010001 81 121 51
01010010 82 122 52
01010011 83 123 53
01010100 84 124 54
01010101 85 125 55
01010110 86 126 56
01010111 87 127 57
01011000 88 130 58
01011001 89 131 59

01011010 90 132 5A
01011011 91 133 5B
01011100 92 134 5C
01011101 93 135 5D
01011110 94 136 5E
01011111 95 137 5F
01100000 96 140 60
01100001 97 141 61
01100010 98 142 62
01100011 99 143 63
01100100 100 144 64
01100101 101 145 65
01100110 102 146 66
01100111 103 147 67
01101000 104 150 68
01101001 105 151 69
01101010 106 152 6A
01101011 107 153 6B
01101100 108 154 6C
01101101 109 155 6D
01101110 110 156 6E
01101111 111 157 6F
01110000 112 160 70
01110001 113 161 71
01110010 114 162 72
01110011 115 163 73
01110100 116 164 74
01110101 117 165 75
01110110 118 166 76
01110111 119 167 77
01111000 120 170 78
01111001 121 171 79
01111010 122 172 7A
01111011 123 173 7B
01111100 124 174 7C
01111101 125 175 7D
01111110 126 176 7E
01111111 127 177 7F
10000000 128 200 80
10000001 129 201 81
10000010 130 202 82
10000011 131 203 83
10000100 132 204 84
10000101 133 205 85
10000110 134 206 86
10000111 135 207 87
10001000 136 210 88
10001001 137 211 89
10001010 138 212 8A
10001011 139 213 8B

10001100 140 214 8C
10001101 141 215 8D
10001110 142 216 8E
10001111 143 217 8F
10010000 144 220 90
10010001 145 221 91
10010010 146 222 92
10010011 147 223 93
10010100 148 224 94
10010101 149 225 95
10010110 150 226 96
10010111 151 227 97
10011000 152 230 98
10011001 153 231 99
10011010 154 232 9A
10011011 155 233 9B
10011100 156 234 9C
10011101 157 235 9D
10011110 158 236 9E
10011111 159 237 9F
10100000 160 240 A0
10100001 161 241 A1
10000010 162 242 A2
10000011 163 243 A3
10000100 164 244 A4
10000101 165 245 A5
10000110 166 246 A6
10000111 167 247 A7
10001000 168 250 A8
10001001 169 251 A9
10001010 170 252 AA
10001011 171 253 AB
10001100 172 254 AC
10001101 173 255 AD
10001110 174 256 AE
10001111 175 257 AF
10010000 176 260 B0
10010001 177 261 B1
10010010 178 262 B2
10010011 179 263 B3
10010100 180 264 B4
10010101 181 265 B5
10010110 182 266 B6
10110111 183 267 B7
10111000 184 270 B8
10111001 185 271 B9
10111010 186 272 BA
10111011 187 273 BB
10111100 188 274 BC
10111101 189 275 BD

10111110 190 276 BE
10111111 191 277 BF
11000000 192 300 C0
11000001 193 301 C1
11000010 194 302 C2
11000011 195 303 C3
11000100 196 304 C4
11000101 197 305 C5
11000110 198 306 C6
11000111 199 307 C7
11001000 200 310 C8
11001001 201 311 C9
11001010 202 312 CA
11001011 203 313 CB
11001100 204 314 CC
11001101 205 315 CD
11001110 206 316 CE
11001111 207 317 CF
11010000 208 320 D0
11010001 209 321 D1
11010010 210 322 D2
11010011 211 323 D3
11010100 212 324 D4
11010101 213 325 D5
11010110 214 326 D6
11010111 215 327 D7
11011000 216 330 D8
11011001 217 331 D9
11011010 218 332 DA
11011011 219 333 DB
11011100 220 334 DC
11011101 221 335 DD
11011110 222 336 DE
11011111 223 337 DF
11100000 224 340 E0
11100001 225 341 E1
11100010 226 342 E2
11100011 227 343 E3
11100100 228 344 E4
11100101 229 345 E5
11100110 230 346 E6
11100111 231 347 E7
11101000 232 350 E8
11101001 233 351 E9
11101010 234 352 EA
11101011 235 353 EB
11101100 236 354 EC
11101101 237 355 ED
11101110 238 356 EE
11101111 239 357 EF

11110000 240 360 F0
11110001 241 361 F1
11110010 242 362 F2
11110011 243 363 F3
11110100 244 364 F4
11110101 245 365 F5
11110110 246 366 F6
11110111 247 367 F7
11111000 248 370 F8
11111001 249 371 F9
11111010 250 372 FA
11111011 251 373 FB
11111100 252 374 FC
11111101 253 375 FD
11111110 254 376 FE
11111111 255 377 FF

APPENDIX D

Two's Complement/Decimal Table

2's Compl	Value
11111111	**-1**
11111110	**-2**
11111101	**-3**
11111100	**-4**
11111011	**-5**
11111010	**-6**
11111001	**-7**
11111000	**-8**
11110111	**-9**
11110110	**-10**
11110101	**-11**
11110100	**-12**
11110011	**-13**
11110010	**-14**
11110001	**-15**
11110000	**-16**
11101111	**-17**
11101110	**-18**
11101101	**-19**
11101100	**-20**
11101011	**-21**
11101010	**-22**
11101001	**-23**
11101000	**-24**
11100111	**-25**
11100110	**-26**
11100101	**-27**
11100100	**-28**
11100011	**-29**
11100010	**-30**
11100001	**-31**
11100000	**-32**
11011111	**-33**
11011110	**-34**
11011101	**-35**
11011100	**-36**
11011011	**-37**
11011010	**-38**
11011001	**-39**
11011000	**-40**

11010111 -41
11010110 -42
11010101 -43
11010100 -44
11010011 -45
11010010 -46
11010001 -47
11010000 -48
11001111 -49
11001110 -50
11001101 -51
11001100 -52
11001011 -53
11001010 -54
11001001 -55
11001000 -56
11000111 -57
11000110 -58
11000101 -59
11000100 -60
11000011 -61
11000010 -62
11000001 -63
11000000 -64
10111111 -65
10111110 -66
10111101 -67
10111100 -68
10111011 -69
10111010 -70
10111001 -71
10111000 -72
10110111 -73
10010110 -74
10010101 -75
10010100 -76
10010011 -77
10010010 -78
10010001 -79
10010000 -80
10001111 -81
10001110 -82
10001101 -83
10001100 -84
10001011 -85
10001010 -86
10001001 -87
10001000 -88
10000111 -89
10000110 -90

10000101 -91
10000100 -92
10000011 -93
10000010 -94
10100001 -95
10100000 -96
10011111 -97
10011110 -98
10011101 -99
10011100 -100
10011011 -101
10011010 -102
10011001 -103
10011000 -104
10010111 -105
10010110 -106
10010101 -107
10010100 -108
10010011 -109
10010010 -110
10010001 -111
10010000 -112
10001111 -113
10001110 -114
10001101 -115
10001100 -116
10001011 -117
10001010 -118
10001001 -119
10001000 -120
10000111 -121
10000110 -122
10000101 -123
10000100 -124
10000011 -125
10000010 -126
10000001 -127
10000000 -128

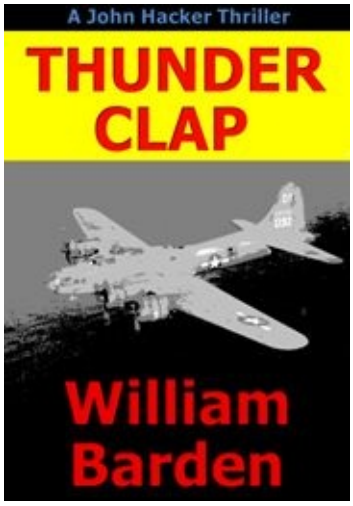
Other Recent Books by William Barden

● Make Money from your Garage and Attic with eBay and Craig's List – Amazon.com eBook

You probably have thousands of dollars in your garage and attic. Discover how you can use the combination of eBay and Craig's List to sell your items rapidly, at top dollar, and without problems in dealing with buyers.

● Thunderclap – Amazon.com eBook Novel

A missing World War II B-17 bomber is headed for Chicago on a devious mission – it's an exciting action novel that takes place in World War II and present day. A master computer hacker discovers the dark secret of a Midwest lake and unravels a complex historical saga that takes him to Japan to battle Yakuza and neo-militarists.



● **The Practical Japan Travel Guide – Amazon.com eBook – Everything You Need Know for a Great Trip**

This is a large guidebook with practical advice on travelling to and around Japan with itineraries. It is fully illustrated with over 350 full-color photographs.